



Vivekanand Education Society's

Institute of Technology

An Autonomous Institute Affiliated to University of Mumbai

Hashu Advani Memorial Complex, Collector Colony, Chembur East, Mumbai - 400074.

Department of Information Technology

CERTIFICATE

This is to certify that **Vedang V. Wajge** of **D20A** semester **VIII**, have successfully completed necessary experiments in the **ITC801: Blockchain & DLT Lab** under my supervision in **VES Institute of Technology** during the academic year **2025-2026**.

Subject Teacher

Name: Mrs. Kajal Joseph

Signature:

Head of Department

Name: Dr. Mrs. Shalu Chopra

Signature:

Lab Teacher

Name: Kajal Joseph

Signature:



**VIVEKANAND EDUCATION SOCIETY'S
INSTITUTE OF TECHNOLOGY
Department of Information Technology
Academic Year: 2025-26**

Lab Code	Lab Name	Credit
ITC801	Blockchain and DLT Lab	1

Programme Outcomes: The graduate will be able to:

PO1	Engineering Knowledge: Apply knowledge of mathematics, natural science, computing, engineering fundamentals and an engineering specialization as specified in WK1 to WK4 respectively to develop to the solution of complex engineering problems.
PO2	Problem Analysis: Identify, formulate, review research literature and analyze complex engineering problems reaching substantiated conclusions with consideration for sustainable development. (WK1 to WK4)
PO3	Design/Development of Solutions: Design creative solutions for complex engineering problems and design / develop systems /components / processes to meet identified needs with consideration for the public health and safety, whole-life cost, net zero carbon, culture, society and environment as required. (WK5)
PO4	PO4: Conduct Investigations of Complex Problems: Conduct investigations of complex engineering problems using research-based knowledge including design of experiments, modelling, analysis & interpretation of data to provide valid conclusions. (WK8).
PO5	Engineering Tool Usage: Create, select and apply appropriate techniques, resources and modern engineering & IT tools, including prediction and modelling recognizing their limitations to solve complex engineering problems. (WK2 and WK6)
PO6	The Engineer and The World: Analyze and evaluate societal and environmental aspects while solving complex engineering problems for its impact on sustainability with reference to economy, health, safety, legal framework, culture and environment. (WK1, WK5, and WK7).
PO7	Ethics: Apply ethical principles and commit to professional ethics, human values, diversity and inclusion; adhere to national & international laws. (WK9)
PO8	Individual and Collaborative Team work: Function effectively as an individual, and as a member or leader in diverse/multi-disciplinary teams.
PO9	Communication: Communicate effectively and inclusively within the engineering community and society at large, such as being able to comprehend and write effective reports and design documentation, make effective presentations considering cultural, language, and learning differences



**VIVEKANAND EDUCATION SOCIETY'S
INSTITUTE OF TECHNOLOGY**
Department of Information Technology
Academic Year: 2025-26

PO10	Project Management and Finance: Apply knowledge and understanding of engineering management principles and economic decision-making and apply these to one's own work, as a member and leader in a team, and to manage projects and in multidisciplinary environments.
PO11	Life-Long Learning: Recognize the need for, and have the preparation and ability for i) independent and life-long learning ii) adaptability to new and emerging technologies and iii) critical thinking in the broadest context of technological change.
PSO 1	Computing & Emerging Technology: Apply core computing and modern technologies, such as AI, Cloud Computing, IoT, cybersecurity, and data-driven tools, to build efficient and innovative solutions.
PSO 2	Professionalism & Innovation : Demonstrate ethics, teamwork, and an entrepreneurial mindset to deliver sustainable solutions for society.

Course Outcomes: Student will be able	
ITC801.1	Describe the basic concept of Blockchain and Distributed Ledger Technology.
ITC801.2	Interpret the knowledge of the Bitcoin network, nodes, keys, wallets and transactions
ITC801.3	Implement smart contracts in Ethereum using different development frameworks.
ITC801.4	Develop applications in permissioned Hyperledger Fabric network.
ITC801.5	Interpret different Crypto assets and Crypto currencies
ITC801.6	The use of Blockchain with AI, IoT and Cyber Security using case studies.

ITC801 Blockchain and DLT Lab													
CO	PO											PSO	
	PO 1	PO 2	PO 3	PO 4	PO 5	PO 6	PO 7	PO 8	PO 9	PO 10	PO 11	PSO 1	PSO 2
ITC801.1	2	3	3	3	3	3	3	3	3	2	2	3	3
ITC801.2	2	3	3	3	3	3	3	3	3	2	2	3	3
ITC801.3	2	3	3	3	3	3	3	3	3	2	2	3	3
ITC801.4	2	3	3	3	3	3	3	3	3	2	2	3	3
ITC801.5	2	3	3	3	3	3	3	3	3	2	2	3	3
ITC801.6	2	3	3	3	3	3	3	3	3	2	2	3	3
Avg PO AL	2	3	3	3	3	3	3	3	3	2	2	3	3



**VIVEKANAND EDUCATION SOCIETY'S
INSTITUTE OF TECHNOLOGY**
Department of Information Technology
Academic Year: 2025-26

ITC801 Blockchain and DLT Lab												
CO	Experiment											
	Exp 1	Exp 2	Exp 3	Exp 4	Exp 5	Exp 6	Exp 7	Exp 8	Exp 9	Exp 10	Exp 11	Exp 12
ITC801.1	3	3	2	-	2	2	2	-	-	3	-	3
ITC801.2	2	3	3	-	2	2	3	2	-	3	-	3
ITC801.3	-	-	2	3	3	3	3	3	2	-	-	3
ITC801.4	-	-	-	-	-	-	-	-	-	-	3	3
ITC801.5	2	3	3	-	-	2	2	2	-	3	-	3
ITC801.6	-	-	-	-	-	-	-	-	-	2	-	3

INDEX

Sr. No.	List of Experiments
1	Write a Python program to understand SHA and Cryptography in Blockchain, Merkle root tree hash
2	Create a Blockchain using Python
3	Create a Cryptocurrency using Python and perform mining in the Blockchain created.
4	Hands-on Solidity Programming Assignments for creating Smart Contracts
5	Deploying a Voting/Ballot Smart Contract
6	Creating Smart Contracts and performing transactions using Solidity and Remix IDE
7	Implement the embedding wallet (Metamask) and transaction using Solidity
8	To implement the Blockchain platform Ganache and deploy a simple Solidity smart contract using Metamask and Remix IDE.



**VIVEKANAND EDUCATION SOCIETY'S
INSTITUTE OF TECHNOLOGY**
Department of Information Technology
Academic Year: 2025-26

9	To implement a Private Ethereum Blockchain using Geth.
10	Explore the Cryptocurrency Landscape
11	Implementation of Hyperledger Fabric Network and Asset Management using Chaincode
12	Mini Project
12	Assignment-1
13	Assignment-2
14	Assignment-3
15	Mock Viva

Subject teacher



**VIVEKANAND EDUCATION SOCIETY'S
INSTITUTE OF TECHNOLOGY
Department of Information Technology
Academic Year: 2025-26**

ITC801 : Blockchain & DLT Lab
Experiment No. 01

Aim: Write a Python program to understand SHA and Cryptography in Blockchain, Merkle root tree hash

Roll No.	07
Name	Vedang V. Wajge
Class	D20A
Subject	ITC801: Blockchain and DLT Lab
CO Mapped	CO1: Describe the basic concept of Blockchain and Distributed Ledger Technology. CO2: Interpret the knowledge of the Bitcoin network, nodes, keys, wallets and transactions CO5: Interpret different Crypto assets and Crypto currencies

Experiment No 1

AIM: Cryptography in Blockchain, Merkle root Tree Hash

Theory:

1) Cryptographic Hash Functions in Blockchain:

A cryptographic hash function converts input data of any size into a fixed-length string called a hash.

Key Properties:

- Deterministic – Same input always produces the same hash
- Fast computation – Hashes are generated quickly
- Pre-image resistance – Original data cannot be derived from the hash
- Collision resistance – Two different inputs cannot produce the same hash
- Avalanche effect – Small input change causes a large hash change

Role in Blockchain:

- Secures block data
- Links blocks together
- Enables Proof-of-Work mining
- Ensures data integrity and immutability

2) What is a Merkle Tree?

A Merkle Tree (also called a Hash Tree) is a data structure that organizes transaction data using cryptographic hashes.

- Leaf nodes contain hashes of individual transactions
- Parent nodes contain hashes of their child nodes
- The top hash is called the Merkle Root

The Merkle Root represents all transactions in a block.

What is a Cryptographic Puzzle & the Golden Nonce?

A cryptographic puzzle is a challenge miners must solve to add a new block to the blockchain.

Puzzle Goal:

Find a nonce value such that:

$\text{Hash}(\text{Block Data} + \text{Nonce}) < \text{Target}$

(or starts with a certain number of leading zeros)

Golden Nonce:

- The Golden Nonce is the first nonce that produces a valid hash meeting the difficulty requirement
- It proves computational work has been done
- Once found, the block is accepted by the network

This process is known as Proof-of-Work (PoW).

3) How Does a Merkle Tree Work?

1. Each transaction is hashed
2. Hashes are paired and concatenated
3. Each pair is hashed again to form parent nodes
4. If a level has an odd number of hashes, the last hash is duplicated
5. The process repeats until one hash remains → Merkle Root

Any change in a transaction changes the Merkle Root

4) Benefits of Merkle Tree

- Efficient verification of transactions
- Reduced storage requirements
- High security through hashing
- Quick detection of tampering
- Supports lightweight (SPV) clients

5) Use Cases of Merkle Tree

- Blockchain transaction verification
- Bitcoin & Ethereum block structure
- Data integrity verification
- Distributed file systems
- Git version control system
- Peer-to-peer networks
- Database synchronization

Tasks Performed:**1. Hash Generation using SHA-256:**

- o Developed a Python program to compute a SHA-256 hash for any given input string using the `hashlib` library.

```
import hashlib

def generate_sha256_hash(input_string):

    encoded_string = input_string.encode('utf-8')

    hash_object = hashlib.sha256(encoded_string)

    return hash_object.hexdigest()

# Example usage
text = input("Enter a string: ")
print("SHA-256 Hash:", generate_sha256_hash(text))
```

Output:

```
Enter a string: Vedang
SHA-256 Hash: 31ef00ea9ae265c4681883ed25f84690ff34e160b1a032eb448c3055997ec446
```

2. Target Hash Generation with Nonce:

- o Created a program to generate a hash code by concatenating a user input string and a nonce value to simulate the mining process.

Code:

```
import hashlib

def hash_with_nonce(input_string, nonce):
    combined = f"{input_string}{nonce}"
    return hashlib.sha256(combined.encode('utf-8')).hexdigest()

# Example usage
data = input("Enter data: ")
nonce = int(input("Enter nonce: "))
```

```
print("Generated Hash:", hash_with_nonce(data, nonce))
```

Output:

```
Enter data: Vedang
Enter nonce: 2
Generated Hash: d06c523fbc36d280af6ca8ad1d2d525443007c66b05a9a336c6086be35a4f9fa
```

3. Proof-of-Work Puzzle Solving:

- o Implemented a program to find the nonce that, when combined with a given input string, produces a hash starting with a specified number of leading zeros.

Code:

```
import hashlib

def proof_of_work(input_string, difficulty):
    nonce = 0
    target = '0' * difficulty

    while True:
        combined = f"{input_string}{nonce}"
        hash_result = hashlib.sha256(combined.encode('utf-8')).hexdigest()

        if hash_result.startswith(target):
            return nonce, hash_result

        nonce += 1

# Example usage
data = input("Enter data: ")
difficulty = int(input("Enter difficulty (number of leading zeros): "))

nonce, hash_result = proof_of_work(data, difficulty)
print("Nonce found:", nonce)
print("Hash:", hash_result)
```

Output:

```
Enter data: Wajge
Enter difficulty (number of leading zeros): 2
Nonce found: 44
Hash: 0008ef5674b632a97ad0eb0093fb1ed03d952b9eb5586638648b2abccc7440be
```

4. Merkle Tree Construction:

- o Built a Merkle Tree from a list of transactions by recursively hashing pairs of transaction hashes, doubling up last nodes if needed, and generated the Merkle Root hash for blockchain transaction integrity.

Code:

```
import hashlib

def sha256(data):
    return hashlib.sha256(data.encode('utf-8')).hexdigest()

def merkle_root(transactions):

    hashes = [sha256(tx) for tx in transactions]

    while len(hashes) > 1:
        if len(hashes) % 2 != 0:
            hashes.append(hashes[-1])
        new_level = []
        for i in range(0, len(hashes), 2):
            combined_hash = sha256(hashes[i] + hashes[i + 1])
            new_level.append(combined_hash)

        hashes = new_level

    return hashes[0]

# Example usage
transactions = [
    "Vedang pays Diksha 10 BTC",
    "Aditi pays Siddhant 5 BTC",
    "Rutuja pays Shruti 2 BTC",
    "Dave pays Eve 1 BTC"
]
```

```
print("Merkle Root:", merkle_root(transactions))
```

Output:

```
Merkle Root: 9974430a5836b3816d28c8fefca2c324e2ca009693b1b590e43eafa56b89b14d
```

Conclusion:

This study highlights the basic cryptographic principles behind blockchain technology. SHA-256 hashing ensures data integrity by generating unique hashes, while nonce and target hashing demonstrate the proof-of-work process that makes block mining computationally difficult. Merkle Trees efficiently verify large numbers of transactions using a single root hash. Together, these techniques show how cryptography secures blockchain systems by ensuring data integrity, immutability, and efficient decentralized verification.



**VIVEKANAND EDUCATION SOCIETY'S
INSTITUTE OF TECHNOLOGY
Department of Information Technology
Academic Year: 2025-26**

ITC801: Blockchain & DLT Lab
Experiment 02

Aim: Create a Blockchain using Python

Roll No.	07
Name	Vedang V. Wajge
Class	D20A
Subject	ITC801: Blockchain and DLT Lab
CO Mapped	CO1: Describe the basic concept of Blockchain and Distributed Ledger Technology. CO2: Interpret the knowledge of the Bitcoin network, nodes, keys, wallets and transactions CO5: Interpret different Crypto assets and Crypto currencies

Experiment No 2

Aim: Create a Blockchain using Python

Theory:

Blockchain is a distributed and immutable ledger that records data in a sequence of linked blocks. Each block contains a timestamp, proof (from Proof-of-Work), and the hash of the previous block, ensuring integrity and security.

In this program:

1. **Block Creation** – A block is generated using `create_block()` with a proof and previous hash.
2. **Proof-of-Work (PoW)** – Implemented to mine a block by solving a cryptographic puzzle (finding a hash with leading zeros).
3. **Hashing** – SHA-256 algorithm ensures data immutability.
4. **Validation** – `is_chain_valid()` checks that every block correctly references the previous hash and satisfies PoW conditions.
5. **Flask Web API** – Routes `/mine_block`, `/get_chain`, and `/is_valid` allow mining, fetching the chain, and verifying blockchain integrity through a browser or API client.

This program runs on a local server and simulates mining a blockchain without a network of nodes, making it an ideal introductory model to understand blockchain basics.

1) What is a Blockchain?

A **Blockchain** is a **distributed and decentralized digital ledger** that stores data in the form of **blocks**.

Each block contains data, a timestamp, and the hash of the previous block, which links the blocks together in a chain.

Once data is added to the blockchain, it **cannot be easily altered**, ensuring **security, transparency, and immutability**.

2) Process of Mining

Mining is the process of **adding a new block** to the blockchain.

Steps involved in mining:

1. The miner collects transaction data.
2. A **nonce** is generated and combined with block data.
3. The miner computes a **hash using SHA-256**.
4. This process is repeated until the hash satisfies the **difficulty condition** (e.g., starts with leading zeros).
5. Once found, the block is added to the blockchain and broadcast to the network.

Mining ensures **security and consensus** in the blockchain.

3) How to Check the Validity of Blocks in a Blockchain

The validity of a blockchain is checked by verifying each block using the following conditions:

1. The **previous hash** stored in a block must match the **actual hash of the previous block**.
2. The **Proof-of-Work** must be valid (hash satisfies the difficulty condition).
3. Any change in block data changes the hash, making tampering easily detectable.

If all blocks satisfy these conditions, the blockchain is considered **valid and secure**.

Code:

```
# Module 1 - Create a Blockchain (VS Code Compatible)
# Author: Vedang

import datetime
import hashlib
import json
from flask import Flask, jsonify

# PART 1: BLOCKCHAIN STRUCTURE

class Blockchain:
    def __init__(self):
        self.chain = []
        self.create_block(proof=1, previous_hash='0') # Genesis block

    def create_block(self, proof, previous_hash):
        block = {
            'index': len(self.chain) + 1,
            'timestamp': str(datetime.datetime.now()),
            'proof': proof,
            'previous_hash': previous_hash
        }
        self.chain.append(block)
        return block

    def get_previous_block(self):
        return self.chain[-1]

# PART 2: PROOF OF WORK
def proof_of_work(self, previous_proof):
    new_proof = 1
    while True:
        hash_operation = hashlib.sha256(
            str(new_proof**2 - previous_proof**2).encode()
        ).hexdigest()
        if hash_operation[:4] == '0000':
            return new_proof
        new_proof += 1

# PART 3: HASHING
def hash(self, block):
    encoded_block = json.dumps(block, sort_keys=True).encode()
    return hashlib.sha256(encoded_block).hexdigest()

# PART 4: VALIDATION
def is_chain_valid(self, chain):
    previous_block = chain[0]
    index = 1
    while index < len(chain):
        block = chain[index]
```

```
        if block['previous_hash'] != self.hash(previous_block):
            return False

        hash_operation = hashlib.sha256(
            str(block['proof']**2 - previous_block['proof']**2).encode()
        ).hexdigest()

        if hash_operation[:4] != '0000':
            return False

        previous_block = block
        index += 1

    return True

# PART 5: FLASK APP

app = Flask(__name__)
blockchain = Blockchain()

@app.route('/', methods=['GET'])
def home():
    return "Welcome to Vedang's Blockchain API!"

@app.route('/mine_block', methods=['GET'])
def mine_block():
    previous_block = blockchain.get_previous_block()
    proof = blockchain.proof_of_work(previous_block['proof'])
    previous_hash = blockchain.hash(previous_block)
    block = blockchain.create_block(proof, previous_hash)

    return jsonify({
        'message': 'Block mined successfully',
        'block': block,
        'created_by': 'Aditi'
    }), 200

@app.route('/get_chain', methods=['GET'])
def get_chain():
    return jsonify({
        'chain': blockchain.chain,
        'length': len(blockchain.chain)
    }), 200

@app.route('/is_valid', methods=['GET'])
def is_valid():
    return jsonify({
        'valid': blockchain.is_chain_valid(blockchain.chain)
    }), 200

# PART 6: RUN SERVER
if __name__ == "__main__":
    app.run(port=5000, debug=True)
```

Output:

```
127.0.0.1:5000/mine_block

{
  "block": {
    "index": 2,
    "previous_hash": "758f660d7fcfa2d5431ed2be760ff76eb78c195f8013cd918555fab6acce2b4b",
    "proof": 533,
    "timestamp": "2026-02-03 21:01:11.509943"
  },
  "created_by": "Aditi",
  "message": "Block mined successfully"
}
```

```
127.0.0.1:5000/get_chain

{
  "chain": [
    {
      "index": 1,
      "previous_hash": "0",
      "proof": 1,
      "timestamp": "2026-02-03 20:59:18.386505"
    },
    {
      "index": 2,
      "previous_hash": "758f660d7fcfa2d5431ed2be760ff76eb78c195f8013cd918555fab6acce2b4b",
      "proof": 533,
      "timestamp": "2026-02-03 21:01:11.509943"
    }
  ],
  "length": 2
}
```

```
127.0.0.1:5000/is_valid

{
  "valid": true
}
```

Conclusion:

In this experiment, a basic blockchain was successfully implemented using Python and Flask. The system supports block mining, viewing the blockchain, and validating its integrity. The **Proof-of-Work mechanism** adds security by requiring computational effort, while **cryptographic hashing** ensures that data remains immutable. This implementation clearly demonstrates the fundamental principles of blockchain such as security, immutability, and consensus, and serves as a foundation for understanding more advanced blockchain applications.



**VIVEKANAND EDUCATION SOCIETY'S
INSTITUTE OF TECHNOLOGY
Department of Information Technology
Academic Year: 2025-26**

ITC801 : Blockchain & DLT Lab
Experiment 03

Aim: Create a Cryptocurrency using Python and perform mining in the Blockchain created.

Roll No.	07
Name	Vedang V. Wajge
Class	D20A
Subject	ITC801: Blockchain and DLT Lab
CO Mapped	CO1: Describe the basic concept of Blockchain and Distributed Ledger Technology. CO2: Interpret the knowledge of the Bitcoin network, nodes, keys, wallets and transactions CO3: Implement smart contracts in Ethereum using different development frameworks. CO5: Interpret different Crypto assets and Crypto currencies

Blockchain Exp - 3

Aim: Create a Cryptocurrency using Python and perform mining in the Blockchain created.

Theory:

1. Blockchain Overview

Blockchain is a **distributed and decentralized ledger** that stores information in a series of linked blocks.

Each block contains:

- Transaction data
- Timestamp
- Previous block's hash
- Its own unique hash (digital fingerprint)

Once data is recorded in a blockchain, it becomes **immutable** because altering one block would require recalculating all subsequent blocks.

2. Mining

Mining is the process of:

1. Collecting pending transactions into a block.
2. Performing a computational puzzle (Proof-of-Work) to find a valid hash.
3. Adding the new block to the blockchain.
Broadcasting it to all connected peers.

Miners are rewarded with cryptocurrency for successfully mining a block.

3. Multi-Node Blockchain Network

In this lab, we simulate **three independent blockchain nodes** (5001, 5002, 5003).

Each node:

- Runs on a separate port.
- Maintains its own copy of the blockchain.
- Can connect with peers to share and validate blocks.

4. Consensus Mechanism

We use the **Longest Chain Rule**:

- If multiple versions of the chain exist, the **longest valid chain** is chosen.
- This ensures all nodes agree on a single transaction history.

5. Transactions & Mining Reward

Each transaction has:

- Sender
- Receiver
- Amount

When mining a block:

- Pending transactions are added to the block.
- A **reward transaction** is added automatically to pay the miner.

6. Chain Replacement

When /replace_chain is called:

1. Node requests chains from peers.
2. If it finds a longer and valid chain, it replaces its own.
3. This keeps the blockchain consistent across all nodes.

Tools & Libraries Used

- **Python 3.x**
- **Flask** – Web framework for API endpoints
pip install Flask
- **Requests** – For HTTP communication between nodes
pip install requests==2.18.4
- **Postman** – For testing API requests
- Python Standard Libraries:
 - datetime
 - jsonify
 - hashlib
 - uuid4
 - urlparse
 - request

Code :

```

# Module 2 - Create a Cryptocurrency

# To be installed:
# Flask==0.12.2: pip install Flask==0.12.2
# Postman HTTP Client: https://www.getpostman.com/
# requests==2.18.4: pip install requests==2.18.4

# Importing the
libraries import
datetime
import hashlib
import json
from flask import Flask, jsonify, request
import requests
from uuid import uuid4
hex from urllib.parse import urlparse

# Generate a unique id that is in
# To parse url of the nodes

# Part 1 - Building a Blockchain
class Blockchain:
    def __init__(self):
        self.chain = []
        self.transactions = []
        # Adding transactions before they
        # are added to a block
        self.create_block(proof = 1, previous_hash = '0')
        self.nodes = set()
        # Set is used as there is no order to
        # be maintained as the nodes can be from all around the globe

    def create_block(self, proof, previous_hash):
        block = {'index': len(self.chain) + 1,
                  'timestamp': str(datetime.datetime.now()),
                  'proof': proof,
                  'previous_hash': previous_hash,
                  'transactions': self.transactions}
        # Adding transactions to make
        # the blockchain a cryptocurrency
        self.transactions = []
        # The list of transaction should
        # become empty after they are added to a block
        self.chain.append(block)
        return block

```

```

def get_previous_block(self):
    return self.chain[-1]

def proof_of_work(self, previous_proof):
    new_proof = 1
    check_proof = False
    while check_proof is False:
        hash_operation = hashlib.sha256(str(new_proof**2 -
previous_proof**2).encode()).hexdigest()
        if hash_operation[:4] == '0000':
            check_proof = True
        else:
            new_proof += 1
    return new_proof

def hash(self, block):
    encoded_block = json.dumps(block, sort_keys = True).encode()
    return hashlib.sha256(encoded_block).hexdigest()

def is_chain_valid(self, chain):
    previous_block = chain[0]
    block_index = 1
    while block_index < len(chain):
        block = chain[block_index]
        if block['previous_hash'] != self.hash(previous_block):
            return False
        previous_proof = previous_block['proof']
        proof = block['proof']
        hash_operation = hashlib.sha256(str(proof**2 -
previous_proof**2).encode()).hexdigest()
        if hash_operation[:4] != '0000':
            return False
        previous_block = block
        block_index += 1
    return True

• This method will add the transaction to the list of
transactions
def add_transaction(self, sender, receiver, amount):
    self.transactions.append({'sender': sender,
                              'receiver': receiver,
                              'amount': amount})
    previous_block = self.get_previous_block()

```

```

    return previous_block['index'] + 1                # It will return the block index
to which the transaction should be added

```

- This function will add the node containing an address to the set of nodes created in init function

```

def add_node(self, address):
    parsed_url = urlparse(address)                    # urlparse will parse the url from
the address                                           the address
    self.nodes.add(parsed_url.netloc)                  # Add is used and not append as
it's a set. Netloc will only return '127.0.0.1:5000'

```

- Consensus Protocol. This function will replace all the shorter chain with the longer chain in all the nodes on the network

```

def replace_chain(self):
    network = self.nodes                             # network variable is the set of nodes
all around the globe
    longest_chain = None                             # It will hold the longest chain when
we scan the network
    max_length = len(self.chain)                      # This will hold the length of the
chain held by the node that runs this function
    for node in network:
        response = requests.get(f'http://{node}/get_chain')    # Use get chain
method already created to get the length of the chain
        if response.status_code == 200:
            length = response.json()['length']                # Extract the length of the chain
from get_chain function
            chain = response.json()['chain']
            if length > max_length and self.is_chain_valid(chain):    # We check if the length
is bigger and if the chain is valid then
                max_length = length                                # We update the max length
                longest_chain = chain                                # We update the longest chain
            if longest_chain:                                       # If longest_chain is not none that means
it was replaced
                self.chain = longest_chain                        # Replace the chain of the current
node with the longest chain
                return True
            return False                                           # Return false if current chain is the longest
one

```

Part 2 - Mining our Blockchain

Creating a Web App

```
app = Flask(__name_)

# Creating an address for the node on Port 5000. We will create some other nodes as well
on different ports
node_address = str(uuid4()).replace('-', '') #

# Creating a Blockchain
blockchain = Blockchain()

# Mining a new block
@app.route('/mine_block', methods = ['GET'])
def mine_block():
    previous_block = blockchain.get_previous_block()
    previous_proof = previous_block['proof']
    proof = blockchain.proof_of_work(previous_proof)
    previous_hash = blockchain.hash(previous_block)
    blockchain.add_transaction(sender = node_address, receiver = 'Richard', amount = 1) #
    Hadcoins to mine the block (A Reward). So the node gives 1 hadcoin to Abcde for mining the
    block
    block = blockchain.create_block(proof, previous_hash)
    response = {'message': 'Congratulations, you just mined a block!',
                'index': block['index'],
                'timestamp': block['timestamp'],
                'proof': block['proof'],
                'previous_hash': block['previous_hash'],
                'transactions': block['transactions']}
    return jsonify(response), 200

# Getting the full Blockchain
@app.route('/get_chain', methods = ['GET'])
def get_chain():
    response = {'chain': blockchain.chain,
                'length': len(blockchain.chain)}
    return jsonify(response), 200

# Checking if the Blockchain is valid
@app.route('/is_valid', methods = ['GET'])
def is_valid():
    is_valid = blockchain.is_chain_valid(blockchain.chain)
    if is_valid:
        response = {'message': 'All good. The Blockchain is valid.'}
    else:
```

```

    response = {'message': 'Houston, we have a problem. The Blockchain is not valid.}
    return jsonify(response), 200

```

Adding a new transaction to the Blockchain

```

@app.route('/add_transaction', methods = ['POST'])
def add_transaction():
    json = request.get_json()
    transaction_keys = ['sender', 'receiver', 'amount']
    if not all(key in json for key in transaction_keys):
        return 'Some elements of the transaction are missing', 400
    index = blockchain.add_transaction(json['sender'], json['receiver'], json['amount'])
    response = {'message': f'This transaction will be added to Block {index}'}
    return jsonify(response), 201

```

Post method as we have to pass something to get something in return

This will get the json file from postman. In Postman we will create a json file in which we will pass the values for the keys in the json file

Checking if all keys are available in json

Code 201 for creation

Part 3 - Decentralizing our Blockchain

Connecting new nodes

```

@app.route('/connect_node', methods = ['POST'])
def connect_node():
    json = request.get_json()
    nodes = json.get('nodes')
    file if nodes is None:
        return "No node", 400
    for node in nodes:
        blockchain.add_node(node)
    response = {'message': 'All the nodes are now connected. The Hadcoin Blockchain now contains the following nodes:',
                'total_nodes': list(blockchain.nodes)}
    return jsonify(response), 201

```

POST request to register the new nodes from the json file

Get the nodes from json

Replacing the chain by the longest chain if needed

```

@app.route('/replace_chain', methods = ['GET'])
def replace_chain():
    is_chain_replaced = blockchain.replace_chain()
    if is_chain_replaced:
        response = {'message': 'The nodes had different chains so the chain was replaced by the longest one.'}

```

```
        'new_chain': blockchain.chain}
    else:
        response = {'message': 'All good. The chain is the largest one.',
                    'actual_chain': blockchain.chain}
    return jsonify(response), 200

# Running the app
app.run(host = '0.0.0.0', port = 500*)
```

Output :**1. Connecting nodes: Node 1 -> 2,3**

The screenshot shows the Postman interface for a REST client. The request is a POST to `http://127.0.0.1:5001/connect_node`. The body is raw JSON:

```
{  "nodes": [    "http://127.0.0.1:5002",    "http://127.0.0.1:5003"  ]}
```

. The response is a 201 CREATED status with a response time of 7 ms and a body size of 331 B. The response body is JSON:

```
{  "message": "All the nodes are now connected. The Hadcoin Blockchain now contains the following nodes:",  "total_nodes": [    "127.0.0.1:5003",    "127.0.0.1:5002"  ]}
```

Node 2 -> 1,3

The screenshot shows the Postman interface for a REST client. The request is a POST to `http://127.0.0.1:5002/connect_node`. The body is raw JSON:

```
{  "nodes": [    "http://127.0.0.1:5001",    "http://127.0.0.1:5003"  ]}
```

. The response is a 201 CREATED status with a response time of 6 ms and a body size of 331 B. The response body is JSON:

```
{  "message": "All the nodes are now connected. The Hadcoin Blockchain now contains the following nodes:",  "total_nodes": [    "127.0.0.1:5003",    "127.0.0.1:5001"  ]}
```

Node 3 -> 1,2

The screenshot shows a REST client interface with the following details:

- URL:** `http://127.0.0.1:5003/connect_node`
- Method:** `POST`
- Body (raw):**

```
1 {  
2   "nodes": [  
3     "http://127.0.0.1:5001",  
4     "http://127.0.0.1:5002"  
5   ]  
6 }
```
- Response:** `201 CREATED` (5 ms, 331 B). The response body is:

```
1 {  
2   "message": "All the nodes are now connected. The Hadcoin Blockchain now contains the following  
3     nodes:",  
4   "total_nodes": [  
5     "127.0.0.1:5001",  
6     "127.0.0.1:5002"  
7   ]  
8 }
```

2. Adding transaction from node 1

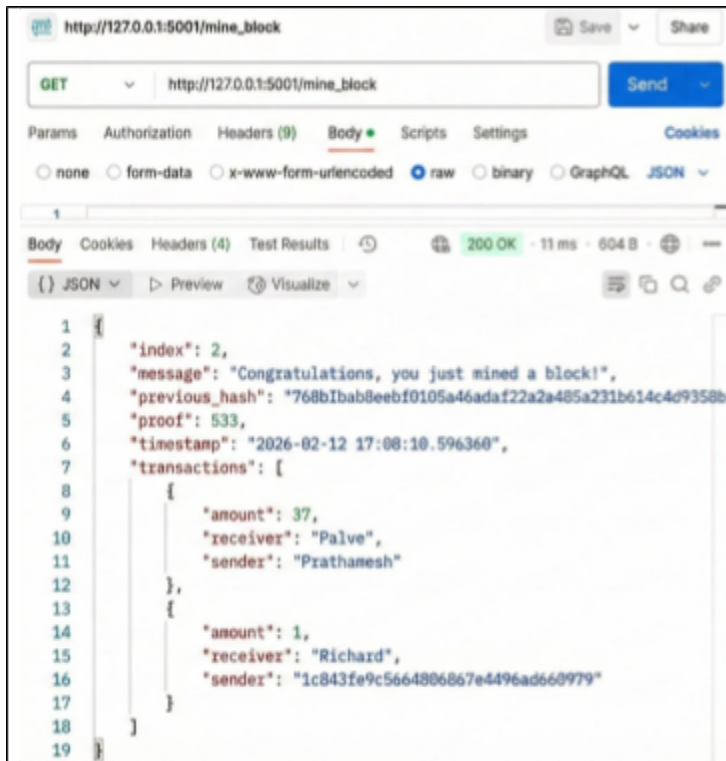
The screenshot shows a REST client interface with the following details:

- URL:** `http://127.0.0.1:5001/add_transaction`
- Method:** `POST`
- Body (raw):**

```
1 {  
2   "sender": "Prathanesh",  
3   "receiver": "Palve",  
4   "amount": 37  
5 }
```
- Response:** `201 CREATED` (6 ms, 213 B). The response body is:

```
1 {  
2   "message": "This transaction will be added to Block 2"  
3 }
```


3. Mining a Block (Only on 5001)

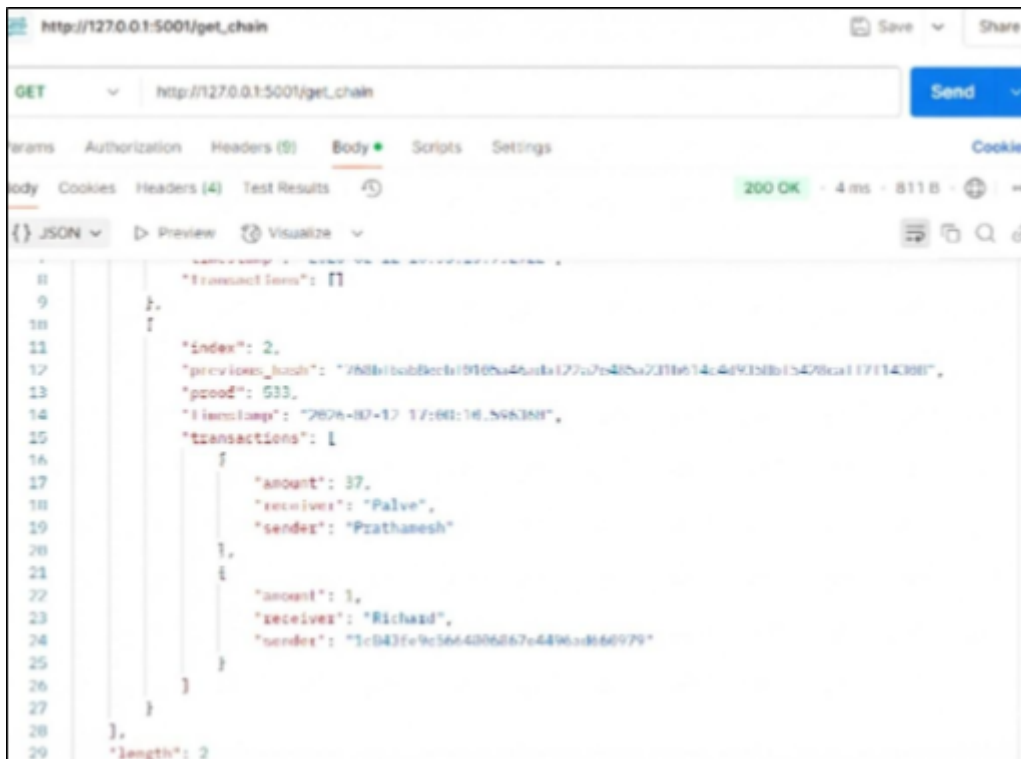


GET http://127.0.0.1:5001/mine_block

200 OK - 11 ms - 604 B

```
{
  "index": 2,
  "message": "Congratulations, you just mined a block!",
  "previous_hash": "768b1bab8eebf0105a46adaf22a2a485a231b614c4d9358b",
  "proof": 533,
  "timestamp": "2026-02-12 17:08:10.596360",
  "transactions": [
    {
      "amount": 37,
      "receiver": "Palve",
      "sender": "Prathamesh"
    },
    {
      "amount": 1,
      "receiver": "Richard",
      "sender": "1c843fe9c5664806867e4496ad660979"
    }
  ]
}
```

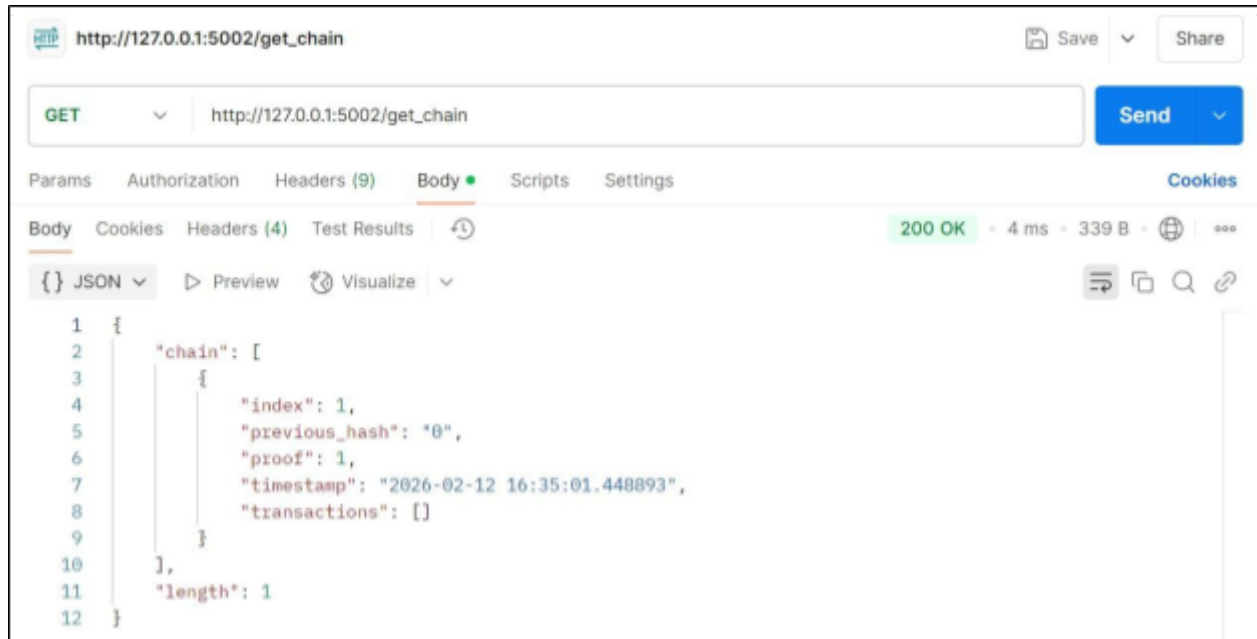
4. Checking chain length difference: Node 1



GET http://127.0.0.1:5001/get_chain

200 OK - 4 ms - 811 B

```
{
  "transactions": [
    {
      "amount": 37,
      "receiver": "Palve",
      "sender": "Prathamesh"
    },
    {
      "amount": 1,
      "receiver": "Richard",
      "sender": "1c843fe9c5664806867e4496ad660979"
    }
  ],
  "index": 2,
  "previous_hash": "768b1bab8eebf0105a46adaf22a2a485a231b614c4d9358b",
  "proof": 533,
  "timestamp": "2026-02-12 17:08:10.596360",
  "length": 2
}
```

Node 2

http://127.0.0.1:5002/get_chain

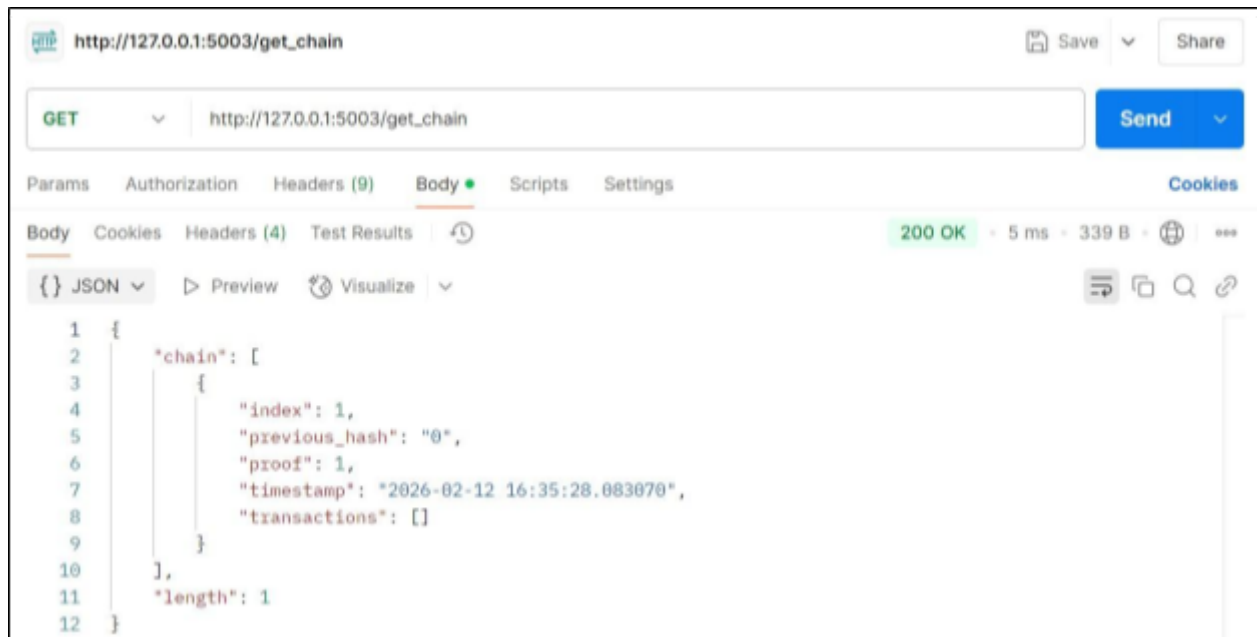
GET http://127.0.0.1:5002/get_chain Send

Params Authorization Headers (9) **Body** Scripts Settings Cookies

Body Cookies Headers (4) Test Results 200 OK · 4 ms · 339 B

{ } JSON Preview Visualize

```
1 {
2   "chain": [
3     {
4       "index": 1,
5       "previous_hash": "0",
6       "proof": 1,
7       "timestamp": "2026-02-12 16:35:01.448893",
8       "transactions": []
9     }
10  ],
11  "length": 1
12 }
```

Node 3

http://127.0.0.1:5003/get_chain

GET http://127.0.0.1:5003/get_chain Send

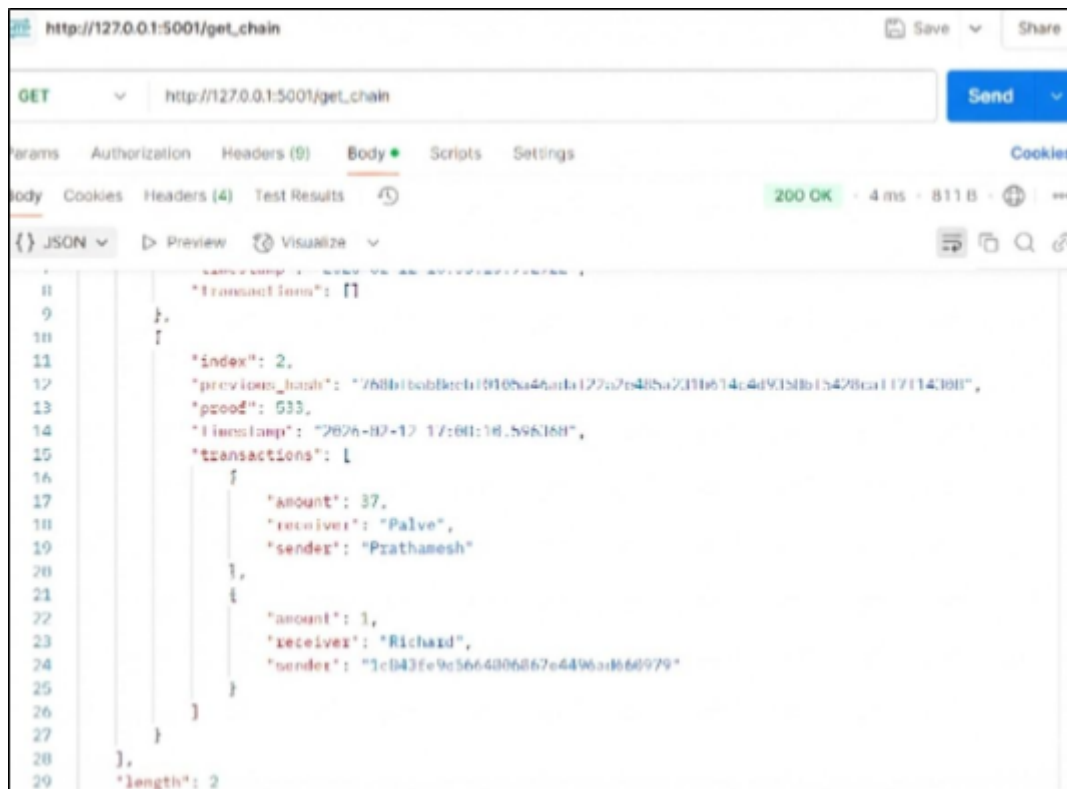
Params Authorization Headers (9) **Body** Scripts Settings Cookies

Body Cookies Headers (4) Test Results 200 OK · 5 ms · 339 B

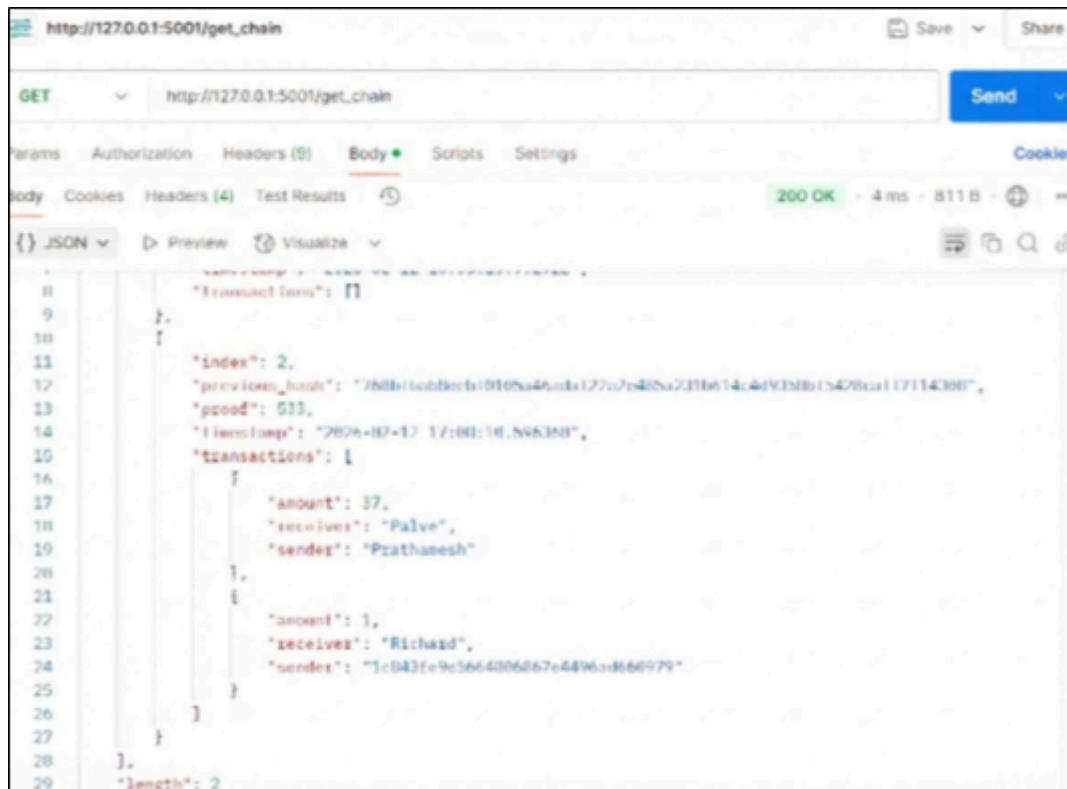
{ } JSON Preview Visualize

```
1 {
2   "chain": [
3     {
4       "index": 1,
5       "previous_hash": "0",
6       "proof": 1,
7       "timestamp": "2026-02-12 16:35:28.083070",
8       "transactions": []
9     }
10  ],
11  "length": 1
12 }
```

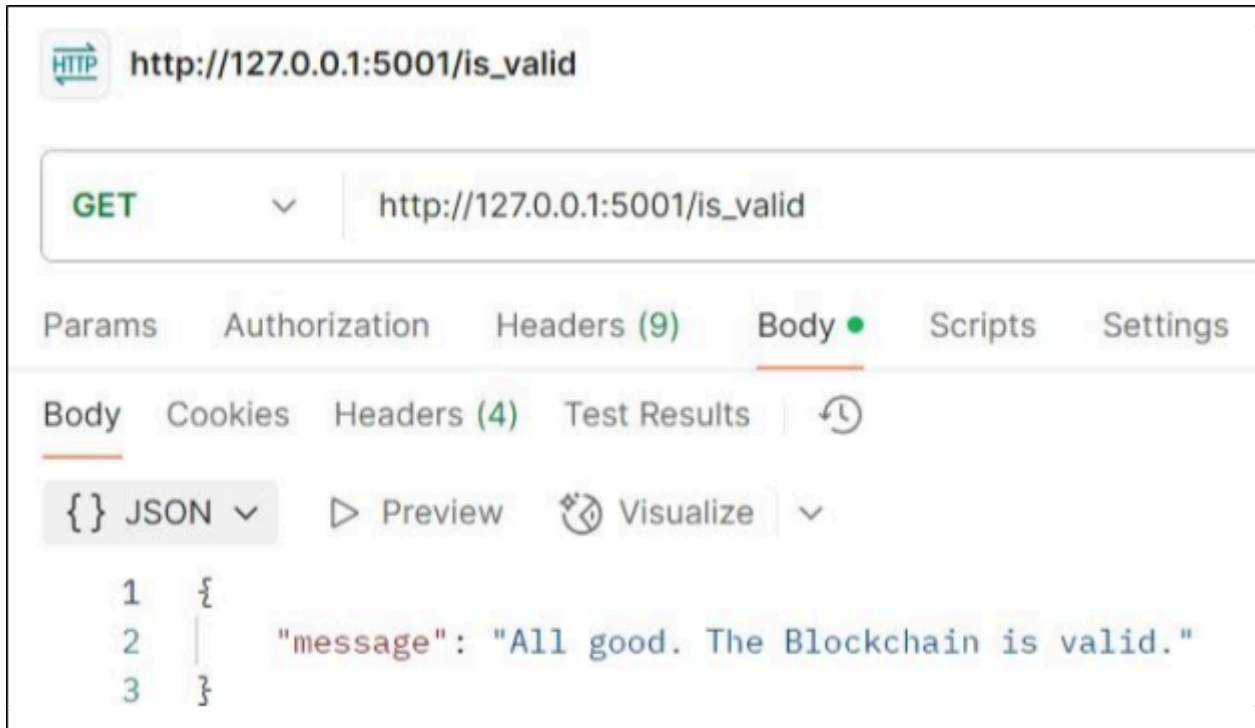
5. Replace Chain (Consensus Mechanism) : Node 2



Node 3



6. Final Validation



HTTP http://127.0.0.1:5001/is_valid

GET http://127.0.0.1:5001/is_valid

Params Authorization Headers (9) **Body** Scripts Settings

Body Cookies Headers (4) Test Results ↻

{ } JSON Preview Visualize

```

1  {
2  |   "message": "All good. The Blockchain is valid."
3  }
```

7. Terminal Activity



```

(venv) PS C:\Users\prajj\Downloads\Lab_3_Create a Cryptocurrency-20260211T182232Z-1-001\Lab_3_Create a Cryptocurrency> p
ython hadcoin_node_5001.py
>>
File "C:\Users\prajj\Downloads\Lab_3_Create a Cryptocurrency-20260211T182232Z-1-001\Lab_3_Create a Cryptocurrency\hadc
oin_node_5001.py", line 164, in connect_node
    nodes = json.get('nodes')
AttributeError: 'NoneType' object has no attribute 'get'
# Get the nodes from json file
127.0.0.1 - - [12/Feb/2026 16:50:52] "POST /connect_node HTTP/1.1" 500 -
127.0.0.1 - - [12/Feb/2026 16:54:03] "POST /connect_node HTTP/1.1" 201 -
127.0.0.1 - - [12/Feb/2026 17:06:56] "POST /add_transaction HTTP/1.1" 201 -
127.0.0.1 - - [12/Feb/2026 17:07:55] "POST /mine_block HTTP/1.1" 405 -
127.0.0.1 - - [12/Feb/2026 17:08:10] "GET /mine_block HTTP/1.1" 200 -
127.0.0.1 - - [12/Feb/2026 17:11:22] "GET /get_chain HTTP/1.1" 200 -
127.0.0.1 - - [12/Feb/2026 17:15:11] "GET /get_chain HTTP/1.1" 200 -
127.0.0.1 - - [12/Feb/2026 17:16:11] "GET /get_chain HTTP/1.1" 200 -
127.0.0.1 - - [12/Feb/2026 17:17:08] "GET /is_valid HTTP/1.1" 200 -

```

Conclusion: In this experiment, we successfully created a basic cryptocurrency using Python and Flask and implemented mining in a multi-node blockchain network. We demonstrated how transactions are added, stored temporarily, and included in a block during the mining process using a Proof-of-Work mechanism. By running three separate nodes, we simulated a peer-to-peer network and applied the Longest Chain Rule to maintain consensus across all nodes. The experiment helped us understand blockchain structure, decentralized networking, mining rewards, and chain synchronization in a practical manner.



**VIVEKANAND EDUCATION SOCIETY'S
INSTITUTE OF TECHNOLOGY
Department of Information Technology
Academic Year: 2025-26**

ITC801 : Blockchain & DLT Lab
Experiment 04

Aim: Hands-on Solidity Programming Assignments for creating Smart Contracts

Roll No.	07
Name	Vedang V. Wajge
Class	D20A
Subject	ITC801: Blockchain and DLT Lab
CO Mapped	CO3:Implement smart contracts in Ethereum using different development frameworks.

Blockchain Experiment 4

AIM: Hands on Solidity Programming Assignments for creating Smart Contracts

THEORY:

Q1: Primitive Data Types, Variables, Functions - pure, view

In Solidity, primitive data types form the foundation of smart contract development. Commonly used types include:

- **uint / int:** unsigned and signed integers of different sizes (e.g., uint256, int128).
- **bool:** represents logical values (true or false).
- **address:** holds a 20-byte Ethereum account address, often used for storing user accounts or contract addresses.
- **bytes / string:** store binary data or textual

data. Variables in Solidity can be

- **state variables:** stored on the blockchain permanently
- **local variables:** temporary, created during function execution
- **global variables:** special predefined variables such as msg.sender, msg.value, and block.timestamp

Functions allow execution of contract logic. Special types of functions include:

- **pure:** cannot read or modify blockchain state; they work only with inputs and internal computations.
- **view:** can read state variables but cannot alter them. This classification helps optimize gas usage and enforces function integrity.

Q2: Inputs and Outputs to Functions

Functions in Solidity can accept input arguments and return one or more output values. Inputs enable users or other contracts to pass data into the contract, while outputs make it possible to return results after computation.

For example, a function can accept an amount in Ether and return whether the transfer was successful. Solidity also allows named return variables, which improve readability and debugging.

Q3: Visibility, Modifiers and Constructors

Function Visibility defines who can access a function:

- **public:** available both inside and outside the contract.
- **private:** only accessible within the same contract.
- **internal:** accessible within the contract and its child contracts.
- **external:** can be called only by external accounts or other contracts.

Modifiers are reusable code blocks that change the behavior of functions. They are often used for access control, such as restricting sensitive functions to the contract owner (onlyOwner).

Constructors are special functions executed only once during contract deployment. They initialize important values, such as setting the deploying account as the owner of the contract.

Q4: Control Flow : if-else, loops

Control flow in Solidity is similar to traditional programming languages:

- **if-else** allows conditional decision-making in contract logic, e.g., checking if a balance is sufficient before transferring funds.
- **Loops (for, while, do-while)** enable repeated execution of code. For example, iterating through an array of users. However, loops must be used carefully, as excessive iterations increase gas consumption, potentially making the contract expensive to execute.

Q5: Data Structures : Arrays, Mappings, structs, enums

- **Arrays:** Can be fixed or dynamic and are used to store ordered lists of elements. Example: an array of addresses for registered users.
- **Mappings:** Key-value pairs that allow quick lookups. Example: mapping(address => uint) for storing balances. Unlike arrays, mappings do not support iteration.
- **Structs:** Allow grouping of related properties into a single data type. Example: struct Player {string name; uint score;}.
- **Enums:** Used to define a set of predefined constants, making code more readable. Example: enum Status { Pending, Active, Closed }.

Q6: Data Locations

Solidity uses three primary data locations for storing variables:

- **storage:** Data stored permanently on the blockchain. Examples: state variables.
- **memory:** Temporary data storage that exists only while a function is executing. Used for local variables and function inputs.
- **calldata:** A non-modifiable and non-persistent location used for external function parameters. It is gas-efficient compared to memory.

Understanding data locations is essential, as they directly impact gas costs and performance.

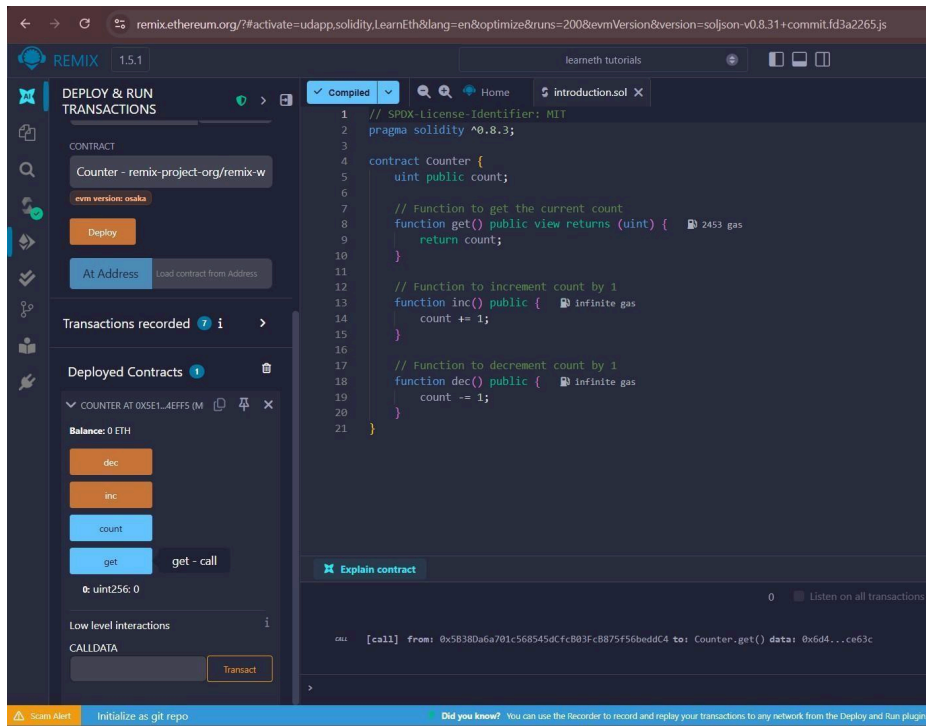
Q7: Transactions : Ether and wei, Gas and Gas Price, Sending Transactions

- **Ether and Wei:** Ether is the main currency in Ethereum. All values are measured in Wei, the smallest unit (1 Ether = 10^{18} Wei). This ensures high precision in financial transactions.
- **Gas and Gas Price:** Every transaction consumes gas, which represents computational effort. The gas price determines how much Ether is paid per unit of gas. A higher gas price incentivizes miners to prioritize the transaction.
- **Sending Transactions:** Transactions are used for transferring Ether or interacting with contracts. Functions like transfer() and send() are commonly used, while call() provides more flexibility. Each transaction requires gas, making efficiency in contract design very important.

TASKS PERFORMED:

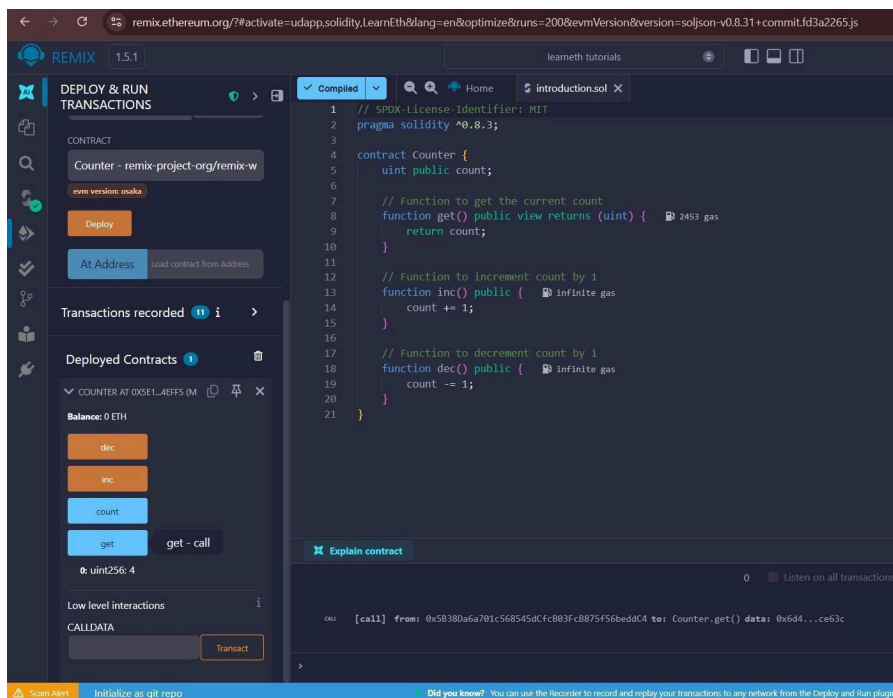
Tutorial 1: Introduction

a. get



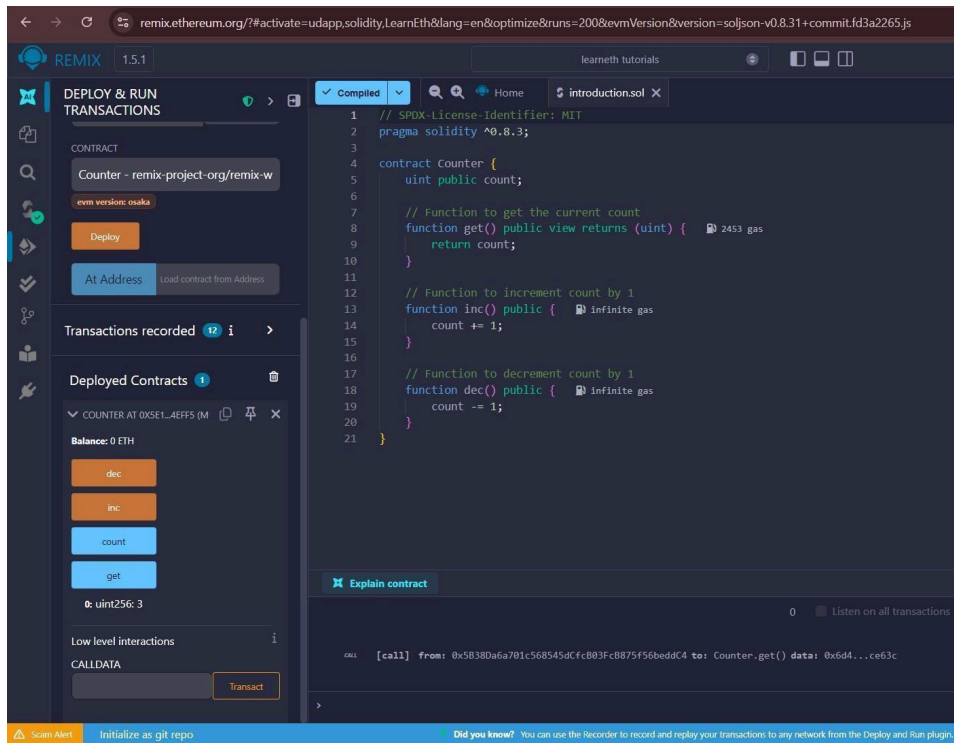
The screenshot shows the Remix IDE interface. On the left, the 'DEPLOY & RUN TRANSACTIONS' panel displays the 'Counter' contract from 'remix-project-org/remix-w'. The 'Deployed Contracts' section shows the contract at address 0x5E1...4E7F5. The 'get' button is highlighted, and the 'get - call' button is also visible. The 'CALLDATA' field is empty, and the 'Transact' button is at the bottom. The main editor shows the Solidity code for the 'Counter' contract, which includes a 'get()' function that returns the current count. The 'Explain contract' panel at the bottom shows the transaction details: 'CALL [call] from: 0x5B380a6a701c568545dCfcB03Fc8875f56beddC4 to: Counter.get() data: 0x6d4...ce63c'.

b. inc



The screenshot shows the Remix IDE interface. On the left, the 'DEPLOY & RUN TRANSACTIONS' panel displays the 'Counter' contract from 'remix-project-org/remix-w'. The 'Deployed Contracts' section shows the contract at address 0x5E1...4E7F5. The 'inc' button is highlighted, and the 'get - call' button is also visible. The 'CALLDATA' field is empty, and the 'Transact' button is at the bottom. The main editor shows the Solidity code for the 'Counter' contract, which includes an 'inc()' function that increments the count by 1. The 'Explain contract' panel at the bottom shows the transaction details: 'CALL [call] from: 0x5B380a6a701c568545dCfcB03Fc8875f56beddC4 to: Counter.get() data: 0x6d4...ce63c'.

c. dec



REMIX 1.5.1

learnmeth tutorials

DEPLOY & RUN TRANSACTIONS

CONTRACT

Counter - remix-project-org/remix-w

em version: osaka

Deploy

At Address Load contract from Address

Transactions recorded 0

Deployed Contracts

COUNTER AT 0x5E1...4EFF5 (M)

Balance: 0 ETH

dec

inc

count

get

uint256: 3

Low level interactions

CALLDATA

Transact

```
1 // SPDX-License-Identifier: MIT
2 pragma solidity ^0.8.3;
3
4 contract Counter {
5     uint public count;
6
7     // Function to get the current count
8     function get() public view returns (uint) { 2453 gas
9         return count;
10    }
11
12    // Function to increment count by 1
13    function inc() public { infinite gas
14        count += 1;
15    }
16
17    // Function to decrement count by 1
18    function dec() public { infinite gas
19        count -= 1;
20    }
21 }
```

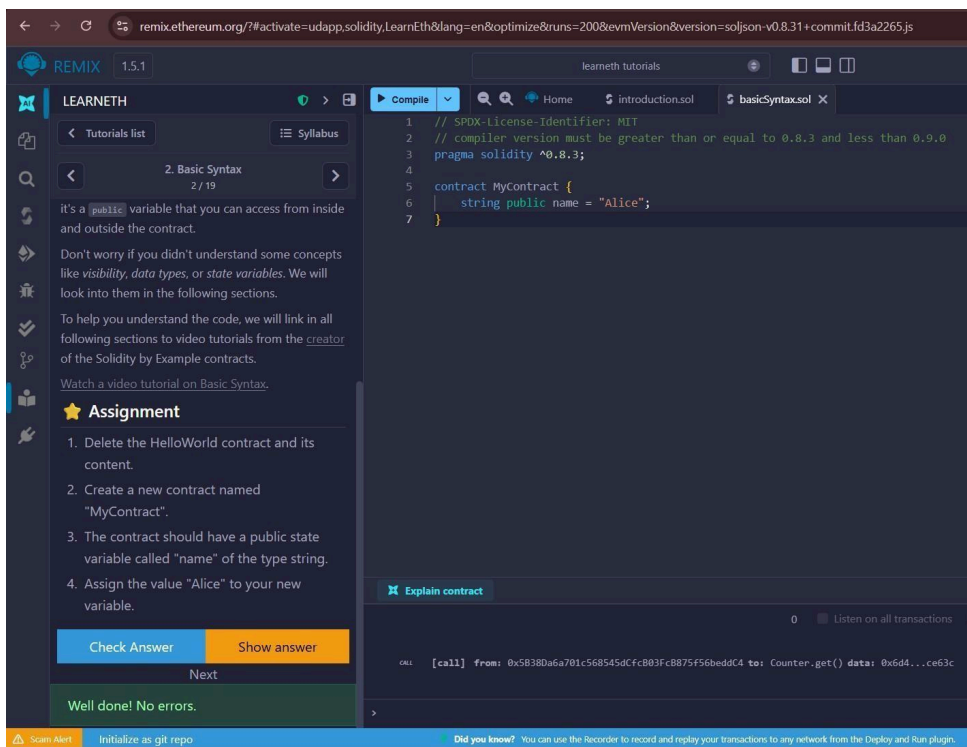
Explain contract

0 Listen on all transactions

CALL [call] from: 0x58380a6a701c568545dcfc803fc8875f56beddC4 to: Counter.get() data: 0x6d4...ce63c

Scan Alert Initialize as git repo Did you know? You can use the recorder to record and replay your transactions to any network from the Deploy and Run plugin.

Tutorial 2: Basic Syntax



REMIX 1.5.1

learnmeth tutorials

LEARNMETH

Tutorials list

Syllabus

2. Basic Syntax 2 / 19

it's a `public` variable that you can access from inside and outside the contract.

Don't worry if you didn't understand some concepts like *visibility*, *data types*, or *state variables*. We will look into them in the following sections.

To help you understand the code, we will link in all following sections to video tutorials from the [creator](#) of the Solidity by Example contracts.

Watch a video tutorial on Basic Syntax.

★ Assignment

1. Delete the HelloWorld contract and its content.
2. Create a new contract named "MyContract".
3. The contract should have a public state variable called "name" of the type string.
4. Assign the value "Alice" to your new variable.

Check Answer Show answer

Next

Well done! No errors.

```
1 // SPDX-License-Identifier: MIT
2 // compiler version must be greater than or equal to 0.8.3 and less than 0.9.0
3 pragma solidity ^0.8.3;
4
5 contract MyContract {
6     string public name = "Alice";
7 }
```

Explain contract

0 Listen on all transactions

CALL [call] from: 0x58380a6a701c568545dcfc803fc8875f56beddC4 to: Counter.get() data: 0x6d4...ce63c

Scan Alert Initialize as git repo Did you know? You can use the Recorder to record and replay your transactions to any network from the Deploy and Run plugin.

Tutorial 3: Primitive Data Types

REMUX 1.5.1

learneth tutorials

LEARNETH

Tutorials list

Syllabus

3. Primitive Data Types 3 / 19

You can learn more about these data types as well as [Fixed Point Numbers](#), [Byte Arrays](#), [Strings](#), and more in the [Solidity documentation](#).

Later in the course, we will look at data structures like [Mappings](#), [Arrays](#), [Enums](#), and [Structs](#).

Watch a video tutorial on Primitive Data Types.

★ Assignment

1. Create a new variable `newAddr` that is a `public address` and give it a value that is not the same as the available variable `addr`.
2. Create a `public` variable called `neg` that is a negative number, decide upon the type.
3. Create a new variable, `newI` that has the smallest `uint` size type and the smallest `uint` value and is `public`.

Tip: Look at the other address in the contract or search the internet for an Ethereum address.

Check Answer Show answer

Next

Well done! No errors.

Scam Alert Initialize as git repo

Did you know? You can use the Recorder to record and replay your transactions to any network from the Deploy and Run plugin.

```
20 // Negative numbers are allowed for int types.
21 // (like uint, different ranges are available from int8 to int256)
22 //
23 int8 public i8 = -1;
24 int public i256 = 456;
25 int public i = -123; // int is same as int256
26
27 address public addr = 0xcA37957d915458f540a06040dfc2f4818fa733c;
28
29 // Default values
30 // Unassigned variables have a default value
31 bool public defaultBool; // false
32 uint public defaultUint; // 0
33 int public defaultInt; // 0
34 address public defaultAddr; // 0x0000000000000000000000000000000000000000
35
36 // New values
37 address public newAddr = 0x0000000000000000000000000000000000000000;
38 int public neg = -12;
39 uint8 public newI = 0;
40 }
```

Explain contract

0 Listen on all transactions

[call] from: 0xc58380a6a701c568545dcfc803fc8875f56bedc4 to: Counter.get() data: 0x6d4...ce63c

Tutorial 4: Variables

REMUX 1.5.1

learneth tutorials

LEARNETH

Tutorials list

Syllabus

4. Variables 4 / 19

addresses, contracts, and transactions.

In this example, we use `block.timestamp` (line 14) to get a Unix timestamp of when the current block was generated and `msg.sender` (line 15) to get the caller of the contract function's address.

A list of all Global Variables is available in the [Solidity documentation](#).

Watch video tutorials on [State Variables](#), [Local Variables](#), and [Global Variables](#).

★ Assignment

1. Create a new public state variable called `blockNumber`.
2. Inside the function `doSomething()`, assign the value of the current block number to the state variable `blockNumber`.

Tip: Look into the global variables section of the Solidity documentation to find out how to read the current block number.

Check Answer Show answer

Next

Well done! No errors.

Scam Alert Initialize as git repo

Did you know? You can use the Recorder to record and replay your transactions to any network from the Deploy and Run plugin.

```
1 // SPDX-License-Identifier: MIT
2 pragma solidity ^0.8.3;
3
4 contract Variables {
5     // State variables are stored on the blockchain.
6     string public text = "Hello";
7     uint public num = 123;
8     uint public blockNumber;
9
10    function doSomething() public {
11        // Local variables are not saved to the blockchain.
12        uint i = 456;
13
14        // Here are some global variables
15        uint timestamp = block.timestamp; // Current block timestamp
16        address sender = msg.sender; // address of the caller
17        blockNumber = block.number;
18    }
19 }
```

Explain contract

0 Listen on all transactions

[call] from: 0xc58380a6a701c568545dcfc803fc8875f56bedc4 to: Counter.get() data: 0x6d4...ce63c

Tutorial 5: Functions - Reading and Writing to a State Variable

LEARNETH 1.5.1

Tutorials list **Syllabus**

5.1 Functions - Reading and Writing to a State Variable 1/19

names. A common convention is to use an underscore as a prefix for the parameter name to distinguish them from state variables.

You can then set the visibility of a function and declare them `view` or `pure` as we do for the `get` function if they don't modify the state. Our `get` function also returns values, so we have to specify the return types. In this case, it's a `uint` since the state variable `num` that the function returns is a `uint`.

We will explore the particularities of Solidity functions in more detail in the following sections.

[Watch a video tutorial on Functions.](#)

Assignment

1. Create a public state variable called `num` that is of type `uint` and initialize it to `true`.
2. Create a public function called `get_b` that returns the value of `num`.

Check Answer **Show answer**

Next

Well done! No errors.

SimpleStorage.sol

```
1 // SPDX-License-Identifier: MIT
2 pragma solidity ^0.8.3;
3
4 contract SimpleStorage {
5     // State variable to store a number
6     uint public num;
7     bool public b = true;
8
9     // You need to send a transaction to write to a state variable.
10    function set(uint _num) public {
11        num = _num;
12    }
13
14    // You can read from a state variable without sending a transaction.
15    function get() public view returns (uint) {
16        return num;
17    }
18
19    function get_b() public view returns (bool) {
20        return b;
21    }
22 }
```

Explain contract

0 Listen on all transactions

[call] from: 0x5B38Da7a701c568545dCfcB03Fc8B75F56beddC4 to: Counter.get() data: 0x64...ce63c

Tutorial 6: Functions - View and Pure

LEARNETH 1.5.1

Tutorials list **Syllabus**

5.2 Functions - View and Pure 6/19

opcodes."

From the [Solidity documentation](#).

You can declare a pure function using the keyword `pure`. In this contract, `add` (line 13) is a pure function. This function takes the parameters `i` and `j`, and returns the sum of them. It neither reads nor modifies the state variable `x`.

In Solidity development, you need to optimise your code for saving computation cost (gas cost). Declaring functions view and pure can save gas cost and make the code more readable and easier to maintain. Pure functions don't have any side effects and will always return the same result if you pass the same arguments.

[Watch a video tutorial on View and Pure Functions.](#)

Assignment

Create a function called `addrox2` that takes the parameter `y` and updates the state variable `x` with the sum of the parameter and the state variable `x`.

Check Answer **Show answer**

Next

Well done! No errors.

ViewAndPure.sol

```
1 // SPDX-License-Identifier: MIT
2 pragma solidity ^0.8.3;
3
4 contract ViewAndPure {
5     uint public x = 1;
6
7     // Promise not to modify the state.
8     function addrox(uint y) public view returns (uint) {
9         return x + y;
10    }
11
12    // Promise not to modify or read from the state.
13    function add(uint i, uint j) public pure returns (uint) {
14        return i + j;
15    }
16
17    function addrox2(uint y) public {
18        x = x + y;
19    }
20 }
```

Explain contract

0 Listen on all transactions

[call] from: 0x5B38Da7a701c568545dCfcB03Fc8B75F56beddC4 to: Counter.get() data: 0x64...ce63c

Tutorial 7: Functions - Modifiers and Constructors

The screenshot shows the Remix IDE interface for Tutorial 7. The sidebar on the left displays the tutorial list, with '5.3 Functions - Modifiers and Constructors' selected. The main editor shows the Solidity code for 'modifiersAndConstructors.sol'. The code includes a constructor that initializes a variable 'x' to 1, and two functions: 'increaseX' which increments 'x' by 'y', and 'decrement' which decrements 'x' by 1. The 'increaseX' function uses a 'require' statement to ensure 'x' is not greater than 100. The 'decrement' function uses a 'require' statement to ensure 'x' is not less than 1. The console at the bottom shows a transaction call to 'Counter.get()' with data '0x6d4...ce63c'.

```
43 _x
44 x = x + y;
45 }
46
47 function increaseX(uint y) public onlyOwner biggerThan0(y) increaseXbyY(y) {
48
49
50 // Modifiers can be called before and / or after a function.
51 // This modifier prevents a function from being called while
52 // it is still executing.
53 modifier noReentrancy() {
54     require(!locked, "No reentrancy");
55
56     locked = true;
57     _x
58     locked = false;
59 }
60
61 function decrement(uint i) public noReentrancy {
62     x -= i;
63
64     if (i > 1) {
65         decrement(i - 1);
66     }
67 }
68 }
```

Tutorial 8: Functions - Inputs and Outputs

The screenshot shows the Remix IDE interface for Tutorial 8. The sidebar on the left displays the tutorial list, with '5.4 Functions - Inputs and Outputs' selected. The main editor shows the Solidity code for 'inputsAndOutputs.sol'. The code includes a function 'returnTwo' that returns an array of two integers, and a function 'arrayInput' that takes an array of integers as input. The 'returnTwo' function uses a 'pure' modifier and returns an array of two integers. The 'arrayInput' function takes an array of integers as input and returns the array. The console at the bottom shows a transaction call to 'Counter.get()' with data '0x6d4...ce63c'.

```
64
65 return (i, b, j, x, y);
66 }
67
68 // Cannot use map for neither input nor output
69
70 // Can use array for input
71 function arrayInput(uint[] memory _arr) public {
72
73 // Can use array for output
74 uint[] public arr;
75
76 function arrayOutput() public view returns (uint[] memory) {
77     return arr;
78 }
79
80 function returnTwo()
81 public
82 pure
83 returns (
84     int i,
85     bool b
86 )
87 {
88     i = -2;
89     b = true;
90 }
91 }
```

Tutorial 9: Visibility

LEARNETH 15.1

6. Visibility 9/19

- State variables can not be `external`.

In this example, we have two contracts, the `Base` contract (line 4) and the `Child` contract (line 55) which inherits the functions and state variables from the `Base` contract.

When you uncomment the `testPrivateFunc()` (lines 58-60) you get an error because the child contract doesn't have access to the private function `testPrivateFunc()` from the `Base` contract.

If you compile and deploy the two contracts, you will not be able to call the functions `testPrivateFunc()` and `testInternalFunc()` directly. You will only be able to call them via `testPrivateFunc()` and `testInternalFunc()`.

Watch a video tutorial on Visibility.

Assignment

Create a new function in the `Child` contract called `testStateVars()` that returns the values of all state variables from the `Base` contract that are possible to return.

Check Answer **Show answer**

Next

Well done! No errors.

```
pragma solidity ^0.8.3;

contract Base {
    // State variables
    string private privateVar = "my private variable";
    string internal internalVar = "my internal variable";
    string public publicVar = "my public variable";
    // State variables cannot be external so this code won't compile.
    // string external externalVar = "my external variable";
}

contract Child is Base {
    // Inherited contracts do not have access to private functions
    // and state variables.
    // function testPrivateFunc() public pure returns (string memory) {
    //     return privateVar();
    // }

    // Internal function call be called inside child contracts.
    function testInternalFunc() public pure override returns (string memory) {
        return internalFunc();
    }

    function testInternalVar() public view returns (string memory, string memory) {
        return (internalVar, publicVar);
    }
}
```

Explain contract

0 Listen on all transactions

[call] from: 0x58380a6a701c568545dcfc803fc8875f56beddC4 to: Counter.get() data: 0x6d4...ce63c

Tutorial 10: Control Flow - If/Else

LEARNETH 15.1

7.1 Control Flow - If/Else 10/19

else if

With the `else if` statement we can combine several conditions.

If the first condition (line 6) of the `foo` function is not met, but the condition of the `else if` statement (line 8) becomes true, the function returns `1`.

Watch a video tutorial on the If/Else statement.

Assignment

Create a new function called `evenCheck` in the `IfElse` contract:

- That takes in a `uint` as an argument.
- The function returns `true` if the argument is even, and `false` if the argument is odd.
- Use a ternary operator to return the result of the `evenCheck` function.

Tip: The modulo (%) operator produces the remainder of an integer division.

Check Answer **Show answer**

Next

Well done! No errors.

```
pragma solidity ^0.8.3;

contract IfElse {
    function foo(uint x) public pure returns (uint) {
        if (x < 10) {
            return 0;
        } else if (x < 20) {
            return 1;
        } else {
            return 2;
        }
    }

    function ternary(uint x) public pure returns (uint) {
        // if (x < 10) {
        //     return 1;
        // }
        // return 2;

        // shorthand way to write if / else statement
        return x < 10 ? 1 : 2;
    }

    function evenCheck(uint y) public pure returns (bool) {
        return y % 2 == 0 ? true : false;
    }
}
```

Explain contract

0 Listen on all transactions

[call] from: 0x58380a6a701c568545dcfc803fc8875f56beddC4 to: Counter.get() data: 0x6d4...ce63c

Tutorial 11: Control Flow - Loops

The screenshot shows the Remix IDE interface for Tutorial 11: Control Flow - Loops. The sidebar on the left displays the tutorial list, with the current tutorial selected. The main editor shows the Solidity code for a contract named `loop`. The code includes a `uint public count;` and a `function loop() public` that uses a `for` loop and a `while` loop. The console at the bottom shows the output of the `loop` function, indicating that the count is 5.

```
1 // SPDX-License-Identifier: MIT
2 pragma solidity ^0.8.3;
3
4 contract loop {
5     uint public count;
6     function loop() public {
7         // for loop
8         for (uint i = 0; i < 10; i++) {
9             if (i == 5) {
10                 // Skip to next iteration with continue
11                 continue;
12             }
13             if (i == 5) {
14                 // Exit loop with break
15                 break;
16             }
17             count++;
18         }
19
20         // while loop
21         uint j;
22         while (j < 10) {
23             j++;
24         }
25     }
26 }
27
```

Check Answer Show answer

Well done! No errors.

Tutorial 12: Data Structures - Arrays

The screenshot shows the Remix IDE interface for Tutorial 12: Data Structures - Arrays. The sidebar on the left displays the tutorial list, with the current tutorial selected. The main editor shows the Solidity code for a contract named `CompactArray`. The code includes a `uint[] public arr;` and a `function remove(uint index) public` that removes an element from the array. The console at the bottom shows the output of the `remove` function, indicating that the array is compacted.

```
40 // Delete does not change the array length.
41 // It resets the value at index to it's default value,
42 // in this case 0
43 delete arr[index];
44 }
45
46
47 contract CompactArray {
48     uint[] public arr;
49
50     // Deleting an element creates a gap in the array.
51     // One trick to keep the array compact is to
52     // move the last element into the place to delete.
53     function remove(uint index) public {
54         // Move the last element into the place to delete
55         arr[index] = arr[arr.length - 1];
56         // Remove the last element
57         arr.pop();
58     }
59
60     function test() public {
61         arr.push(1);
62         arr.push(2);
63         arr.push(3);
64         arr.push(4);
65         // [1, 2, 3, 4]
66
67         remove(1);
68         // [1, 4, 3]
69
70         remove(2);
71         // [1, 4]
72     }
73 }

```

Check Answer Show answer

Well done! No errors.

Tutorial 13: Data Structures - Mappings

The screenshot shows the Remix IDE interface for Tutorial 13. The sidebar on the left contains the tutorial content, including a list of steps and an assignment section. The main editor displays the Solidity code for the 'NestedMapping' contract. The code includes functions for setting and removing values from a mapping, and a nested mapping structure. The bottom status bar indicates that the code is compiled and there are no errors.

LEARNETH

8.2 Data Structures - Mappings

mapping's name and key in brackets and assigning it a new value (line 16).

Removing values

We can use the delete operator to delete a value associated with a key, which will set it to the default value of 0. As we have seen in the arrays section.

Watch a video tutorial on Mappings.

Assignment

1. Create a public mapping `balances` that associates the key type `address` with the value type `uint`.
2. Change the functions `get` and `remove` to work with the mapping `balances`.
3. Change the function `set` to create a new entry to the `balances` mapping, where the key is the address of the parameter and the value is the balance associated with the address of the parameter.

[Check Answer](#) [Show answer](#)

Next

Well done! No errors.

contracts

```
11
12
13
14 function set(address _addr) public {
15     // update the value at this address
16     balances[_addr] = _addr.balance;
17 }
18
19 function remove(address _addr) public {
20     // Reset the value to the default value.
21     delete balances[_addr];
22 }
23
24
25 contract NestedMapping {
26     // Nested mapping (mapping from address to another mapping)
27     mapping(address => mapping(uint => bool)) public nested;
28
29     function get(address _addr1, uint _i) public view returns (bool) {
30         // You can get values from a nested mapping
31         // even when it is not initialized
32         return nested[_addr1][_i];
33     }
34
35     function set(
36         address _addr1,
37         uint _i,
38         bool _boo
39     ) public {
40         nested[_addr1][_i] = _boo;
41     }
42
43     function remove(address _addr1, uint _i) public {
44         delete nested[_addr1][_i];
45     }
46 }
```

Tutorial 14: Data Structures - Structs

The screenshot shows the Remix IDE interface for Tutorial 14. The sidebar on the left contains the tutorial content, including a list of steps and an assignment section. The main editor displays the Solidity code for the 'TODOS' contract. The code includes functions for pushing new todos, getting a todo by index, updating a todo, and removing a todo. The bottom status bar indicates that the code is compiled and there are no errors.

LEARNETH

8.3 Data Structures - Structs

parameters in parentheses (line 16).

Key-value mapping: We provide the name of the struct and the keys and values as a mapping inside curly braces (line 19).

Initialize and update a struct: We initialize an empty struct first and then update its member by assigning it a new value (line 23).

Accessing structs

To access a member of a struct we can use the dot operator (line 33).

Updating structs

To update a struct's member we also use the dot operator and assign it a new value (lines 39 and 45).

Watch a video tutorial on Structs.

Assignment

Create a function `remove` that takes a `uint` as a parameter and deletes a struct member with the given index in the `todos` mapping.

[Check Answer](#) [Show answer](#)

Next

Well done! No errors.

contracts

```
16
17
18
19 // key value mapping
20 todos.push(Todo({text: _text, completed: false}));
21
22 // initialize an empty struct and then update it
23 Todo memory todo;
24 todo.text = _text;
25 // todo.completed initialized to false
26
27 todos.push(todo);
28
29
30 // Solidity automatically created a getter for 'todos' so
31 // you don't actually need this function.
32 function get(uint _index) public view returns (string memory text, bool completed) {
33     Todo storage todo = todos[_index];
34     return (todo.text, todo.completed);
35 }
36
37 // update text
38 function update(uint _index, string memory _text) public {
39     Todo storage todo = todos[_index];
40     todo.text = _text;
41 }
42
43 // update completed
44 function toggleCompleted(uint _index) public {
45     Todo storage todo = todos[_index];
46     todo.completed = !todo.completed;
47 }
48
49 function remove(uint _index) public {
50     delete todos[_index];
51 }
```

Tutorial 15: Data Structures - Enums

REMUX 1.5.1 learneth tutorials

LEARNETH

Tutorials list

8.4 Data Structures - Enums 15 / 19

We can update the enum value of a variable by assigning it the `value` representing the enum member (line 30). Shipped would be 1 in this example. Another way to update the value is using the dot operator by providing the name of the enum and its member (line 35).

Removing an enum value

We can use the delete operator to delete the enum value of the variable, which means as for arrays and mappings, to set the default value to 0.

Watch a video tutorial on Enums.

★ Assignment

1. Define an enum type called `Size` with the members `S`, `M`, and `L`.
2. Initialize the variable `size` of the enum type `Size`.
3. Create a getter function `getSize()` that returns the value of the variable `size`.

Check Answer Show answer

Next

Well done! No errors.

Explain contract

```
13
14
15 enum Size {
16     S,
17     M,
18     L
19 }
20
21 // Default value is the first element listed in
22 // definition of the type, in this case "Pending"
23 Status public status;
24 Size public sizes;
25
26 function get() public view returns (Status) {
27     return status;
28 }
29
30 function getSize() public view returns (Size) {
31     return sizes;
32 }
33
34 // Update status by passing uint into input
35 function set(status _status) public {
36     status = _status;
37 }
38
39 // You can update to a specific enum like this
40 function cancel() public {
41     status = Status.Canceled;
42 }
43
44 // delete resets the enum to its first value, 0
45 function reset() public {
46     delete status;
47 }
```

Tutorial 16: Data Locations

REMUX 1.5.1 learneth tutorials

LEARNETH

Tutorials list

9. Data Locations 16 / 19

amount of gas possible.

★ Assignment

1. Change the value of the `myStruct` member `foo`, inside the function `f`, to 4.
2. Create a new struct `myMemStruct2` with the data location `memory` inside the function `f` and assign it the value of `myMemStruct`. Change the value of the `myMemStruct2` member `foo` to 1.
3. Create a new struct `myMemStruct3` with the data location `memory` inside the function `f` and assign it the value of `myStruct`. Change the value of the `myMemStruct3` member `foo` to 3.
4. Let the function `f` return `myStruct`, `myMemStruct2`, and `myMemStruct3`.

Tip: Make sure to create the correct return types for the function `f`.

Check Answer Show answer

Next

Well done! No errors.

Explain contract

```
13
14
15 function f() public returns (MyStruct memory, MyStruct memory, MyStruct memory) {
16     // call f with state variables
17     f(arr, map, myStructs[1]);
18     // get a struct from a mapping
19     MyStruct storage myStruct = myStructs[1];
20     myStruct.foo = 4;
21     // create a struct in memory
22     MyStruct memory myMemStruct = MyStruct(0);
23     MyStruct memory myMemStruct2 = myMemStruct;
24     myMemStruct2.foo = 1;
25
26     MyStruct memory myMemStruct3 = myStruct;
27     myMemStruct3.foo = 3;
28     return (myStruct, myMemStruct2, myMemStruct3);
29 }
30
31 function f(
32     uint[] storage _arr,
33     mapping(uint => address) storage _map,
34     MyStruct storage _myStruct
35 ) internal {
36     // do something with storage variables
37
38     // You can return memory variables
39     function g(uint[] memory _arr) public returns (uint[] memory) {
40         // do something with memory array
41         _arr[0] = 1;
42     }
43
44     function h(uint[] calldata _arr) external {
45         // do something with calldata array
46         _arr[0] = 1;
47     }
48 }
```


Tutorial 17: Transactions - Ether and Wei

LEARNETH 15.1

Tutorials list 17 / 19

Wei is the smallest subunit of Ether, named after the cryptographer Wei Dai. Ether numbers without a suffix are treated as `wei` (line 7).

One `gwei` (giga-wei) is equal to 1,000,000,000 (10^{10}) `wei`.

One `ether` is equal to 1,000,000,000,000,000 (10^{18}) `wei` (line 11).

Watch a video tutorial on Ether and Wei.

Assignment

1. Create a `public` `uint` called `oneWei` and set it to 1 `gwei`.
2. Create a `public` `bool` called `isOneWei` and set it to the result of a comparison operation between 1 `gwei` and 10^{10} .

Tip: Look at how this is written for `gwei` and `ether` in the contract.

etherAndWei.sol

```
1 // SPDX-License-Identifier: MIT
2 pragma solidity ^0.8.3;
3
4 contract etherAndWei {
5     uint public oneWei = 1 gwei;
6     // 1 wei is equal to 1
7     bool public isOneWei = 1 wei == 1;
8
9     uint public oneEther = 1 ether;
10    // 1 ether is equal to 10^18 wei
11    bool public isOneEther = 1 ether == 1e18;
12
13    uint public oneGwei = 1 gwei;
14    // 1 ether is equal to 10^9 wei
15    bool public isOneGwei = 1 gwei == 1e9;
16 }
```

Check Answer Show answer

Next

Well done! No errors.

Explain contract

Scan Alert Initialize as git repo Did you know? You can use the Recorder to record and replay your transactions to any network from the Deploy and Run plugin.

Tutorial 18: Transactions - Gas and Gas Price

LEARNETH 15.1

Tutorials list 18 / 19

Gas limit

When sending a transaction, the sender specifies the maximum amount of gas that they are willing to pay for. If they set the limit too low, their transaction can run out of gas before being completed, reverting any changes being made. In this case, the gas was consumed and can't be refunded.

Learn more about [gas](#) on [ethereum.org](#).

Watch a video tutorial on Gas and Gas Price.

Assignment

Create a new `public` state variable in the `Gas` contract called `cost` of the type `uint`. Store the value of the gas cost for deploying the contract in the new variable, including the cost for the value you are storing.

Tip: You can check in the Remix terminal the details of a transaction, including the gas cost. You can also use the Remix plugin *Gas Profiler* to check for the gas cost of transactions.

gasAndGasPrice.sol

```
1 // SPDX-License-Identifier: MIT
2 pragma solidity ^0.8.3;
3
4 contract Gas {
5     uint public i = 0;
6     uint public cost = 170367;
7
8     // Using up all of the gas that you send causes your transaction to fail.
9     // State changes are undone.
10    // Gas spent are not refunded.
11    function forever() public {
12        // Here we run a loop until all of the gas are spent
13        // and the transaction fails
14        while (true) {
15            i += 1;
16        }
17    }
18 }
```

Check Answer Show answer

Next

Well done! No errors.

Explain contract

Scan Alert Initialize as git repo Did you know? You can use the Recorder to record and replay your transactions to any network from the Deploy and Run plugin.

Tutorial 19: Transactions - Sending Ether

```
17 // This function is no longer recommended for sending ether.
18 // to.transfer(msg.value);
19 }
20
21 function sendViaSend(address payable _to) public payable {
22     // Send returns a boolean value indicating success or failure.
23     // This function is not recommended for sending ether.
24     bool sent = _to.send(msg.value);
25     require(sent, "Failed to send Ether");
26 }
27
28 function sendViaCall(address payable _to) public payable {
29     // Call returns a boolean value indicating success or failure.
30     // This is the current recommended method to use.
31     (bool sent, bytes memory data) = _to.call{value: msg.value}("");
32     require(sent, "Failed to send Ether");
33 }
34
35 contract Charity {
36     address public owner;
37
38     constructor() {
39         owner = msg.sender;
40     }
41
42     function donate() public payable {
43
44     }
45
46     function withdraw() public {
47         uint amount = address(this).balance;
48
49         (bool sent, bytes memory data) = owner.call{value: amount}("");
50         require(sent, "Failed to send Ether");
51     }
52 }
```

CONCLUSION:

Through this experiment, the basic concepts of Solidity programming were learned by completing practical assignments using the Remix IDE. Important topics such as data types, variables, different types of functions, visibility, modifiers, constructors, control flow statements, data structures, and transactions were studied and applied while creating smart contracts. The hands-on practice helped in understanding how to design, compile, and deploy contracts using the Remix VM. Overall, this experiment helped in building a clear understanding of blockchain concepts and provided a strong foundation for developing and managing smart contracts effectively.



**VIVEKANAND EDUCATION SOCIETY'S
INSTITUTE OF TECHNOLOGY
Department of Information Technology
Academic Year: 2025-26**

ITC801 : Blockchain & DLT Lab
Experiment 05

Aim: Deploying a Voting/Ballot Smart Contract

Roll No.	07
Name	Vedang V. Wajge
Class	D20A
Subject	ITC801: Blockchain and DLT Lab
CO Mapped	CO1:Describe the basic concept of Blockchain and Distributed Ledger Technology. CO2:Interpret the knowledge of the Bitcoin network, nodes, keys, wallets and transactions CO3:Implement smart contracts in Ethereum using different development frameworks.

Experiment No. 5

Aim: Deploying a Voting/Ballot Smart Contract

Theory:

1. Relevance of require Statements in Solidity Programs

In Solidity, the **require** statement acts as a **guard condition** within functions. It ensures that only valid inputs or authorized users can execute certain parts of the code. If the condition inside **require** is not satisfied, the function execution stops immediately, and all state changes made during that transaction are reverted to their original state. This rollback mechanism ensures that invalid transactions do not corrupt the blockchain data.

For example, in a **Voting Smart Contract**, **require** can be used to check:

- Whether the person calling the function has the right to vote (`require(voters[msg.sender].weight > 0, "Has no right to vote");`).
- Whether a voter has already voted before allowing them to vote again.
- Whether the function caller is the **chairperson** before granting voting rights.

Thus, **require** statements enforce **security, correctness, and reliability** in smart contracts. They also allow developers to attach error messages, making debugging and contract interaction easier for users.

2. Keywords: mapping, storage, and memory

- **mapping:**

A mapping is a special data structure in Solidity that links keys to values, similar to a hash table. Its syntax is `mapping(keyType => valueType)`. For example:

```
mapping(address => Voter) public voters;
```

Here, each address (Ethereum account) is mapped to a Voter structure. Mappings are very useful for contracts like **Ballot**, where you need to associate voters with their data (whether they voted, which proposal they chose, etc.). Unlike arrays, mappings do not have a length property and cannot be iterated over directly, making them **gas efficient** for lookups but limited for enumeration.

- **storage:**

In Solidity, storage refers to the **permanent memory** of the contract, stored on the Ethereum blockchain. Variables declared at the contract

level are stored in storage by default. Data stored in storage is persistent across transactions, which means once written, it remains available unless explicitly modified. However, because writing to blockchain storage consumes gas, it is more expensive. For example, a voter's information saved in the voters mapping remains available throughout the contract's lifecycle.

- **memory:**

In contrast, memory is **temporary storage**, used only for the lifetime of a function call. When the function execution ends, the data stored in memory is discarded. Memory is mainly used for temporary variables, function arguments, or computations that don't need to be permanently stored on the blockchain. It is cheaper than storage in terms of gas cost. For instance, when handling proposal names or temporary string manipulations, memory is often used.

Thus, a smart contract developer must **balance between storage and memory** to ensure efficiency and cost-effectiveness.

3. Why bytes32 Instead of string?

In earlier implementations of the Ballot contract, bytes32 was used for proposal names instead of string. The reason lies in **efficiency and gas optimization**.

- **bytes32** is a **fixed-size type**, meaning it always stores exactly 32 bytes of data. This makes storage simple, comparison operations faster, and gas costs lower. However, it limits proposal names to 32 characters, which is not very flexible for user-friendly names.
- **string** is a **dynamically sized type**, meaning it can store text of variable length. While it is easier for developers and users (since names can be written normally), it requires more complex handling inside the Ethereum Virtual Machine (EVM). This increases gas usage and may slow down comparisons or manipulations.

To make the system more user-friendly, modern implementations of the Ballot contract often convert from bytes32 to string. Tools like the **Web3 Type Converter** help developers easily switch between these two types for deployment and testing.

In summary, bytes32 is used when performance and gas efficiency are priorities, while string is preferred for readability and ease of use.

Code:

```
// SPDX-License-Identifier: GPL-3.0  
pragma solidity ^0.8.20;
```

```
/**
 * @title Ballot
 * @dev Implements voting process along with vote delegation
 */
//Prajjwal Pandey D20A / 37
contract Ballot {

    struct Voter {
        uint weight;    // weight is accumulated by delegation
        bool voted;     // if true, that person already voted
        address delegate; // person delegated to
        uint vote;      // index of the voted proposal
    }

    struct Proposal {
        string name;    // short name of proposal
        uint voteCount; // number of accumulated votes
    }

    address public chairperson;

    mapping(address => Voter) public voters;
    Proposal[] public proposals;

    /**
     * @dev Create a new ballot to choose one of `proposalNames`.
     */
    constructor(string[] memory proposalNames) {
        require(proposalNames.length > 0, "Must provide proposals");

        chairperson = msg.sender;
        voters[chairperson].weight = 1;

        for (uint i = 0; i < proposalNames.length; i++) {
            proposals.push(Proposal({
                name: proposalNames[i],
                voteCount: 0
            }));
        }
    }

    /**
     * @dev Give `voter` the right to vote on this ballot.
     * Can only be called by chairperson.
     */
}
```

```
function giveRightToVote(address voter) external {
    require(msg.sender == chairperson, "Only chairperson can give right to vote");
    require(!voters[voter].voted, "The voter already voted");
    require(voters[voter].weight == 0, "Voter already has right to vote");

    voters[voter].weight = 1;
}

/**
 * @dev Delegate your vote to the voter `to`.
 */
function delegate(address to) external {
    Voter storage sender = voters[msg.sender];

    require(sender.weight != 0, "You have no right to vote");
    require(!sender.voted, "You already voted");
    require(to != msg.sender, "Self-delegation is disallowed");

    // Follow delegation chain
    while (voters[to].delegate != address(0)) {
        to = voters[to].delegate;
        require(to != msg.sender, "Found loop in delegation");
    }

    Voter storage delegate_ = voters[to];
    require(delegate_.weight >= 1, "Delegate has no right to vote");
    sender.voted = true;
    sender.delegate = to;

    if (delegate_.voted) {
        proposals[delegate_.vote].voteCount += sender.weight;
    } else {
        delegate_.weight += sender.weight;
    }
}

/**
 * @dev Give your vote (including delegated votes) to proposal `proposal`.
 */
function vote(uint proposal) external {
    require(proposal < proposals.length, "Invalid proposal");

    Voter storage sender = voters[msg.sender];
```

```
require(sender.weight != 0, "Has no right to vote");
require(!sender.voted, "Already voted");

sender.voted = true;
sender.vote = proposal;

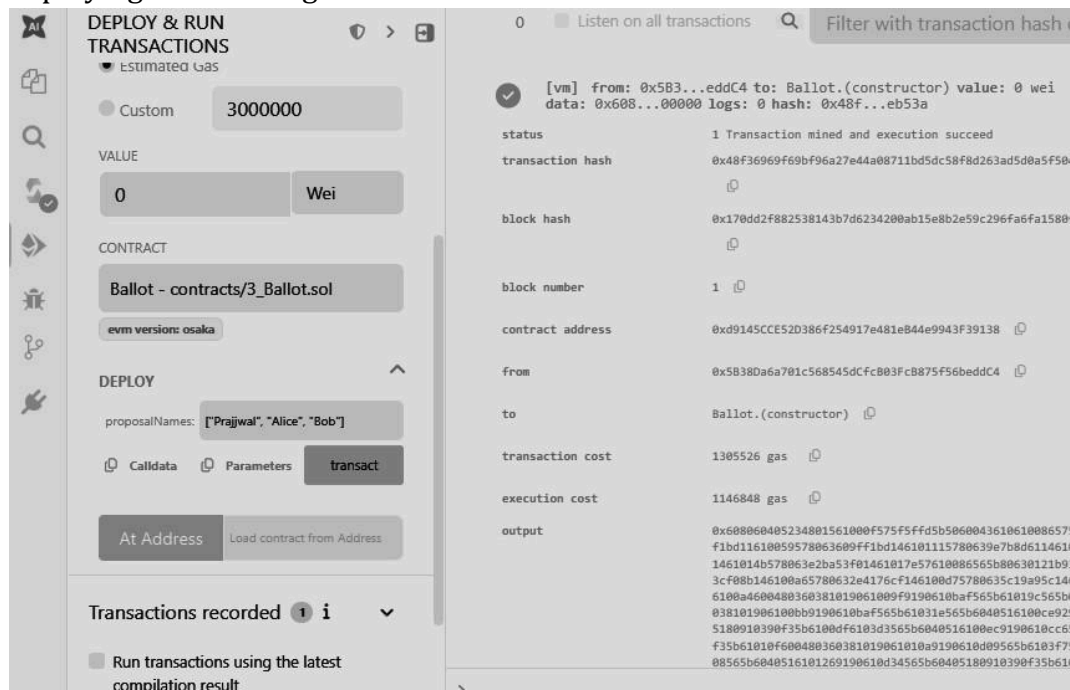
proposals[proposal].voteCount += sender.weight;
}

/**
 * @dev Computes the winning proposal.
 */
function winningProposal() public view returns (uint winningProposal_) {
    uint winningVoteCount = 0;

    for (uint p = 0; p < proposals.length; p++) {
        if (proposals[p].voteCount > winningVoteCount) {
            winningVoteCount = proposals[p].voteCount;
            winningProposal_ = p;
        }
    }
}

/**
 * @dev Returns the name of the winning proposal.
 */
function winnerName() external view returns (string memory winnerName_) {
    winnerName_ = proposals[winningProposal()].name;
}
}
```


1. Compiled Ballot.sol contract



3. Loading the Proposal Candidate's Names (string)

The screenshot shows the Remix IDE interface. On the left, the 'DEPLOY & RUN' panel displays the 'vote' transaction with a value of 2. The 'winningProposal' variable is shown as a string: 'winnerName_Prajwal'. On the right, the code editor shows the Solidity code for the 'winningProposal()' function, which iterates through the 'proposals' array to find the winning proposal based on the highest vote count.

```
101 proposals[proposal].voteCount += sender.weight;
102 }
103
104 /**
105  * @dev Computes the winning proposal.
106  */
107 function winningProposal() public view returns (uint winningP
108     uint winningVoteCount = 0;
109
110     for (uint p = 0; p < proposals.length; p++) {
111         if (proposals[p].voteCount > winningVoteCount) {
112             winningVoteCount = proposals[p].voteCount;
113             winningProposal_ = p;
114         }
115     }
116 }
117
118 /**
119  * @dev Returns the name of the winning proposal.
120  */
121 function winnerName() external view returns (string memory wi
```

4. Viewing the details of the Proposal Candidate

The screenshot shows the 'PROPOSALS' section in the Remix IDE. It displays a list of proposals with their respective details, including the proposal ID, the candidate's name, and the vote count. The 'call' button is visible, indicating that the proposal details can be viewed or updated.

5. Giving the right to an account other than and by the chairman

DEPLOY & RUN
TRANSACTIONS

Deployed Contracts 1

✓ BALLOT AT 0XD91...39138 (ME)

Balance: 0 ETH

delegate

address to

▼

GIVERIGHTTOVOTE

voter: "Prajwal"

Calldata

Parameters

transact

vote

2

▼

chairperson

PROPOSALS

:

"2"

Calldata

Parameters

call

voters

address

▼

winnerName

6. Selecting the account which was given the right to vote and then writing the proposal candidate's index to vote.

The screenshot shows the 'DEPLOY & RUN TRANSACTIONS' interface. Under 'Deployed Contracts', the 'BALLOT AT 0XD91...39138 (ME)' contract is selected. The balance is 0 ETH. The 'GIVERIGHTTOVOTE' section has a 'voter' field with the address '0xAb8483F64d9C6d1EcF9b848Ae6'. Below this are buttons for 'Calldata', 'Parameters', and 'transact'. The 'vote' field is set to '2', and there is a 'chairperson' button. The 'PROPOSALS' section shows a field with the value '2' and buttons for 'Calldata', 'Parameters', and 'call'. At the bottom, there is a 'voters' field and an 'address' dropdown.

7. We can view the Voter's Information

The screenshot shows the 'DEPLOY & RUN TRANSACTIONS' interface with the transaction details for the 'GIVERIGHTTOVOTE' contract. The transaction hash is '0x51130d35c3b4f97b1f77560d71cb9628788cd1e9ed32a09362882e8b8575'. The block hash is '0x579700816fb321547fbed40658270d77ab42f566f3394d6009b3ca4080f751c'. The block number is 1. The 'from' address is '0xAb8483F64d9C6d1EcF9b848Ae6770D315835c2'. The 'to' address is 'Ballot.vote(uint256) 0xd9145CCES2D386F254917e481e844e9943F39138'. The transaction cost is 73135 gas, and the execution cost is 51923 gas. The output is '0x'. The decoded input is '{ "uint256 proposal": "2" }'. The decoded output is '()'. The logs are '[]'. The raw logs are '[]'.

8. Selecting the account of the delegator and then writing the address of the voter to be delegated to.

DEPLOY & RUN TRANSACTIONS

Deployed Contracts

BALLOT AT 0XD91...39138 (ME)

Balance: 0 ETH

delegate address to

GIVERIGHTTOVOTE

voter: 0x4820993Bc481177ec7E85710aC...

Calldata Parameters transact

VOTE

proposal: 0

Calldata Parameters transact

chairperson

PROPOSALS

block number 1

from 0x08483F64D9C6d11cF9B643Ae77dD1115875c92

to Ballot.vote(uint256) 0xd9145CCCE52D386F254917e481e844e9943F39138

transaction cost 71115 gas

execution cost 51923 gas

output 0x

decoded input { "uint256 proposal": "0" }

decoded output {}

logs []

raw logs []

transact to Ballot.giveRightToVote pending ...

[vm] from: 0x583...eddC4
to: Ballot.giveRightToVote(address) 0xd91...39138 value: 0 wei
data: 0x9e7...c82db logs: 0 hash: 0xf4d...36e0c

Debug

9. Proposal Candidate's Information before giving the Delegate's vote.

DEPLOY & RUN TRANSACTIONS

Balance: 0 ETH

delegate address to

GIVERIGHTTOVOTE

voter: 0x4820993Bc481177ec7E85710aC...

Calldata Parameters transact

VOTE

proposal: 1

Calldata Parameters transact

chairperson

PROPOSALS

proposal: 1

Calldata Parameters call

voters address

winnerName

block number 1

from 0x08483F64D9C6d11cF9B643Ae77dD1115875c92

to Ballot.vote(uint256) 0xd9145CCCE52D386F254917e481e844e9943F39138

transaction cost 71115 gas

execution cost 51923 gas

output 0x

decoded input { "uint256 proposal": "0" }

decoded output {}

logs []

raw logs []

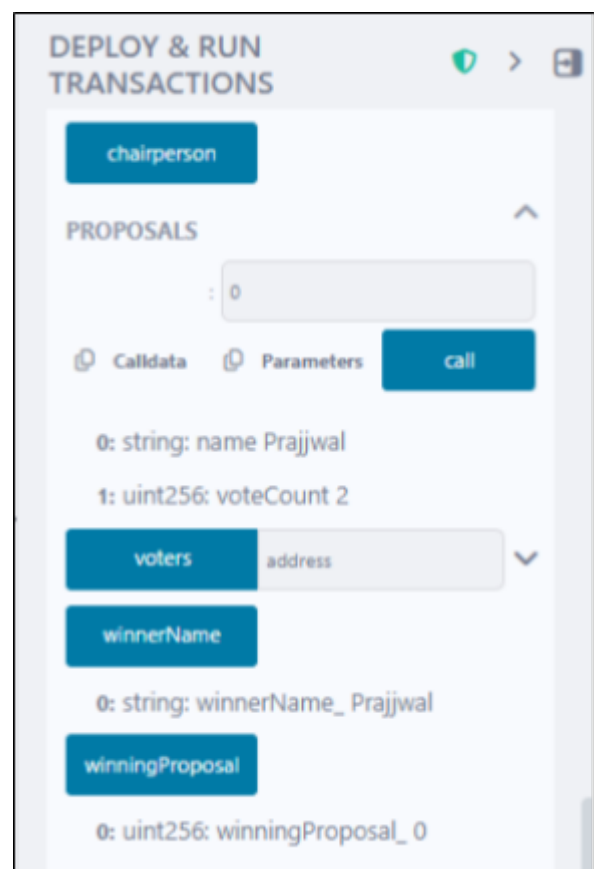
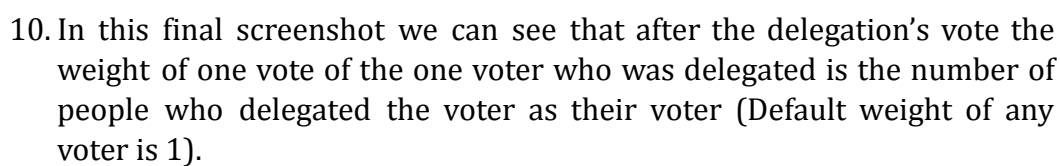
transact to Ballot.giveRightToVote pending ...

[vm] from: 0x583...eddC4
to: Ballot.giveRightToVote(address) 0xd91...39138 value: 0 wei
data: 0x9e7...c82db logs: 0 hash: 0xf4d...36e0c

transact to Ballot.vote pending ...

[vm] from: 0x482...C82db to: Ballot.vote(uint256) 0xd91...39138
value: 0 wei data: 0x012...80001 logs: 0 hash: 0xfb1...c5cd6

Debug



Conclusion: In this experiment, a Voting/Ballot smart contract was deployed using Solidity on the Remix IDE. The concepts of require statements, mapping, and data location specifiers like storage and memory were explored to understand their role in ensuring security, efficiency, and correctness in smart contracts. The difference between using bytes32 and string for proposal names was also studied, highlighting the trade-off between gas efficiency and readability. Overall, the experiment provided practical insights into the design and deployment of voting contracts on the blockchain.



**VIVEKANAND EDUCATION SOCIETY'S
INSTITUTE OF TECHNOLOGY
Department of Information Technology
Academic Year: 2025-26**

ITC801 : Blockchain & DLT Lab
Experiment 06

Aim: Creating Smart Contracts and performing transactions using Solidity and Remix IDE

Roll No.	07
Name	Vedang V. Wajge
Class	D20A
Subject	ITC801: Blockchain and DLT Lab
CO Mapped	CO1:Describe the basic concept of Blockchain and Distributed Ledger Technology. CO2:Interpret the knowledge of the Bitcoin network, nodes, keys, wallets and transactions CO3:Implement smart contracts in Ethereum using different development frameworks. CO5:Interpret different Crypto assets and Crypto currencies

Experiment No 6

Aim: Creating Smart Contracts and performing transactions using Solidity and Remix IDE

Theory:

A Smart Contract is a self-executing program stored on the blockchain that automatically enforces the terms of an agreement when predefined conditions are met. It eliminates the need for intermediaries, ensuring transparency, security, and trust between parties. Smart contracts are written in Solidity (for Ethereum-based systems) and executed on the Ethereum Virtual Machine (EVM).

Significance of Smart Contracts in the Ethereum Blockchain:

- Automation: Executes actions automatically when conditions are satisfied.
- Transparency: The contract code and transactions are visible on the blockchain.
- Security: Once deployed, contracts are immutable and tamper-proof.
- Cost-effective: Removes intermediaries and reduces transaction costs.
- Trustless System: Participants can interact without needing to trust each other directly. Smart contracts form the backbone of decentralized applications (DApps) and decentralized finance (DeFi) systems on Ethereum.

Code:\

```
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.0;
```

```
contract PharmaSupply {

    address public manufacturer;
    address public pharmacy;

    struct Medicine
    { string name;
      uint quantity;
      bool shipped;
      bool received;
    }
    Medicine public med;
    constructor(address _pharmacy) {
        manufacturer = msg.sender;
        pharmacy = _pharmacy;
    }
}
```

```

    }

    function addMedicine(string memory _name, uint _quantity) public {
        require(msg.sender == manufacturer, "Only manufacturer allowed"); med
        = Medicine(_name, _quantity, false, false);
    }

    function shipMedicine() public {
        require(msg.sender == manufacturer, "Only manufacturer allowed");
        med.shipped = true;
    }

    function receiveMedicine() public {
        require(msg.sender == pharmacy, "Only pharmacy allowed");
        require(med.shipped == true, "Not shipped yet"); med.received
        = true;
    }

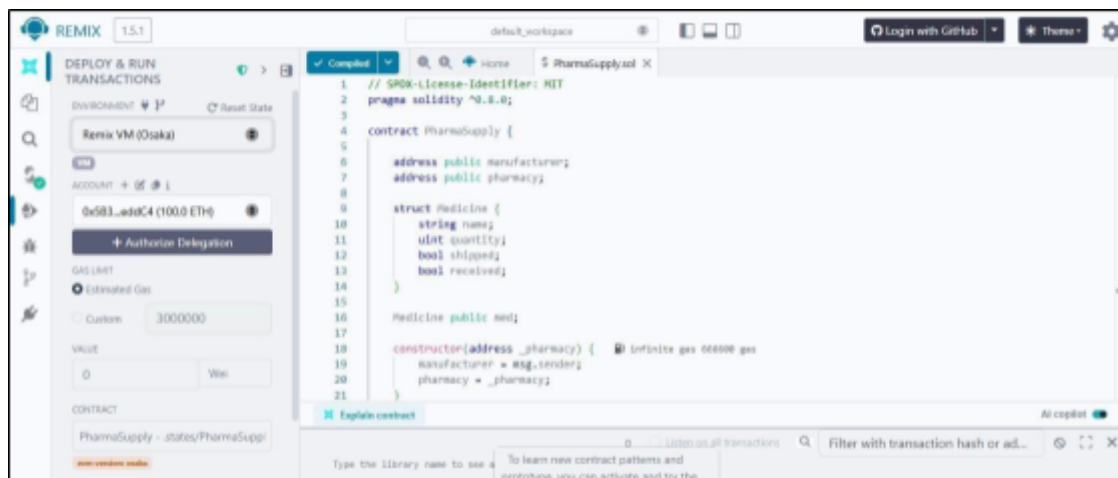
    function getMedicine() public view returns (string memory, uint, bool, bool) { return
        (med.name, med.quantity, med.shipped, med.received);
    }
}

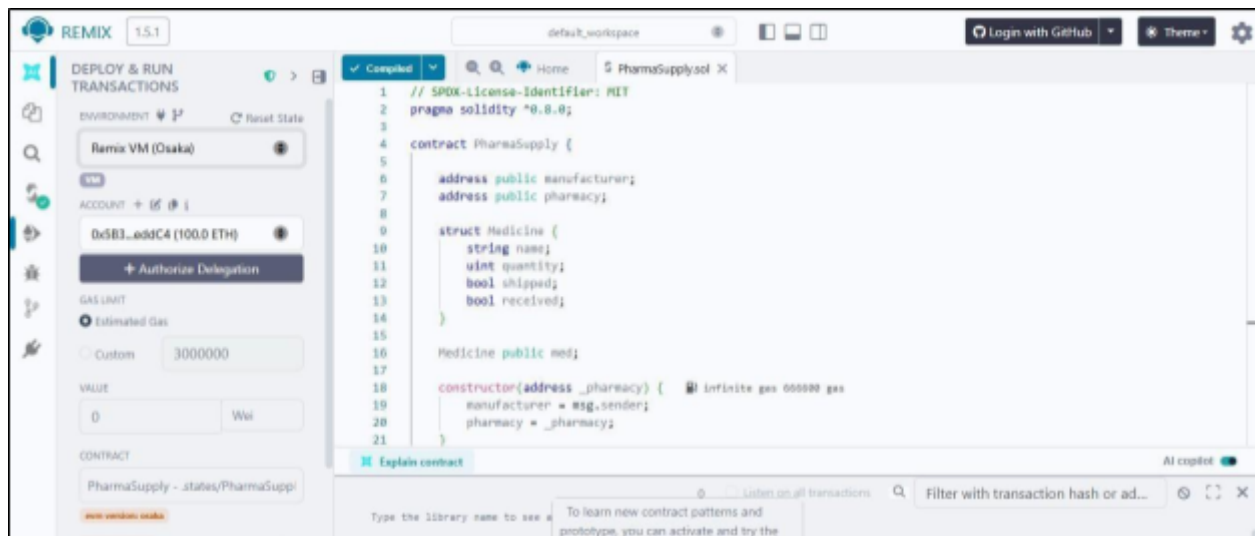
```

Implementation:

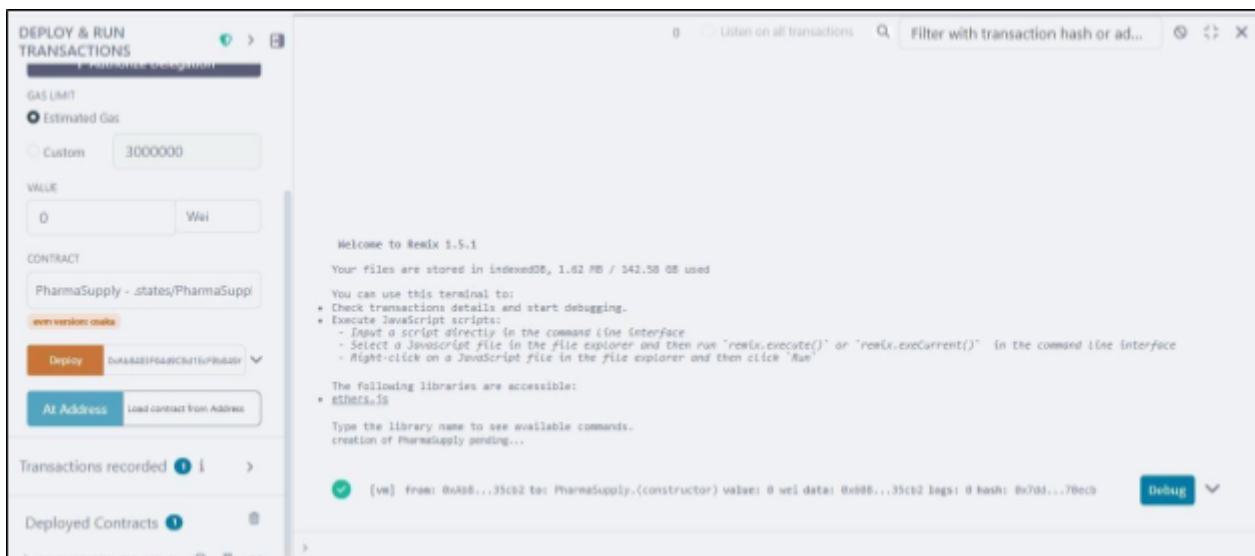
Contract 1: PharmaSupply (Medicine flow)

Contract code

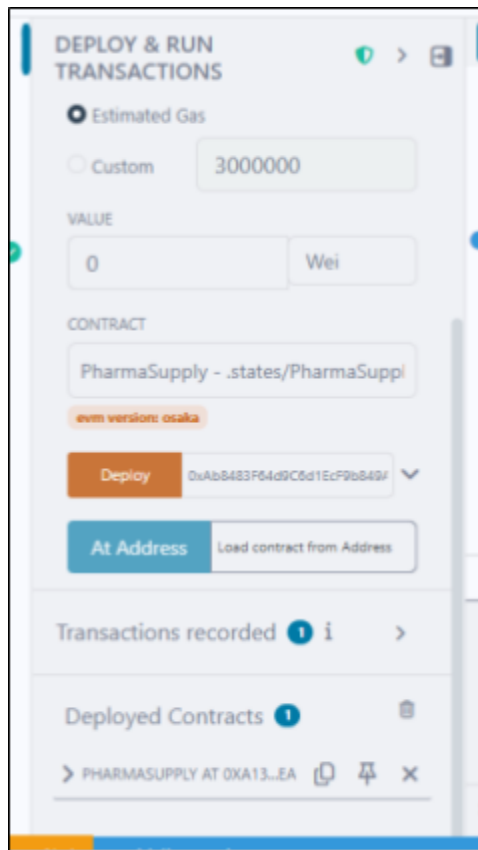




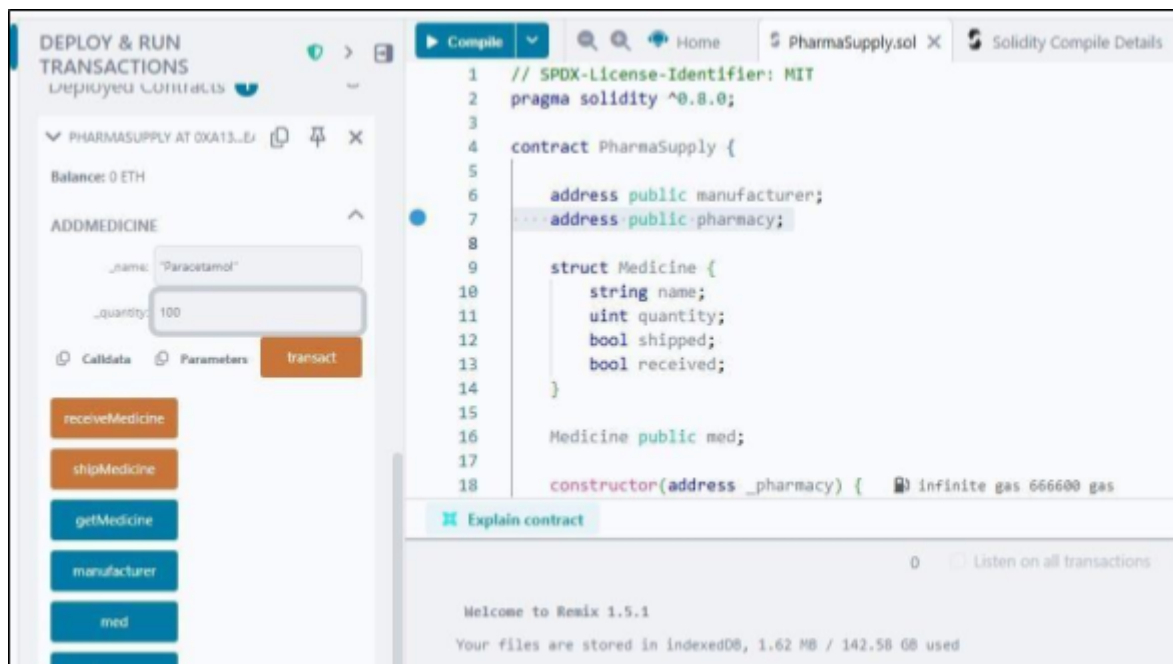
Transaction deployed successfully



Deployed contract



Manufacture adding medicine for shipping



DEPLOY & RUN TRANSACTIONS

▼ PHARMASUPPLY AT 0xA13...
Balance: 0 ETH

ADDMEDICINE

name: "Paracetamol"

quantity: 100

☐ CallData
 ☐ Parameters

Low level interactions

Listen on all transactions

Filter with transaction hash or address

✖

```
[vm] from: 0x583...addC4 to: PharmaSupply.addMedicine(string,uint256) 0x13...eAD95 valae: 0 wei data: 0xa98...00000
logs: 0 hashes: 0xf9a...47e0a
transact to PharmaSupply.addMedicine errored: error occurred: revert.

revert

The transaction has been reverted to the initial state.
Reason provided by the contract: "Only manufacturer allowed".
If the transaction failed for not having enough gas, try increasing the gas limit gently.
```

Debug

✖

```
[vm] from: 0x583...addC4 to: PharmaSupply.addMedicine(string,uint256) 0x13...eAD95 valae: 0 wei data: 0xa98...00000
logs: 0 hashes: 0xc954...3f000
transact to PharmaSupply.addMedicine errored: error occurred: revert.

revert

The transaction has been reverted to the initial state.
Reason provided by the contract: "Only manufacturer allowed".
If the transaction failed for not having enough gas, try increasing the gas limit gently.
```

Debug

✔

```
[vm] from: 0xb08...35cb2 to: PharmaSupply.addMedicine(string,uint256) 0x13...eAD95 valae: 0 wei data: 0xa98...00000
logs: 0 hashes: 0xb32...a5d00
transact to PharmaSupply.addMedicine pending ...

transact to PharmaSupply.addMedicine pending ...
```

Debug

✔

```
[vm] from: 0xb08...35cb2 to: PharmaSupply.addMedicine(string,uint256) 0x13...eAD95 valae: 0 wei data: 0xa98...00000
logs: 0 hashes: 0xc372...ebb30
```

Debug

DEPLOY & RUN TRANSACTIONS

CALLDATA

@ address: 0xAb0483F64d9C6d1Ecf9b649Ae677d03315835cb2

Low level interactions

CALLDATA

Listen on all transactions
Filter with transaction hash or address

```

[call] from: 0xAb0483F64d9C6d1Ecf9b649Ae677d03315835cb2 to: PharmaSupply.manufacturer() data: 0x747...54282

from
  0xAb0483F64d9C6d1Ecf9b649Ae677d03315835cb2

to
  PharmaSupply.manufacturer() 0x133d0247655f02c3a4b136d13bdc130a005

execution cost
  2552 gas (cost only applies when called by a contract)

input
  0x747...54282

output
  0x0000000000000000000000000000000000000000000000000000000000000000

decoded input
  {}

decoded output
  {
    "0": "address: 0xAb0483F64d9C6d1Ecf9b649Ae677d03315835cb2"
  }

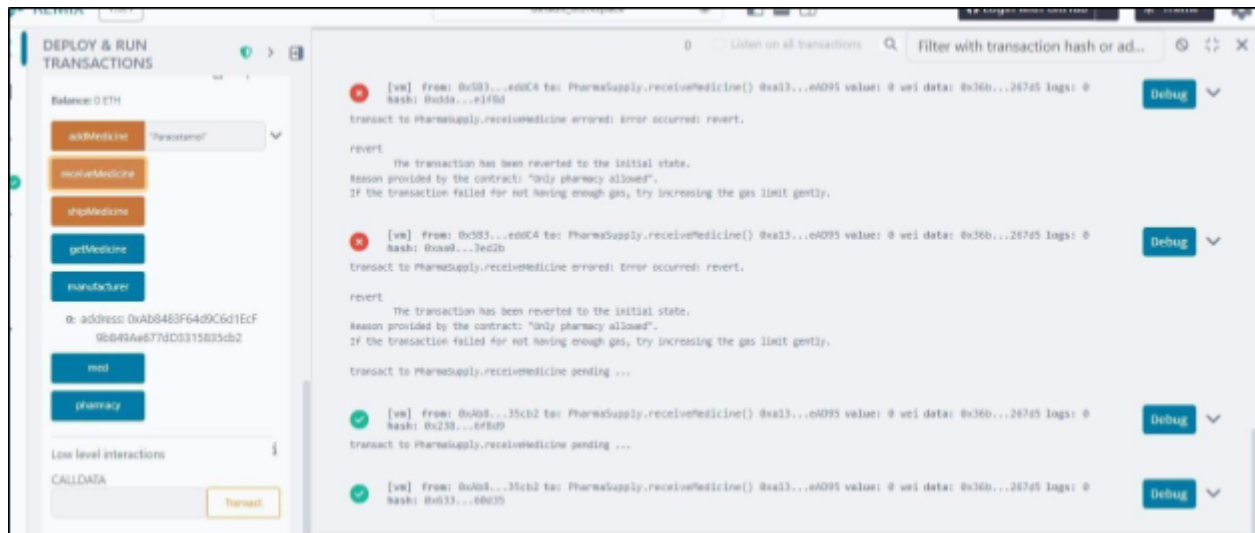
logs
  []

raw logs
  []

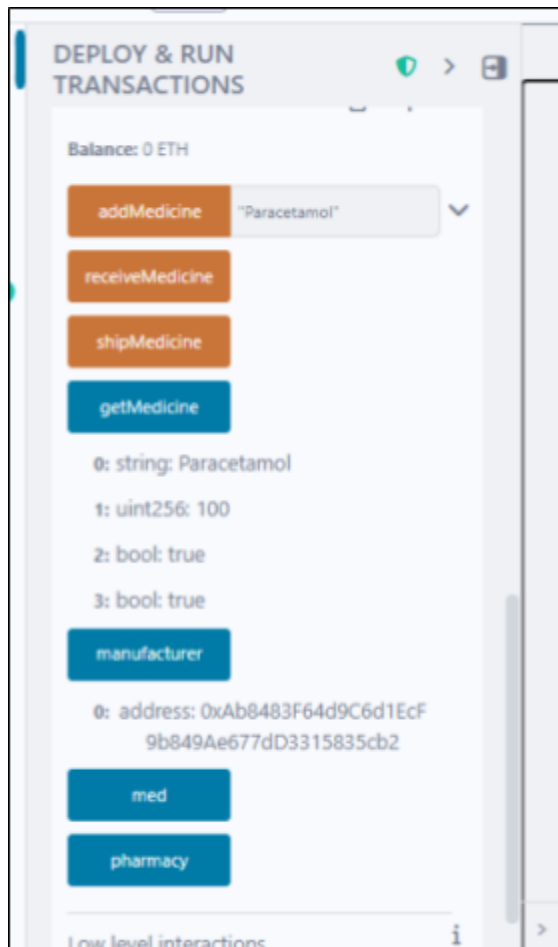
transaction to PharmaSupply.shipMedicine pending ...

[vm] from: 0xAb0...35cb2 to: PharmaSupply.shipMedicine() 0x13...a005 value: 0 wei data: 0x30f...32075 logs: 0
hash: 0xe25...b0946
          
```

Medicine received by pharmacy successfully



getmedicine() we are getting the values of added medicine



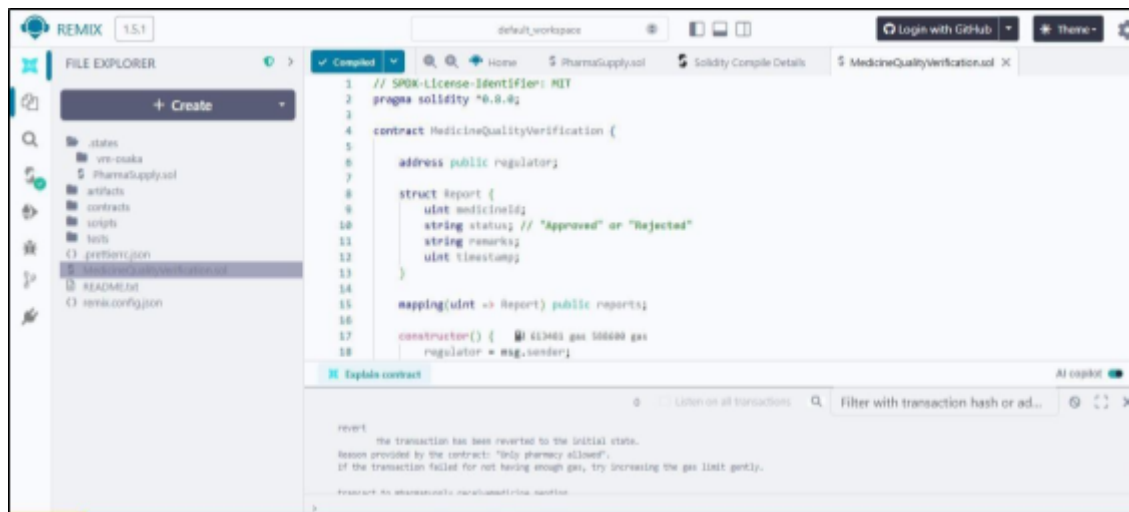
Contract 2: medicine quality verification

While verifying:

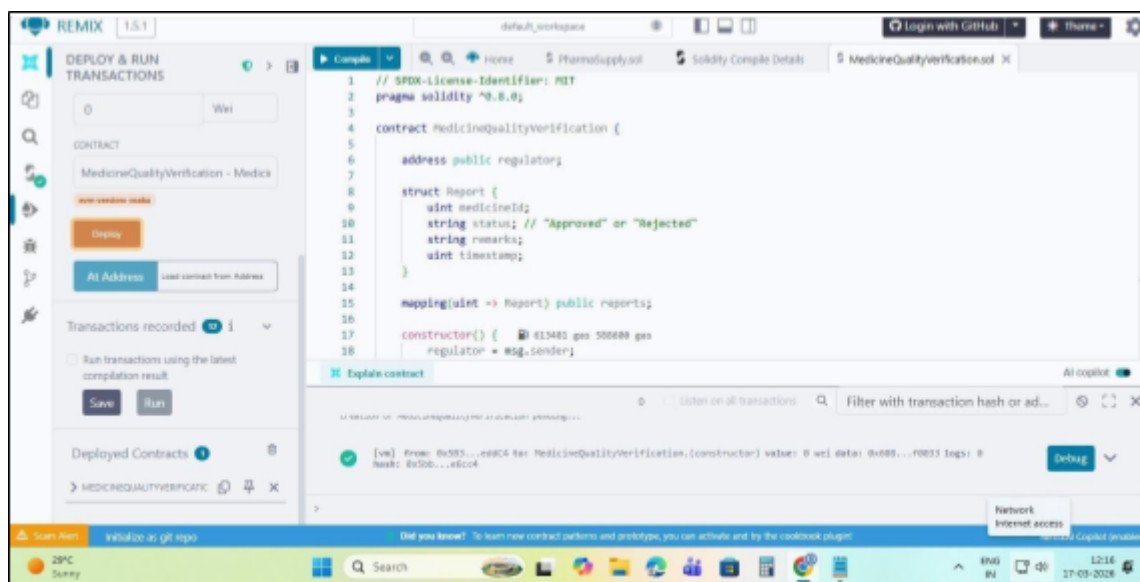
“Here, the regulator verifies the medicine batch and stores its approval status on blockchain.”

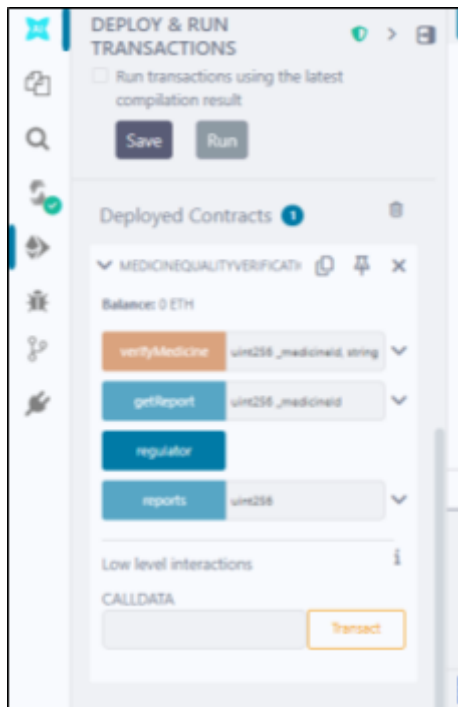
While fetching:

“This function retrieves the verification report, ensuring transparency and immutability.”

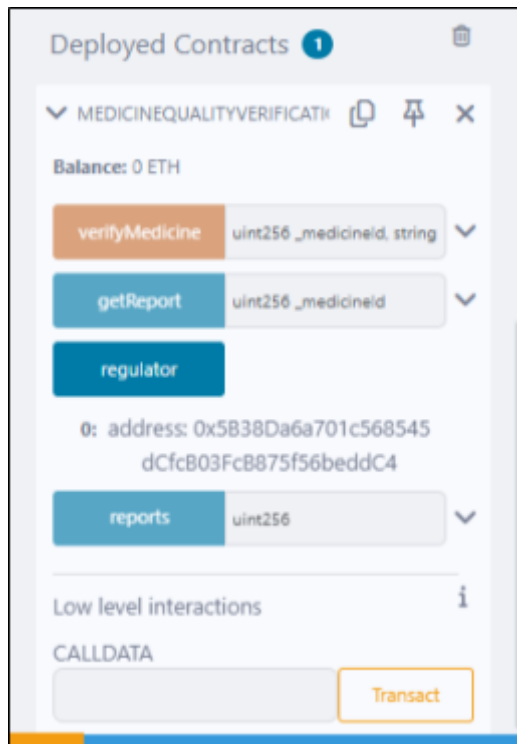


Contract deployed

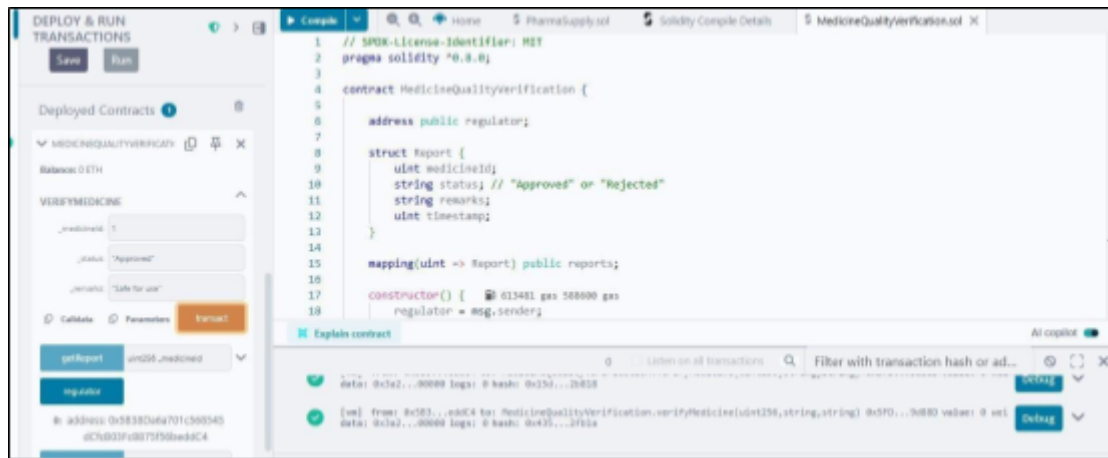




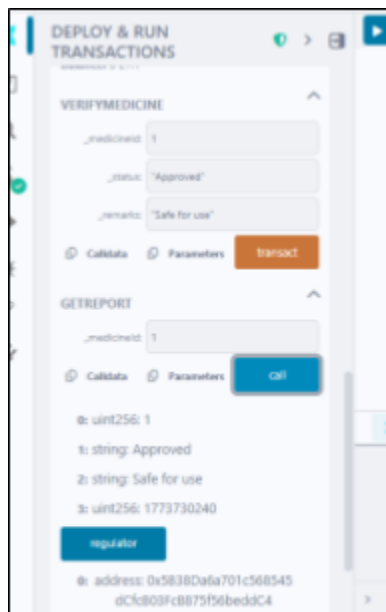
Regulator



Medicine verification done by regulator



getreport

**Conclusion:**

The experiment was successfully completed using Remix IDE, where two Solidity smart contracts were deployed and tested on the Ethereum blockchain. Contract 1 (PharmaSupply) demonstrated the medicine flow from manufacturer to pharmacy, while Contract 2 (MedicineQualityVerification) allowed a regulator to verify and store quality reports on-chain. Role-based access control was enforced using `require` statements, and all transactions were recorded immutably. The experiment successfully demonstrated the use of smart contracts for transparent and trustless pharmaceutical supply chain management.



**VIVEKANAND EDUCATION SOCIETY'S
INSTITUTE OF TECHNOLOGY
Department of Information Technology
Academic Year: 2025-26**

ITC801 : Blockchain & DLT Lab

Experiment 07

Aim: Implement the embedding wallet (Metamask) and transaction using Solidity

Roll No.	07
Name	Vedang V. Wajge
Class	D20A
Subject	ITC801: Blockchain and DLT Lab
CO Mapped	CO1:Describe the basic concept of Blockchain and Distributed Ledger Technology. CO2:Interpret the knowledge of the Bitcoin network, nodes, keys, wallets and transactions CO3:Implement smart contracts in Ethereum using different development frameworks. CO5:Interpret different Crypto assets and Crypto currencies

Experiment No: 7

AIM: Implement the embedding wallet (Metamask) and transaction using Solidity

Theory:

Blockchain is a decentralized and distributed digital ledger that records transactions securely across multiple systems. It ensures transparency, security, and immutability, meaning once data is recorded, it cannot be changed.

Ethereum is a popular blockchain platform that allows the execution of Smart Contracts. Smart contracts are self-executing programs written in Solidity that automatically perform actions when predefined conditions are met, eliminating the need for intermediaries.

MetaMask is a cryptocurrency wallet and browser extension used to interact with blockchain networks. It stores user accounts and private keys securely and allows users to send, receive, and sign transactions. It also acts as a bridge between Remix IDE and the blockchain network.

Testnets such as Sepolia and RSK Testnet are used for testing purposes. They provide free tokens, allowing developers to deploy and test smart contracts without using real cryptocurrency, reducing financial risk.

Remix IDE is a web-based development environment used to write, compile, deploy, and test smart contracts. It provides a user-friendly interface and supports direct integration with MetaMask for deploying contracts on blockchain networks.

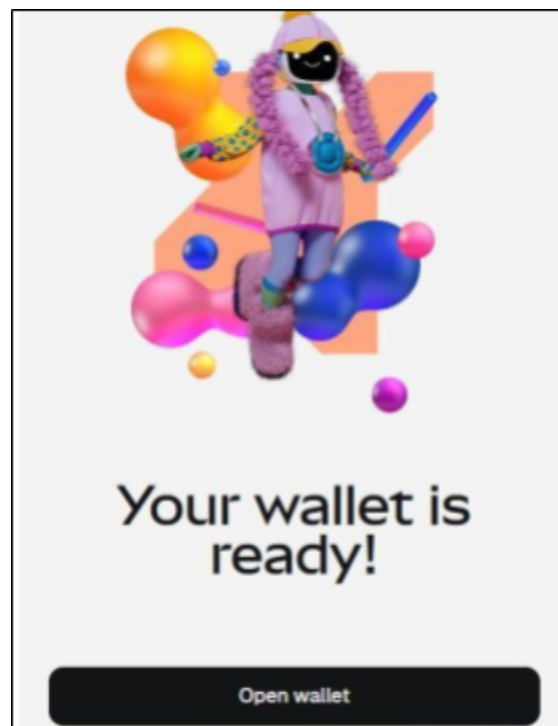
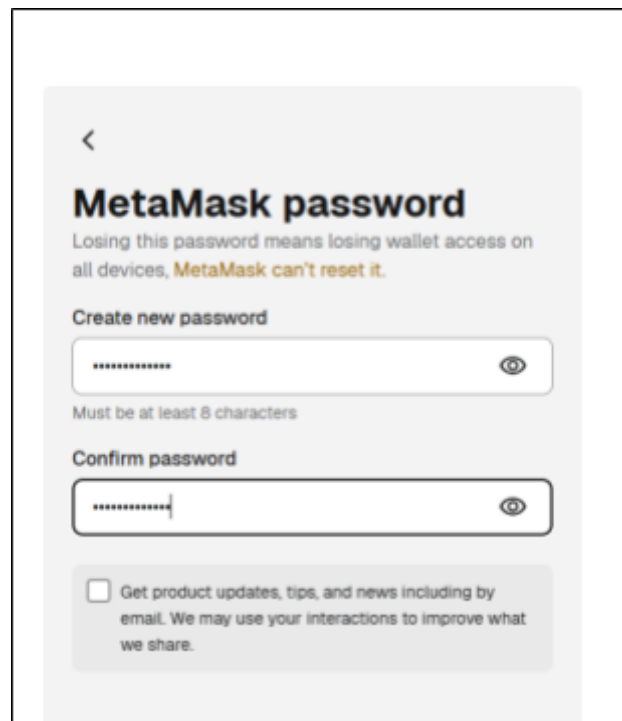
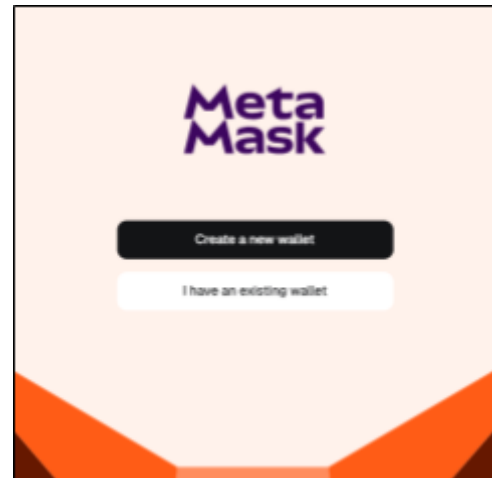
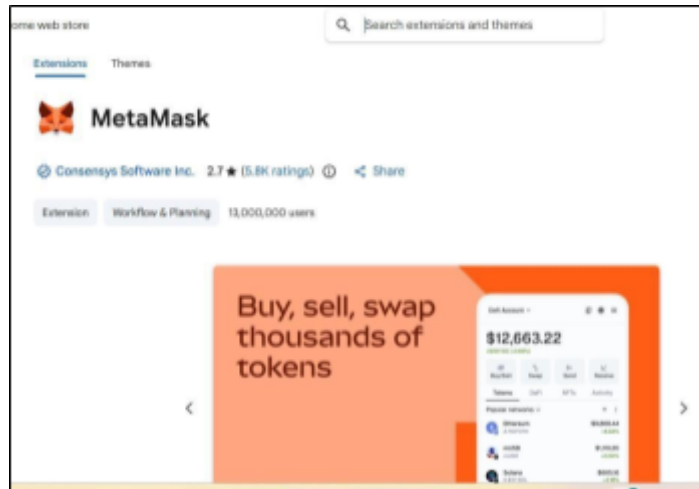
The smart contract deployment process includes writing the contract in Solidity, compiling it to remove errors, connecting MetaMask to Remix, and deploying the contract on a testnet by paying gas fees using test tokens.

A blockchain transaction refers to any interaction with the network, such as deploying a contract or calling its functions. Each transaction requires authentication through a digital signature and is permanently recorded on the blockchain. These transactions can be verified using blockchain explorers.

Implementation

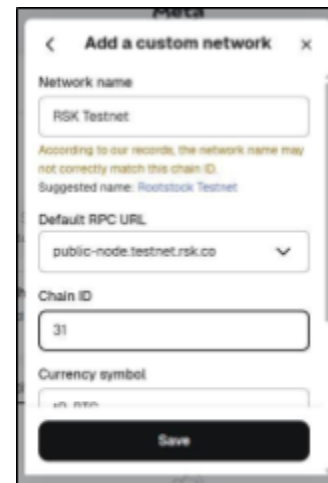
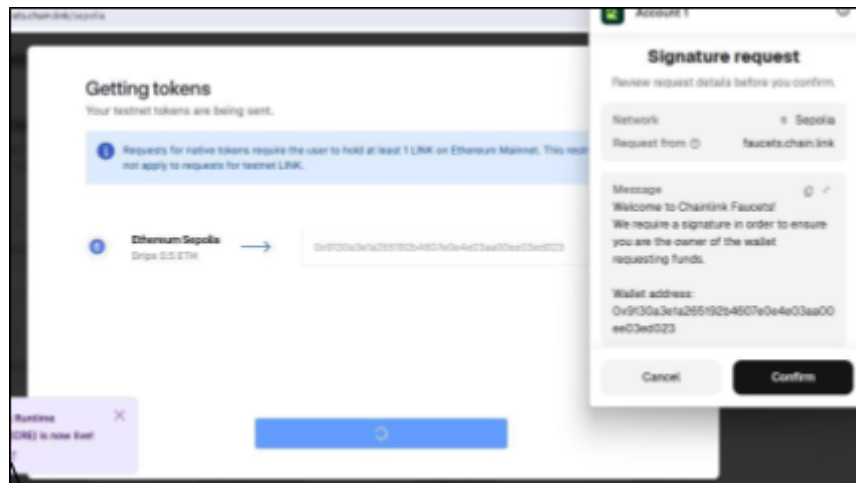
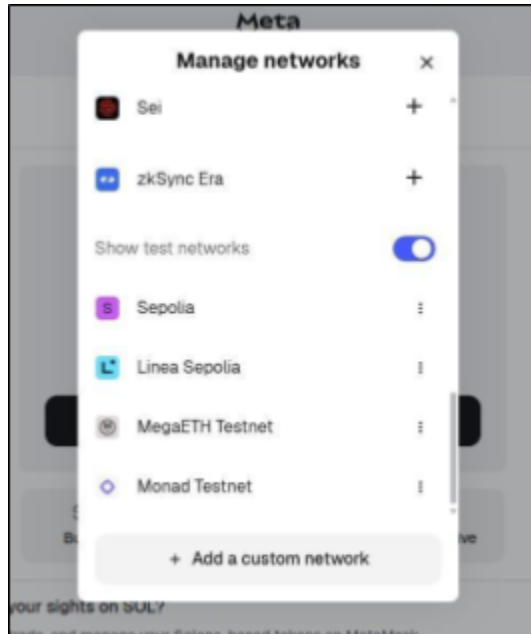
Step 1: Install MetaMask

Install MetaMask extension and create a wallet



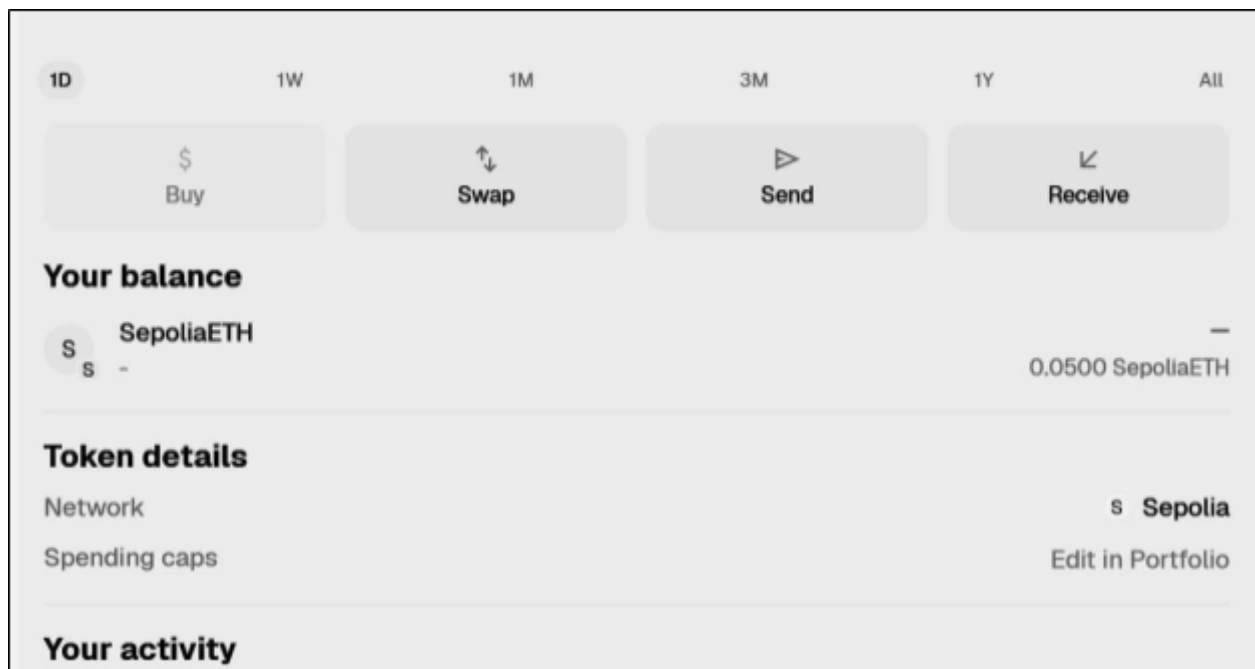
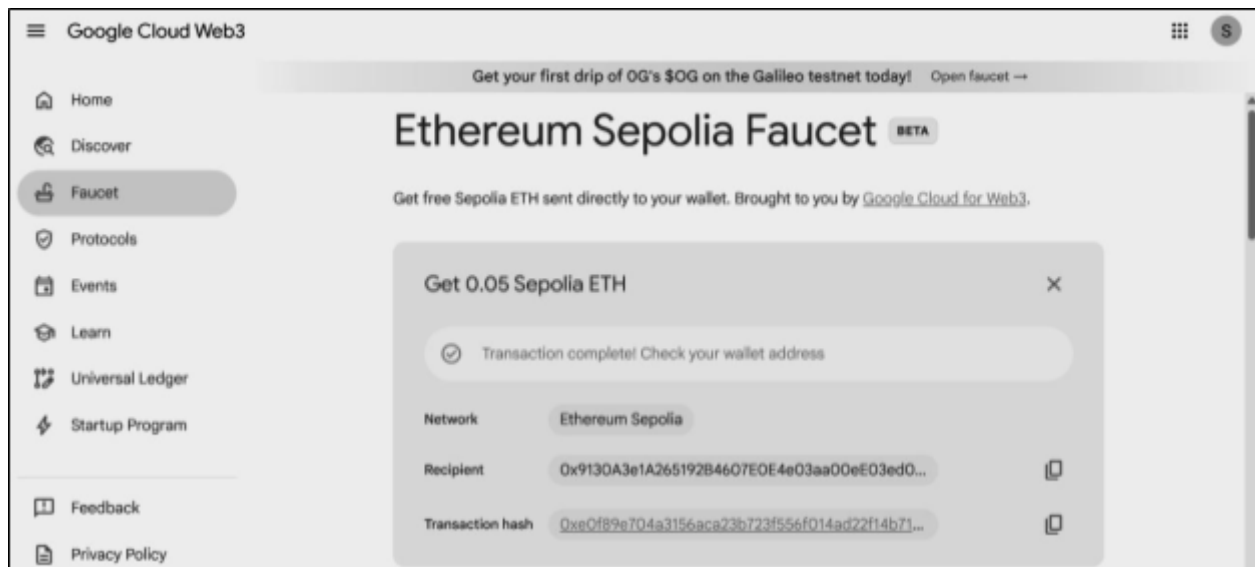
Step 2: Add Sepolia Testnet

Configure MetaMask to connect to Sepolia network.



Step 3: Fund Wallet

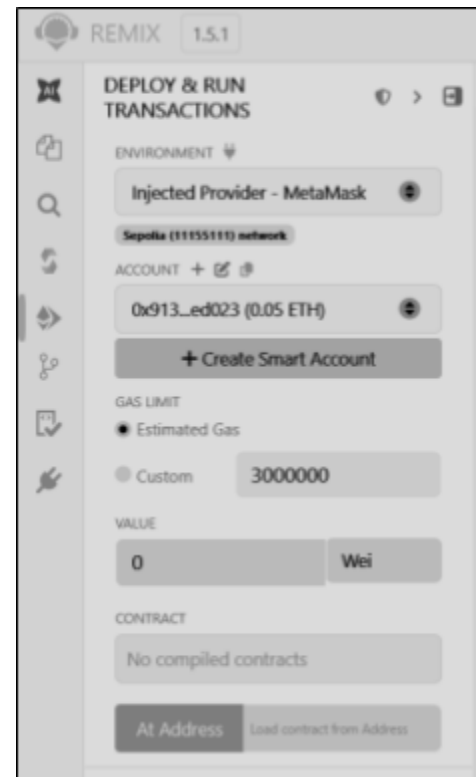
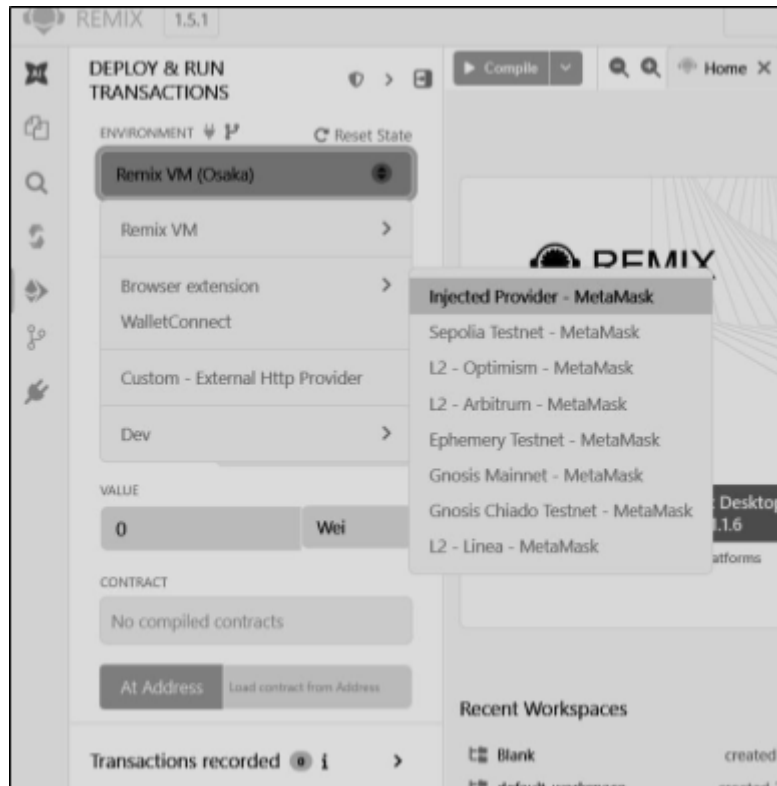
Use faucet to get test ETH



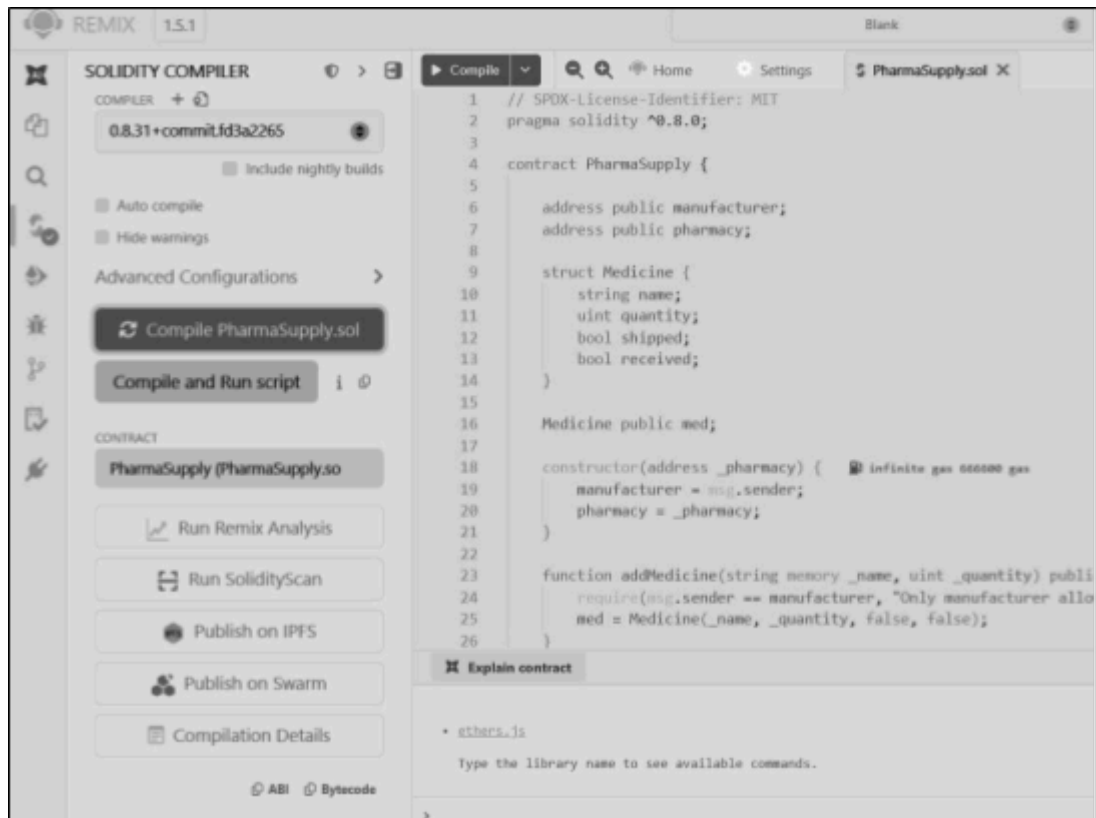
Step4: Connect the Sepolia Testnet / RSK Testnet to Remix IDE

1. In the Remix IDE, select the Environment as Injected Web3.

Note : Injected Web3 connects Remix with the active account in Metamask.

**Compile the Contract**

1. Open Remix
2. Paste your contract code
3. Go to Solidity Compiler (left panel)
4. Select version 0.8.x (match ^0.8.0)
5. Click Compile PharmaSupply.sol

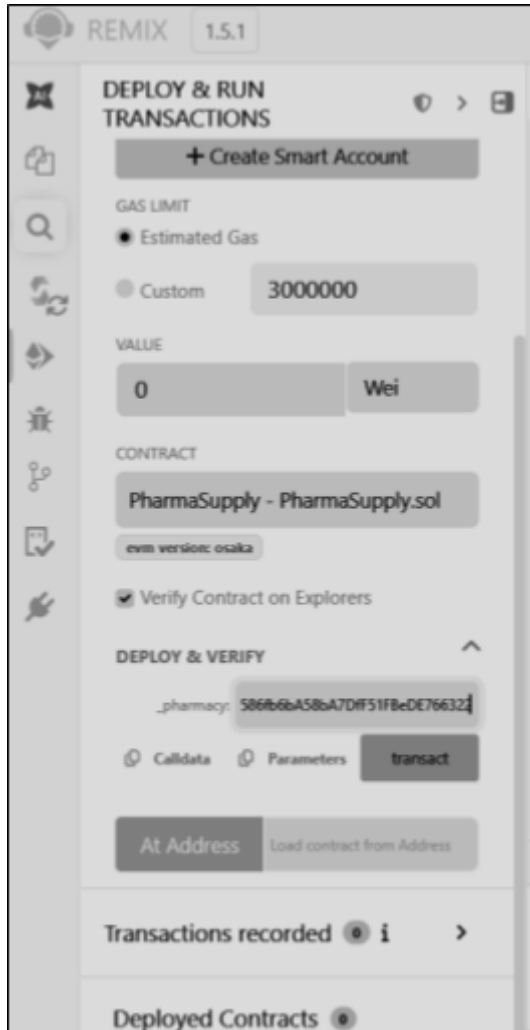


Connect MetaMask (Injected Provider)

1. Go to Deploy & Run Transactions tab
2. In Environment → select Injected Provider - MetaMask
3. MetaMask popup will open → click

Connect Make sure:

- Network = Sepolia Test Network
- You have 0.05 Sepolia ETH



Deploy Contract

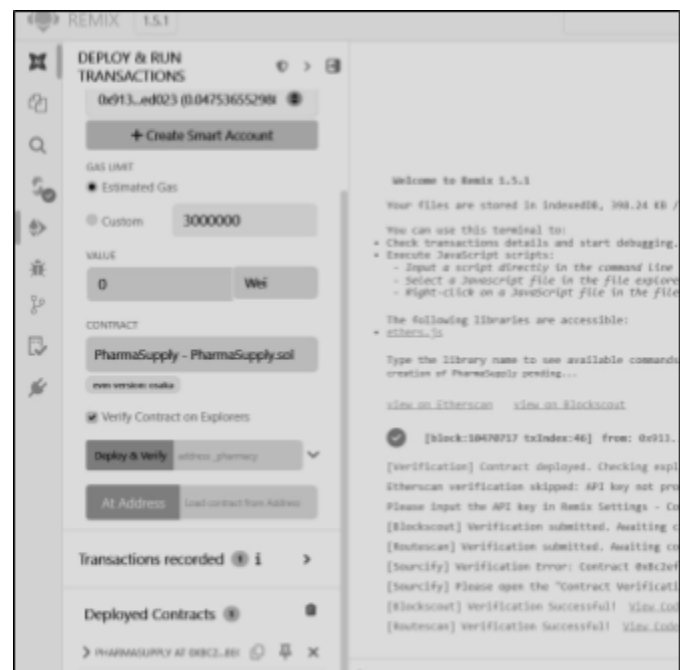
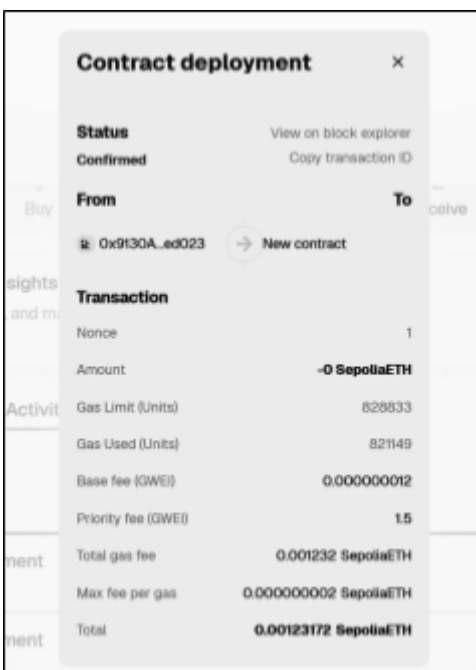
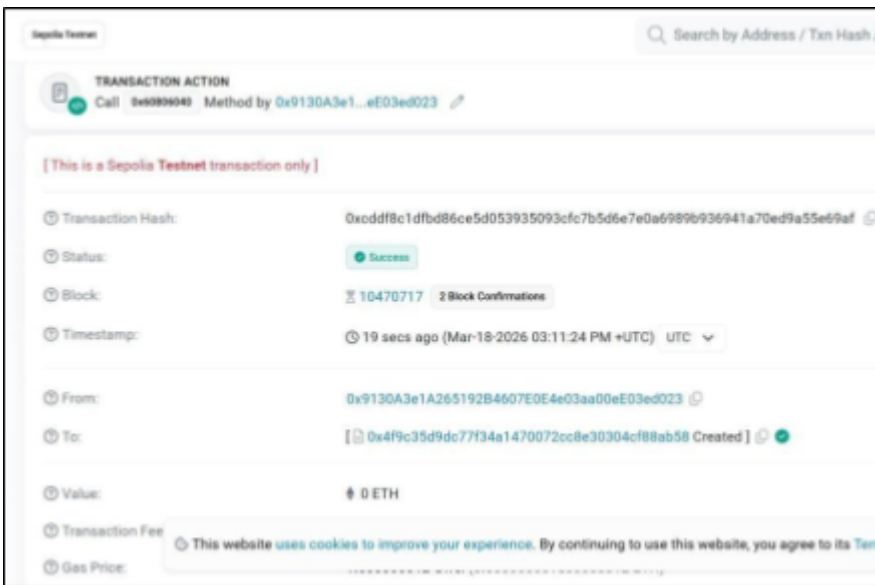
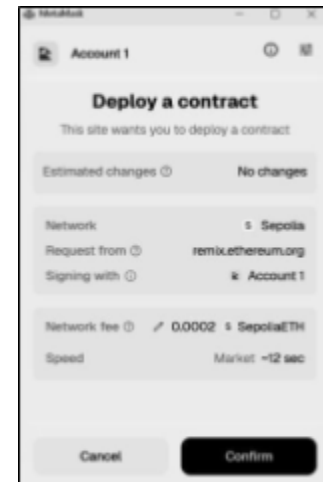
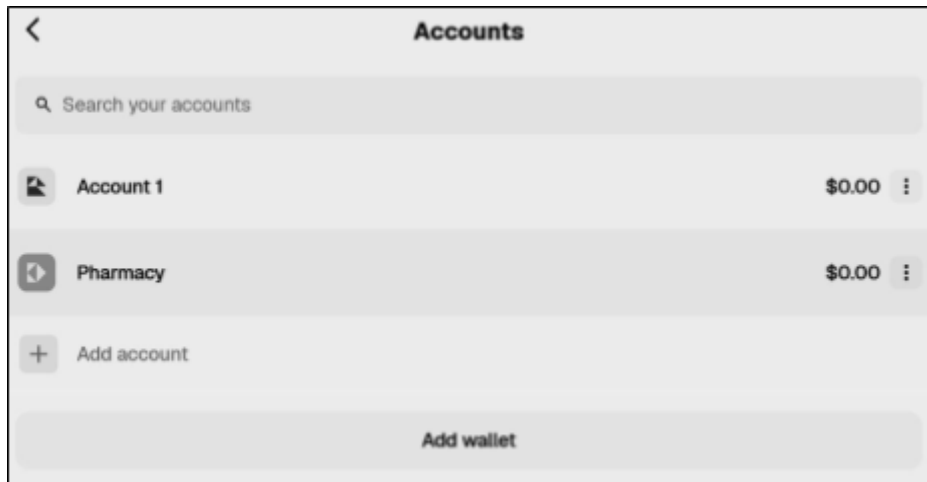
Your constructor needs a pharmacy address

Steps:

1. Copy your second MetaMask account address (pharmacy)
 - MetaMask → Account 2 → Copy address
2. Paste that address in constructor field
3. Click Deploy
4. MetaMask will pop up → Confirm transaction

Wait a few seconds...

After success → your contract appears under Deployed Contracts

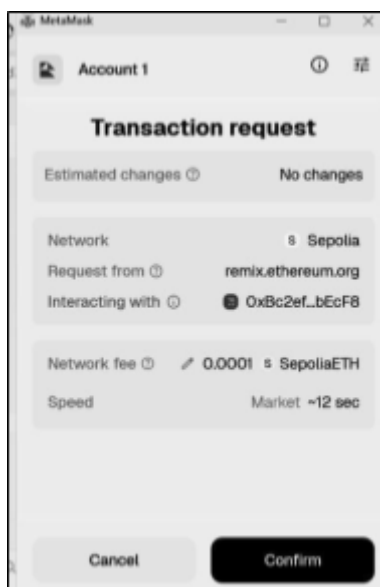
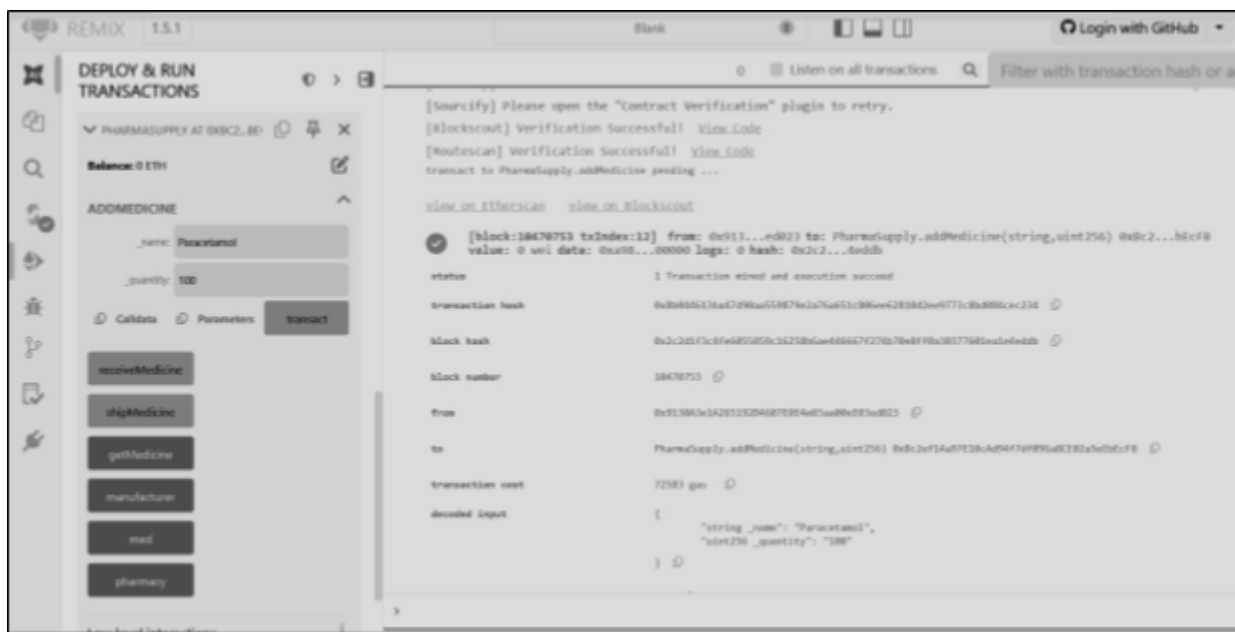


Interact with Contract

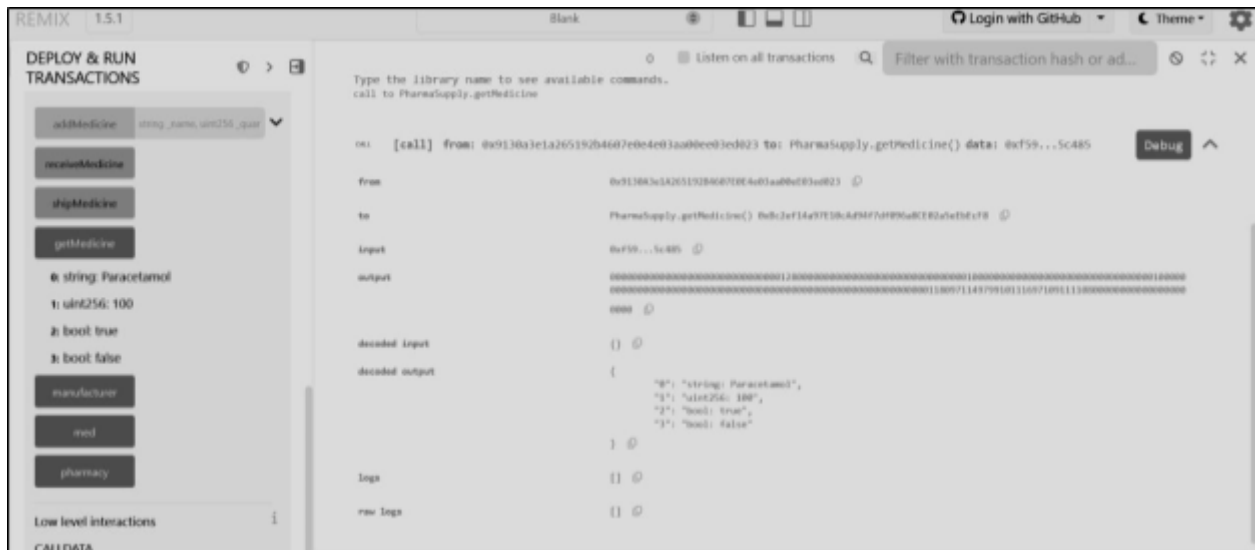
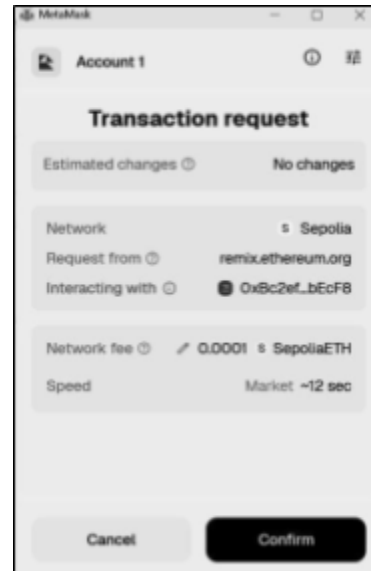
Step 1: Add Medicine (Manufacturer only)

- Make sure MetaMask is on Account 1 (manufacturer)
- In Remix:
 - Enter:
 - `_name: "Paracetamol"`
 - `_quantity: 100`
- Click addMedicine

Confirm in MetaMask

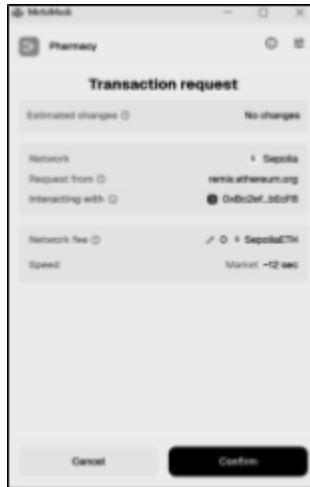


- Still on Account 1
- Click shipMedicine
- Confirm transaction



Step 3: Receive Medicine (Pharmacy)

- Switch MetaMask to Account 2 (pharmacy)
- Click receiveMedicine
- Confirm transaction



Step 4: Check Data

- Click

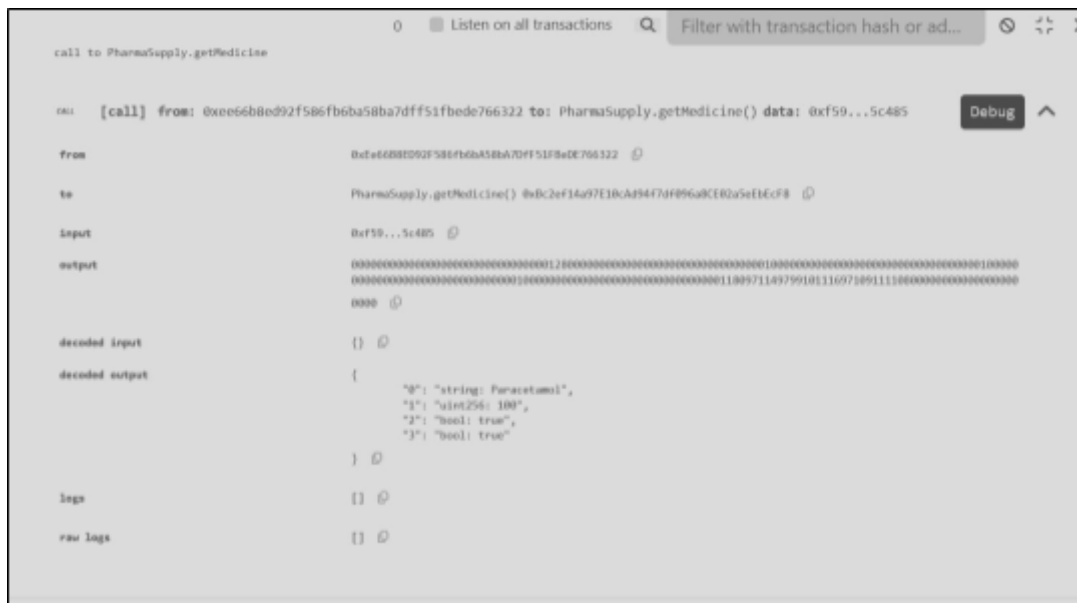
getMedicine Output will

be like: "Paracetamol"

100

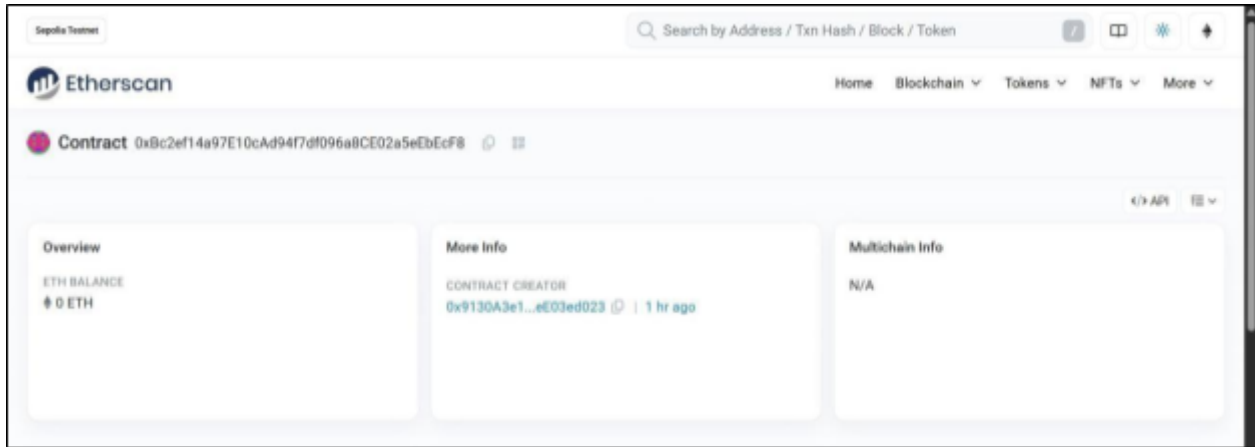
true (shipped)

true (received)

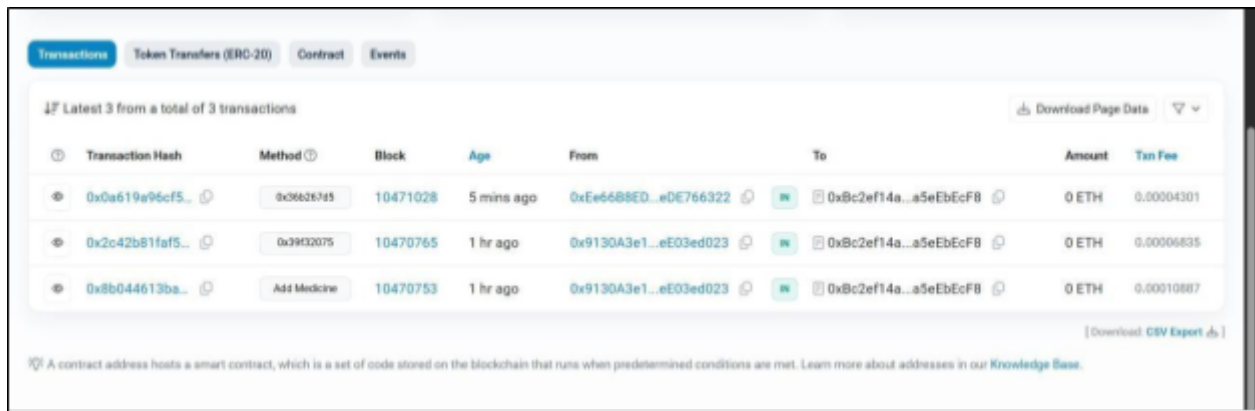


Compiled PharmaSupply.sol

Copy contract address → paste in Etherscan (Sepolia) See all transactions + status



The screenshot shows the Etherscan Sepolia Testnet interface. At the top, there's a search bar and navigation links. The main header identifies the page as a 'Contract' for address 0xBc2ef14a97E10cAd94f7df096a8CE02a5eEbEcF8. Below this, there are three tabs: 'Overview', 'More info', and 'Multichain info'. The 'Overview' tab is active, showing an 'ETH BALANCE' of 0 ETH. The 'More info' tab shows the 'CONTRACT CREATOR' as 0x9130A3e1...eE03ed023, with a timestamp of 1 hr ago. The 'Multichain info' tab shows 'N/A'.



The screenshot shows the 'Transactions' tab for the same contract address. It displays a table of the latest 3 transactions from a total of 3 transactions. The table includes columns for Transaction Hash, Method, Block, Age, From, To, Amount, and Txn Fee. All transactions are successful (status 'W').

Transaction Hash	Method	Block	Age	From	To	Amount	Txn Fee
0x0a619a95cf5...	0x36626745	10471028	5 mins ago	0xEe6688ED...eDE766322	0xBc2ef14a...a5eEbEcF8	0 ETH	0.00004301
0x2c42b81faf5...	0x913075	10470765	1 hr ago	0x9130A3e1...eE03ed023	0xBc2ef14a...a5eEbEcF8	0 ETH	0.00006835
0x8b044613ba...	Add Medicine	10470753	1 hr ago	0x9130A3e1...eE03ed023	0xBc2ef14a...a5eEbEcF8	0 ETH	0.00010887

At the bottom, there is a note: "A contract address hosts a smart contract, which is a set of code stored on the blockchain that runs when predetermined conditions are met. Learn more about addresses in our [Knowledge Base](#)."

Conclusion

The experiment successfully demonstrated MetaMask integration with Remix IDE and execution of blockchain transactions. It provided hands-on experience with smart contracts, ensuring secure and transparent interactions on a testnet.



**VIVEKANAND EDUCATION SOCIETY'S
INSTITUTE OF TECHNOLOGY
Department of Information Technology
Academic Year: 2025-26**

ITC801 : Blockchain & DLT Lab
Experiment 08

Aim: To implement the Blockchain platform Ganache and deploy a simple Solidity smart contract using Metamask and Remix IDE.

Roll No.	07
Name	Vedang V. Wajge
Class	D20A
Subject	ITC801: Blockchain and DLT Lab
CO Mapped	CO2: Interpret the knowledge of the Bitcoin network, nodes, keys, wallets and transactions CO3: Implement smart contracts in Ethereum using different development frameworks. CO5: Interpret different Crypto assets and Crypto currencies

Experiment 8

AIM: Implementing a Blockchain Platform using Ganache

Theory:

1. Blockchain Technology

A blockchain is a distributed, decentralized, and immutable ledger that records transactions across a network of computers. Each record (called a block) contains a cryptographic hash of the previous block, a timestamp, and transaction data, forming a chain. Because every participant holds a copy of the ledger and any alteration to a block invalidates all subsequent blocks, the data stored on a blockchain is highly tamper-resistant and transparent.

Key properties of blockchain:

- Decentralization: No single entity controls the network; authority is distributed across nodes.
- Immutability: Once data is written to the blockchain, it cannot be altered or deleted.
- Transparency: All transactions are visible to network participants.
- Consensus: Nodes agree on the state of the ledger through consensus mechanisms (e.g., Proof of Work, Proof of Stake).
- Security: Cryptographic hashing (SHA-256) ensures data integrity.

2. Ethereum & Smart Contracts

Ethereum is an open-source, decentralized blockchain platform that supports programmable transactions through Smart Contracts. A smart contract is a self-executing program stored on the blockchain whose terms are written directly into code. Once deployed, a smart contract runs automatically when predefined conditions are met, without the need for intermediaries.

Smart contracts in Ethereum are written in Solidity, a statically-typed, contract-oriented programming language. They are compiled into bytecode and deployed to the Ethereum Virtual Machine (EVM), which executes the contract logic on every node in the network.

Key concepts:

- Gas: A unit measuring the computational effort required to execute operations. Users pay gas fees in ETH to incentivize miners/validators.
- ABI (Application Binary Interface): Defines how to interact with the deployed contract — the function signatures and data encodings.
- Address: Every deployed contract gets a unique Ethereum address on the blockchain.
- msg.sender: A global variable in Solidity that holds the address of the account that called the current function.

3. Ganache

Ganache (by Truffle Suite / Consensys) is a personal blockchain for Ethereum development. It simulates

a full Ethereum node locally on your machine, providing:

- 10 pre-funded test accounts, each loaded with 100 ETH.
- Instant mining — transactions are confirmed immediately (no wait time).
- A local RPC server (default: <http://127.0.0.1:7545>) that tools like MetaMask and Remix can connect to.
- A GUI showing real-time accounts, blocks, transactions, and logs for easy inspection.

Ganache is used exclusively for development and testing. It allows developers to deploy and test contracts without spending real ETH or waiting for mainnet/testnet confirmations.

4. MetaMask

MetaMask is a cryptocurrency wallet and browser extension that acts as a bridge between a web browser and the Ethereum blockchain. It allows users to manage Ethereum accounts, sign transactions, and interact with decentralized applications (dApps) directly from the browser. In development, MetaMask is configured to point to a local Ganache network instead of the Ethereum mainnet, enabling safe testing with dummy ETH.

5. Remix IDE

Remix IDE is a web-based integrated development environment for Solidity smart contract development, available at remix.ethereum.org. It provides a code editor, Solidity compiler, contract deployer, and debugger in a single interface. Remix can connect to various environments including the built-in JavaScript VM, Hardhat, or an Injected Provider (MetaMask), the last of which routes deployment through MetaMask to whichever network (Ganache, testnet, or mainnet) MetaMask is currently connected to.

6. Supply Chain Use Case

A supply chain smart contract automates the tracking of goods as they move between parties — in this experiment, a pharmaceutical product (medicine) passing through four stages: Manufacturer, Distributor, Retailer, and Customer. Each stage updates the contract state on-chain, creating a transparent and tamper-proof audit trail. This eliminates the need for a trusted central authority to verify ownership transfers and ensures all parties see the same data.

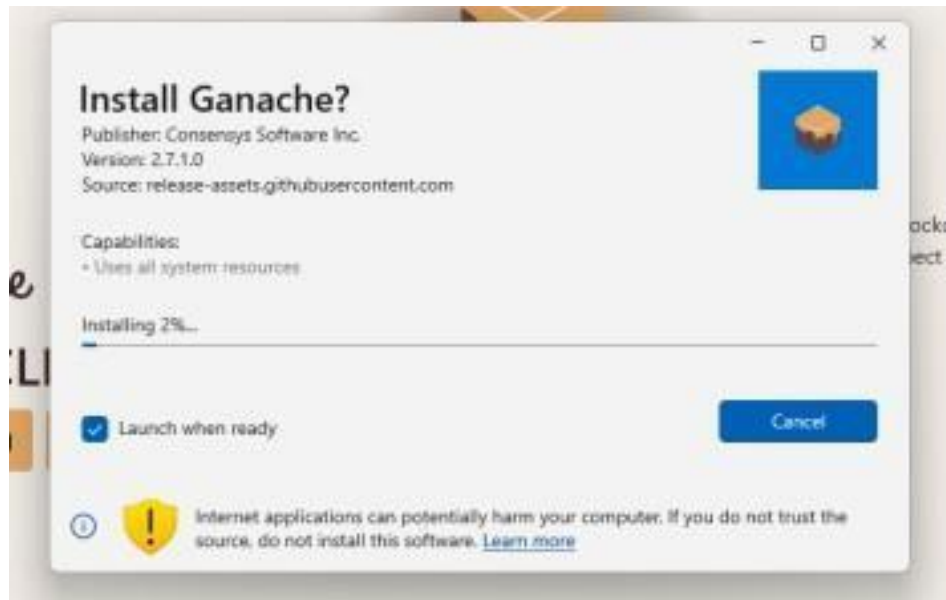
Implementation:

Step 1: Install Ganache

Ganache is a personal blockchain for Ethereum development used to deploy contracts, develop applications, and run tests.

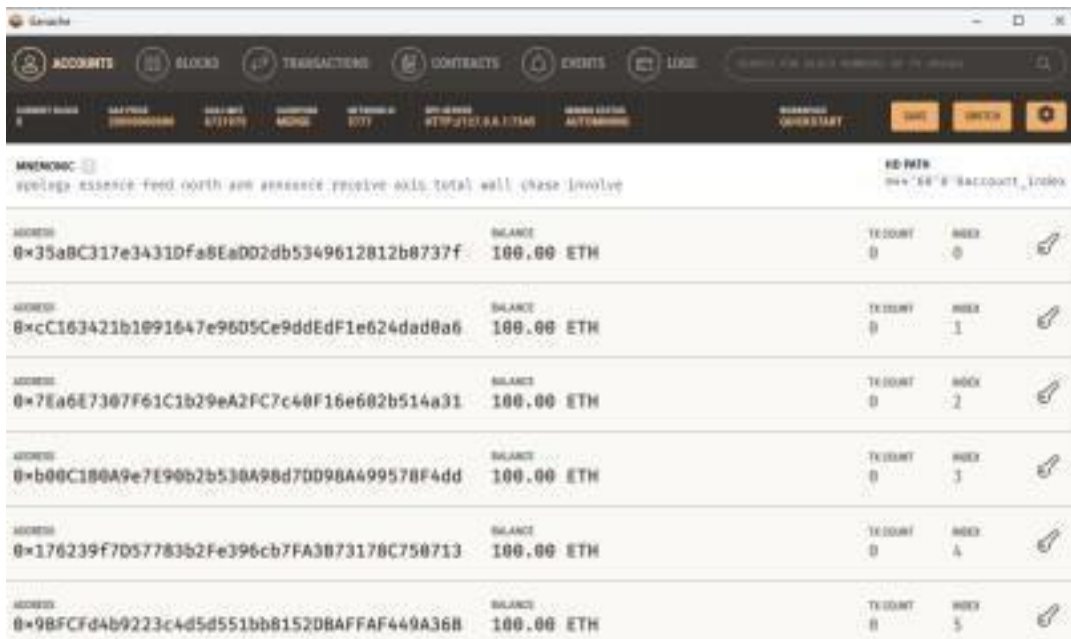
1. Download Ganache v2.7.1 from the official Truffle Suite website (release.assets.githubusercontent.com).

2. Run the installer. The installation prompt shows publisher as Consensys Software Inc.
3. Check 'Launch when ready' and click Install.



4. Once installed, open Ganache and click Quickstart Ethereum to start a local blockchain.





Note the following details from Ganache for later use:

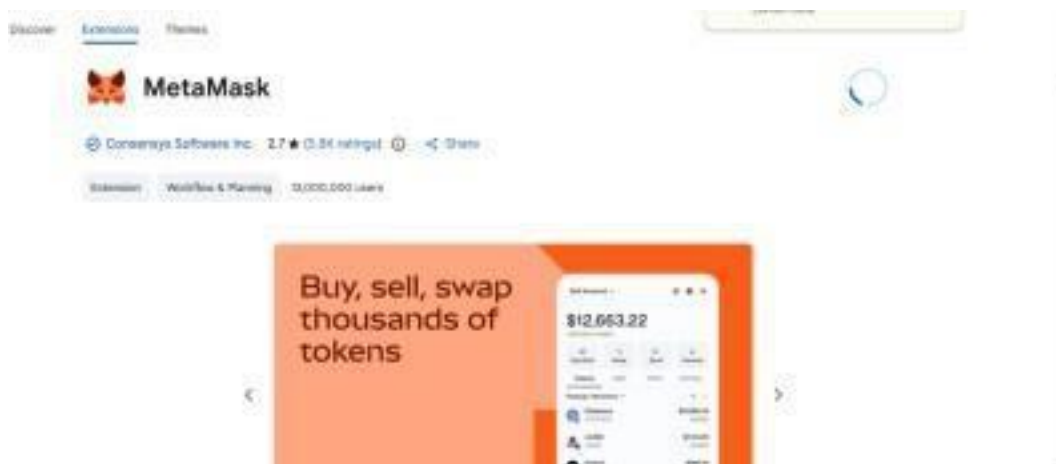
- RPC Server URL: <http://127.0.0.1:7545>
- Network ID: 5777
- Private keys of accounts (click the key icon next to each address)

Step 2: Install & Setup MetaMask

MetaMask is a browser extension that serves as an Ethereum wallet and allows interaction with decentralized applications.

2.1 Install MetaMask

5. Open Google Chrome and go to the Chrome Web Store.
6. Search for MetaMask (by Consensys Software Inc.) and click Add to Chrome.



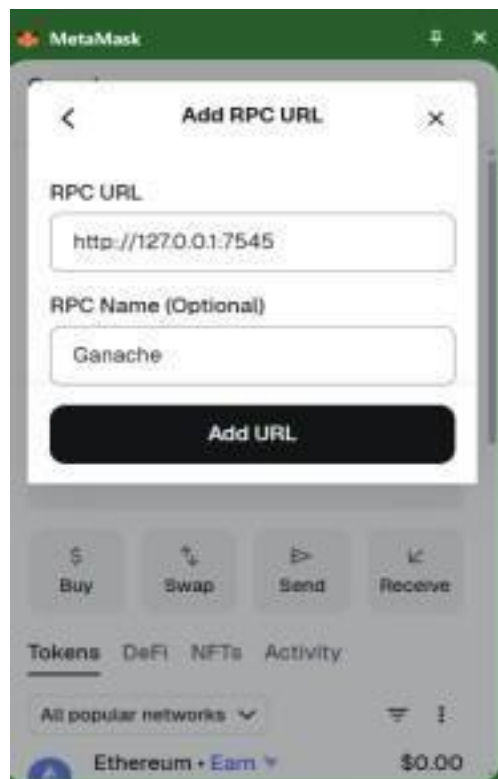
7. After installation, click 'Create a new wallet' on the MetaMask welcome screen.

8. Set a password and save the Secret Recovery Phrase securely.



2.2 Add Ganache Network to MetaMask

9. In MetaMask, click the network dropdown and select 'Add a custom network'.
10. Enter the following details for the RPC
URL: RPC URL: `http://127.0.0.1:7545`
RPC Name: Ganache
11. Click 'Add URL'.



12. Fill in the remaining custom network details:
 - Network Name: Localhost 8545

- Default RPC URL: Ganache (127.0.0.1:7545)
- Chain ID: 1337
- Currency Symbol: ETH

13. Click Save.



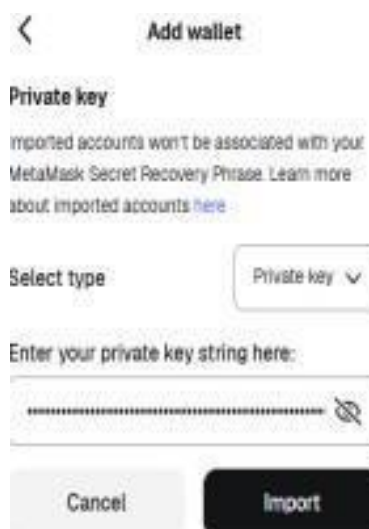
The screenshot shows the 'Add a custom network' screen in the MetaMask mobile app. The title bar at the top says 'Add a custom network' with a back arrow on the left and a close 'X' on the right. Below the title, there are five input fields: 'Network name' with the text 'Localhost 8545', 'Default RPC URL' with a dropdown menu showing 'Ganache' and '127.0.0.1:7545', 'Chain ID' with the text '1337', 'Currency symbol' with the text 'ETH', and 'Block explorer URL' which is empty. At the bottom of the form is a large black button with the word 'Save' in white text.

2.3 Import Ganache Accounts into MetaMask

14. In Ganache, click the key icon next to an account address to reveal its private key.

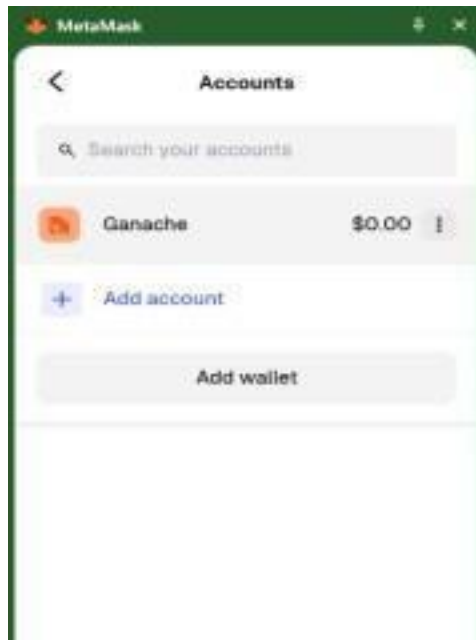
15. In MetaMask, go to Accounts > Add wallet > Import using Private Key.

16. Paste the private key and click Import.



The screenshot shows the 'Add wallet' screen in the MetaMask mobile app. The title bar at the top says 'Add wallet' with a back arrow on the left. Below the title, there is a 'Private key' section with a warning message: 'Imported accounts won't be associated with your MetaMask Secret Recovery Phrase. Learn more about imported accounts [here](#).' Below this is a 'Select type' dropdown menu with 'Private key' selected. Underneath is a text input field with the placeholder text 'Enter your private key string here:' and a small eye icon to toggle visibility. At the bottom are two buttons: a grey 'Cancel' button and a black 'Import' button with white text.

17. The imported account (labeled 'Ganache') now appears in MetaMask under the Localhost 8545 network.



Step 3: Create the Smart Contract in Remix IDE

Remix IDE is a web-based Solidity development environment at remix.ethereum.org.

18. Open remix.ethereum.org in the browser.
19. In the File Explorer, create a new file named SupplyChain.sol.
20. Write the following smart contract for a pharmaceutical supply chain:

```
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.0;

contract PharmaSupplyChain {
    struct Medicine {
        uint id;
        string name;
        uint
        quantity;
        address manufacturer;
        address distributor;
        address retailer;
        address customer;
    }
    mapping(uint => Medicine) public medicines;
```

```
// Step 1: Manufacturer adds medicine
function addMedicine(uint _id, string memory _name, uint _quantity) public {
    medicines[_id] = Medicine(_id, _name, _quantity,
    msg.sender, address(0), address(0), address(0));
}

// Step 2: Distributor receives
function sendToDistributor(uint _id) public {
    medicines[_id].distributor = msg.sender;
}

// Step 3: Retailer receives
function sendToRetailer(uint _id) public {
    medicines[_id].retailer = msg.sender;
}

// Step 4: Customer buys
function purchase(uint _id) public {
    medicines[_id].customer = msg.sender;
}

// View data
function getMedicine(uint _id) public view returns (
    uint, string memory, uint, address, address, address, address) {
    Medicine memory m = medicines[_id];
    return (m.id, m.name, m.quantity, m.manufacturer,
    m.distributor, m.retailer, m.customer);
}
}
```

Step 4: Compile the Smart Contract

21. In Remix, click the Solidity Compiler icon on the left sidebar.
22. Select compiler version 0.8.x.
23. Click Compile SupplyChain.sol.
24. A green checkmark confirms successful compilation.



Step 5: Connect Remix IDE with MetaMask

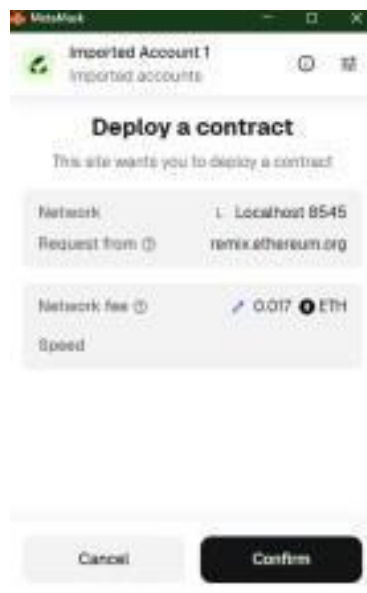
25. In Remix, navigate to the Deploy & Run Transactions tab.
26. In the Environment dropdown, select Injected Provider - MetaMask.
27. MetaMask will prompt a connection request — click Connect.
28. Ensure MetaMask is switched to the Localhost 8545 (Ganache) network.
29. The imported Ganache account will now appear as the active account in Remix.

Step 6: Deploy the Smart Contract

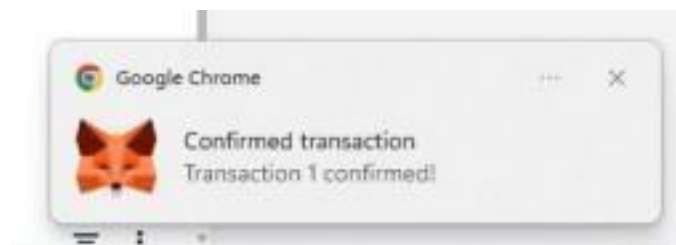
30. In the Deploy & Run panel, confirm PharmaSupplyChain is selected under Contract.
31. Click Deploy. MetaMask will open a confirmation popup.



32. Review the Network fee and click Confirm in MetaMask.



33. A 'Confirmed transaction' notification from Chrome confirms the deployment was successful.



Step 7: Verify Transaction in Ganache

After deployment, Ganache reflects the transaction. The deploying account's balance decreases due to gas fees paid.

ACCOUNT 1

BALANCE

LAST TX

LAST TX

ETH BAL

ETH BAL

ETH BAL

ETH BAL

ETH BAL

ETH BAL

1

100.00000000

0/21175

0/21175

0/21175

0/21175

0/21175

0/21175

0/21175

0/21175

MNEMONIC

apology essence feed north arm announce receive axis total wall chase involve

HD PATH

m/44'/0'/0'/0'

ADDRESS

BALANCE

TX COUNT

INDEX

0x35aBC317e3431Dfa8EaD02db5349612812b0737f

99.94 ETH

1

0

ADDRESS

BALANCE

TX COUNT

INDEX

0xcC163421b1091647e96D5Ce9ddEdF1e624dad8a6

100.00 ETH

0

1

MNEMONIC

apology essence feed north arm announce receive axis total wall chase involve

HD PATH

m/44'/0'/0'/0'

ADDRESS

BALANCE

TX COUNT

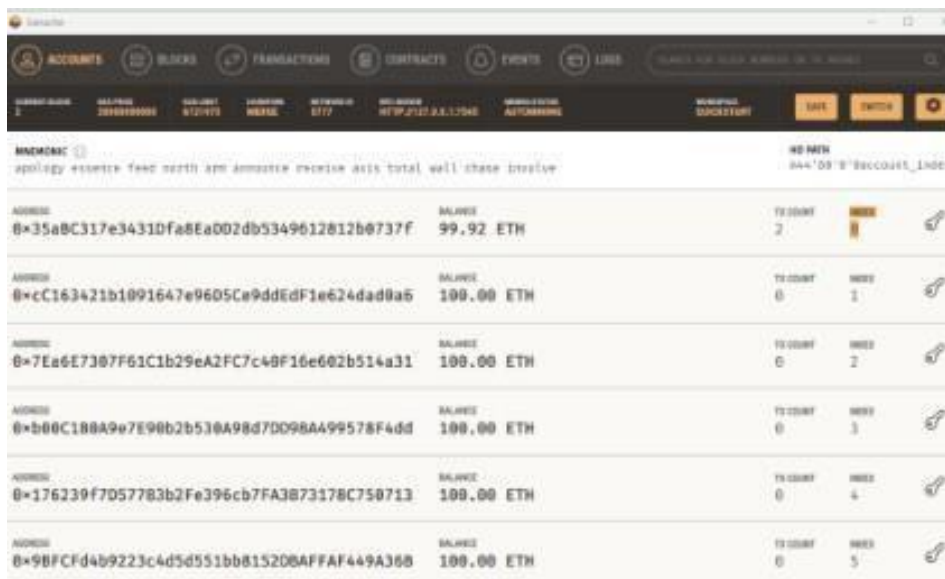
INDEX

0x35aBC317e3431Dfa8EaD02db5349612812b0737f

99.92 ETH

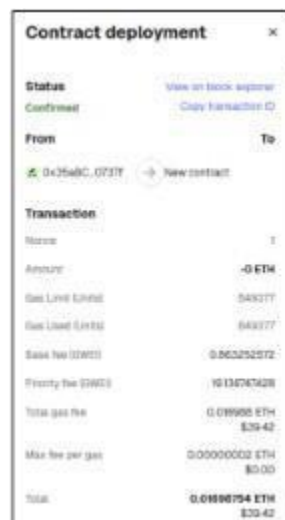
2

0



The contract deployment details in MetaMask show:

- Status: Confirmed
- From: 0x35aBC...0737f → To: New contract
- Gas Used: 849,377 units
- Total gas fee: 0.016988 ETH
- Account balance after: 99.92 ETH



Step 8: Interact with the Smart Contract

After deployment, the contract functions appear in the Remix Deploy & Run panel under Deployed Contracts.

Transaction Flow — Pharma Supply Chain

Account 1 (Manufacturer) — Call addMedicine:

```
addMedicine(1, "Paracetamol", 100)
```

Account 2 (Distributor) — Switch account in MetaMask, then call:

```
sendToDistributor(1)
```

Account 3 (Retailer) — Switch account in MetaMask, then call:

```
sendToRetailer(1)
```

Account 4 (Customer) — Switch account in MetaMask, then call:

```
purchase(1)
```

Call getMedicine to verify the full supply chain:

```
getMedicine(1)
```

Expected output from getMedicine(1):

- id: 1
- name: Paracetamol
- quantity: 100
- manufacturer: 0x35aBC...0737f (Account 1 address)
- distributor: 0xC163...d0a6 (Account 2 address)
- retailer: 0x7Ea6E...a31 (Account 3 address)
- customer: 0xb00C1...4dd (Account 4 address)

Output

- Medicine batch created successfully on the Ganache local blockchain.
- Complete supply chain flow executed: Manufacturer → Distributor → Retailer → Customer.
- All transactions confirmed and visible in Ganache with updated block counts and reduced ETH balances.
- Smart contract deployed with status Confirmed; gas fees deducted correctly.

Conclusion

The experiment was successfully completed using Ganache v2.7.1, MetaMask, and Remix IDE. A pharmaceutical supply chain smart contract (PharmaSupplyChain) was written in Solidity, compiled, and deployed to the local Ganache blockchain via MetaMask. The contract tracked a medicine batch through all four stages of the supply chain — manufacturer, distributor, retailer, and customer — with each stage recorded on-chain. Transaction details were verified in both MetaMask and the Ganache environment, confirming correct gas deduction and block creation.



**VIVEKANAND EDUCATION SOCIETY'S
INSTITUTE OF TECHNOLOGY
Department of Information Technology
Academic Year: 2025-26**

ITC801 : Blockchain & DLT Lab
Experiment 09

Aim: To implement a Private Ethereum Blockchain using Geth.

Roll No.	07
Name	Vedang V. Wajge
Class	D20A
Subject	ITC801: Blockchain and DLT Lab
CO Mapped	CO3:Implement smart contracts in Ethereum using different development frameworks.

Experiment 9

AIM: Implement the Private Ethereum Blockchain using Geth

THEORY:

Q1: What is Geth?

Geth (Go-Ethereum) is one of the most widely used Ethereum clients, written in the Go programming language. It allows users to run an Ethereum node, mine Ether, and interact with the blockchain through JSON-RPC, command-line, or JavaScript consoles.

Q2: Significance of a Private Ethereum Network

Setting up a Private Ethereum Network involves creating a customized blockchain environment

isolated from the public Ethereum network. It enables developers to test smart contracts,

consensus mechanisms, and network behavior without incurring gas costs or depending on

public validators.

Q3: Steps for creating a Private Ethereum Network

Steps Involved:

- **Choosing a Network ID:**

Each private blockchain must have a unique network ID to differentiate it from public or other private networks.

- **Choosing a Consensus Algorithm:**

Ethereum supports different consensus mechanisms such as Proof of Work (PoW) and Proof of Authority (PoA). In private networks, Clique (PoA) is often used for faster block confirmation.

- **Creating a Genesis Block:**

The genesis block is the first block of the blockchain, defining network parameters such as difficulty, gas limit, initial account balances, and consensus settings.

- **Initializing the Geth Database:**

The Geth init command initializes the blockchain using the genesis.json file, preparing the data directory for node operation.

- **Setting up Networking:**

Nodes communicate using enodes (Ethereum Node URLs) that contain IP addresses and public keys for peer discovery and connection.

- **Running the Member Nodes:**

Each participant runs a Geth node that connects to the private network and participates in block validation and transaction processing.

- **Running a Signer (in Clique):**

In the Clique consensus, designated signers validate and sign blocks. These accounts are pre-configured in the genesis file to ensure trusted block production.

NOTE: Download the genesis file and edit the account details, including the public keys of all participating peers in the network.

PROCEDURE

Step 1: Install Geth

```
wget
```

```
https://gethstore.blob.core.windows.net/builds/geth-linux-amd64-1.13.14-2bd6bd01.tar.gz
```

```
tar -xvf geth-linux-amd64-1.13.14-2bd6bd01.tar.gz
```

```
mkdir -p ~/bin
```

```
cp geth-linux-amd64-1.13.14-2bd6bd01/geth ~/bin/geth113
```

export

PATH="\$HOME/bin:\$PATH"

```
Welcome to Cloud Shell! Type "help" to get started, or type "pswd" to try prompting with Secure Chat.
To set your Cloud Platform project in this session use "gcloud config set project [PROJECT_ID]".
You can view your projects by running "gcloud projects list".
g2022_aditi_taksale@cloudshell:~$ wget
https://gethstore.blob.core.windows.net/builds/geth-linux-amd64-1.13.14-2bd6bd01.tar.gz
tar -xvf geth-linux-amd64-1.13.14-2bd6bd01.tar.gz mkdir -p ~/bin
cp geth-linux-amd64-1.13.14-2bd6bd01/geth ~/bin/geth113

Resolving gethstore.blob.core.windows.net (gethstore.blob.core.windows.net)... 20.69.60.164
Connecting to gethstore.blob.core.windows.net (gethstore.blob.core.windows.net)[20.69.60.164]:443... connected.
HTTP request sent, awaiting response... 200 OK
Length: 2924813 (2.7M) [application/octet-stream]
Saving to: 'geth-linux-amd64-1.13.14-2bd6bd01.tar.gz.1'

geth-linux-amd64-1.13.14-2bd6bd01.tar.gz.1  100%[=====] 26.94M  4.90MB/s   16 s.1s

2026-04-08 17:25:28 13.31 MB/s ~ 'geth-linux-amd64-1.13.14-2bd6bd01.tar.gz.1' saved [2924813/2924813]

geth-linux-amd64-1.13.14-2bd6bd01/
geth-linux-amd64-1.13.14-2bd6bd01/COPYING
geth-linux-amd64-1.13.14-2bd6bd01/geth
geth
Version: 1.13.14-stable
Git Commit: 2bd6bd01c2ef561ed77fc21b431f6a38ac14427a1
Git Commit Date: 20240227
Architecture: amd64
Go Version: go1.21.4
Operating System: linux
GORATHU=/home/g2022_ria_chaudhari/gopath:/google/gopath
GIMCOP=
```

geth113 version

Step 2: Create folders and accounts for both nodes

```
g2022_aditi_taksale@cloudshell:~$ mkdir -p ~/eth-private/node1 ~/eth-private/node2
g2022_aditi_taksale@cloudshell:~$ geth113 --datadir ~/eth-private/node1 account new
INFO [04-0817:27:46.537] Maximum peer count      ETH=50 total=50
INFO [04-0817:27:46.540] Smartcard socket not found, disabling  err="stat /run/pcscd/pcscd.comm: no such file or directory"
Your new account is locked with a password. Please give a password. Do not forget this password.
Password:
Repeat password:

Your new key was generated

Public address of the key: 0aC8re5b12bb5a223c4bcb4ca8363ed92a2f9878
Path of the secret key file: /home/g2022_ria_chaudhari/eth-private/node1/keystore/UTC--2026-04-08T17-27-52.8650219652---c8fe5b12bb5a223c4bcb4ca8363ed92a2f9878

- You can share your public address with anyone. Others need it to interact with you.
- You must NEVER share the secret key with anyone! The key controls access to your funds!
- You must BACKUP your key file! Without the key, it's impossible to access account funds!
- You must REMEMBER your password! Without the password, it's impossible to decrypt the key!
```

```
g2022_aditi_taksale@cloudshell:~$ geth113 --datadir ~/eth-private/node2 account new
INFO [04-0817:28:38.211] Maximum peer count      ETH=50 total=50
INFO [04-0817:28:38.214] Smartcard socket not found, disabling  err="stat /run/pcscd/pcscd.comm: no such file or directory"
Your new account is locked with a password. Please give a password. Do not forget this password.
Password:
Repeat password:

Your new key was generated

Public address of the key: 0x41911d883221851rc0C0r7c2a3f623d507bb19
Path of the secret key file: /home/g2022_ria_chaudhari/eth-private/node2/keystore/UTC--2026-04-08T17-28-43.3405116222--41911d883221851dd8b0fe7c2a3f623d507bb19

- You can share your public address with anyone. Others need it to interact with you.
- You must NEVER share the secret key with anyone! The key controls access to your funds!
- You must BACKUP your key file! Without the key, it's impossible to access account funds!
- You must REMEMBER your password! Without the password, it's impossible to decrypt the key!
```

Step 3: Create password files

```
g2022_aditi_taksale@cloudshell:~$ echo "1234" > ~/eth-private/node1/password.txt
echo "1234" > ~/eth-private/node2/password.txt
```

Step 4: Create genesis.json and verify it looks correct

NODE1="PASTE_NODE1_ADDRESS_WITHOUT_0x_HERE"

NODE2="PASTE_NODE2_ADDRESS_WITHOUT_0x_HERE"


```
cat > ~/eth-private/genesis.json << EOF
```

 $\{$

```
"config": {
```

```
"chainId": 1234,
```

```
"homesteadBlock": 0,
```

```
"eip150Block": 0,
```

```
"eip155Block": 0,
```

```
"eip158Block": 0,
```

```
"byzantiumBlock": 0,
```

"constantinopleBlock": 0,

"petersburgBlock": 0,

```
"clique": {
```

```
"period": 5,
```

```
"epoch": 30000
```

}

 $\},$

```
"difficulty": "1",
```

```
"gasLimit": "8000000",
```

```
"extradata":
```

[illegible][illegible][illegible]

[illegible]

Step 5: Initialize both nodes

```

q2022_aditi_taksale@cloudshell:~$ geth113 --datadir ~/eth-private/node1 init ~/eth-private/genesis.json
geth113 --datadir ~/eth-private/node2 init ~/eth-private/genesis.json
INFO [04-00:17:42:33.075] Maximum peer count                      ETH=>50 http=>50
INFO [04-00:17:42:33.079] Smartcard socket not found, disabling    err="stat /run/pcscd/pcscd.sock: no such file or directory"
INFO [04-00:17:42:33.085] Set global gas cap                       cap=50,000,000
INFO [04-00:17:42:33.086] Initializing the EVM library             backend=gokzg
INFO [04-00:17:42:33.218] Defaulting to pebble as the backing database
INFO [04-00:17:42:33.219] Allocated cache and file handles         database=/home/q2022_via_chaudhari/eth-private/node1/geth/chaindata cache=16.50MiB handles=16
INFO [04-00:17:42:33.306] Opened ancient database                  database=/home/q2022_via_chaudhari/eth-private/node1/geth/chaindata/ancient/chain readonly=false
INFO [04-00:17:42:33.306] State schema set to default              schema=hash
INFO [04-00:17:42:33.301] Writing custom genesis block
INFO [04-00:17:42:33.304] Persisted trie from memory database       nodes=4 size=478.00B time=3.365396ms gcsize=0.00B gptime=0s livenodes=0 livepairs=0.00B
INFO [04-00:17:42:33.340] Successfully wrote genesis state          database=chaindata hash=38d61a..afc19d
INFO [04-00:17:42:33.340] Defaulting to pebble as the backing database
INFO [04-00:17:42:33.340] Allocated cache and file handles         database=/home/q2022_via_chaudhari/eth-private/node1/geth/lightchaindata cache=16.50MiB handles=16
INFO [04-00:17:42:33.400] Opened ancient database                  database=/home/q2022_via_chaudhari/eth-private/node1/geth/lightchaindata/ancient/chain readonly=false
INFO [04-00:17:42:33.400] State schema set to default              schema=hash
INFO [04-00:17:42:33.404] Writing custom genesis block
INFO [04-00:17:42:33.404] Persisted trie from memory database       nodes=4 size=478.00B time=3.273748ms gcsize=0.00B gptime=0s livenodes=0 livepairs=0.00B
INFO [04-00:17:42:33.438] Successfully wrote genesis state          database=lightchaindata hash=38d61a..afc19d
INFO [04-00:17:42:33.515] Maximum peer count                      ETH=>50 http=>50
INFO [04-00:17:42:33.518] Smartcard socket not found, disabling    err="stat /run/pcscd/pcscd.sock: no such file or directory"
INFO [04-00:17:42:33.523] Set global gas cap                       cap=50,000,000
INFO [04-00:17:42:33.524] Initializing the EVM library             backend=gokzg
INFO [04-00:17:42:33.701] Defaulting to pebble as the backing database
INFO [04-00:17:42:33.701] Allocated cache and file handles         database=/home/q2022_via_chaudhari/eth-private/node2/geth/chaindata cache=16.50MiB handles=16
INFO [04-00:17:42:33.758] Opened ancient database                  database=/home/q2022_via_chaudhari/eth-private/node2/geth/chaindata/ancient/chain readonly=false
INFO [04-00:17:42:33.758] State schema set to default              schema=hash
INFO [04-00:17:42:33.758] Writing custom genesis block
INFO [04-00:17:42:33.762] Persisted trie from memory database       nodes=4 size=478.00B time=3.351028ms gcsize=0.00B gptime=0s livenodes=0 livepairs=0.00B
INFO [04-00:17:42:33.798] Successfully wrote genesis state          database=chaindata hash=38d61a..afc19d
INFO [04-00:17:42:33.798] Defaulting to pebble as the backing database
INFO [04-00:17:42:33.798] Allocated cache and file handles         database=/home/q2022_via_chaudhari/eth-private/node2/geth/lightchaindata cache=16.50MiB handles=16
INFO [04-00:17:42:33.853] Opened ancient database                  database=/home/q2022_via_chaudhari/eth-private/node2/geth/lightchaindata/ancient/chain readonly=false
INFO [04-00:17:42:33.853] State schema set to default              schema=hash
INFO [04-00:17:42:33.853] Writing custom genesis block
INFO [04-00:17:42:33.857] Persisted trie from memory database       nodes=4 size=478.00B time=3.139211ms gcsize=0.00B gptime=0s livenodes=0 livepairs=0.00B
INFO [04-00:17:42:33.891] Successfully wrote genesis state          database=lightchaindata hash=38d61a..afc19d
q2022_aditi_taksale@cloudshell:~$

```

Step 6: Run Node 1 (signer)

```

q2022_aditi_taksale@cloudshell:~$ eth113 --datadir ~/eth-private/node1 \
--networkid 1234 \
--port 30303 \
--http --http.port 8545 \
--http.api personal,eth,net,web3,miner \
--unlock 0x1MOCK1 \
--password ~/eth-private/node1/password.txt \
--mine \
--miner.etherbase 0x1MOCK1 \
--allow-insecure-unlock \
console
INFO [04-00:17:44:52.541] Maximum peer count                      ETH=>50 http=>50
INFO [04-00:17:44:52.548] Smartcard socket not found, disabling    err="stat /run/pcscd/pcscd.sock: no such file or directory"
INFO [04-00:17:44:52.548] Set global gas cap                       cap=50,000,000
INFO [04-00:17:44:52.549] Initializing the EVM library             backend=gokzg
INFO [04-00:17:44:52.700] Allocated trie memory caches            csize=154.00MiB http=254.50MiB
INFO [04-00:17:44:52.701] Using pebble as the backing database     database=/home/q2022_via_chaudhari/eth-private/node1/geth/chaindata cache=16.50MiB handles=16,288
INFO [04-00:17:44:52.730] Opened ancient database                  database=/home/q2022_via_chaudhari/eth-private/node1/geth/chaindata/ancient/chain readonly=false
INFO [04-00:17:44:52.730] State schema set to already existing      schema=hash
INFO [04-00:17:44:52.732] Initializing Ethereum protocol           network=1234 (devchain=0)
INFO [04-00:17:44:52.732] -----
INFO [04-00:17:44:52.733] Chain ID: 1234 (unknown)
INFO [04-00:17:44:52.733] Consensus: Clique (proof-of-authority)
INFO [04-00:17:44:52.733] -----
INFO [04-00:17:44:52.733] Pre-Merge hard forks (block based):
INFO [04-00:17:44:52.733] - Homestead: #0 (https://github.com/ethereum/execution-specs/blob/master/network-upgrades/mainnet-upgrades/homestead.md)
INFO [04-00:17:44:52.733] - Tangerine Whistle (EIP 150): #0 (https://github.com/ethereum/execution-specs/blob/master/network-upgrades/mainnet-upgrades/tangerine-whistle.md)
INFO [04-00:17:44:52.733] - Spurious Dragon/1 (EIP 153): #0 (https://github.com/ethereum/execution-specs/blob/master/network-upgrades/mainnet-upgrades/spurious-dragon.md)
INFO [04-00:17:44:52.733] - Spurious Dragon/2 (EIP 158): #0 (https://github.com/ethereum/execution-specs/blob/master/network-upgrades/mainnet-upgrades/spurious-dragon.md)
INFO [04-00:17:44:52.733] - Byzantium: #0 (https://github.com/ethereum/execution-specs/blob/master/network-upgrades/mainnet-upgrades/byzantium.md)
INFO [04-00:17:44:52.733] - Constantinople: #0 (https://github.com/ethereum/execution-specs/blob/master/network-upgrades/mainnet-upgrades/constantinople.md)
INFO [04-00:17:44:52.733] - Petersburg: #0 (https://github.com/ethereum/execution-specs/blob/master/network-upgrades/mainnet-upgrades/petersburg.md)
INFO [04-00:17:44:52.733] - Istanbul: #null (https://github.com/ethereum/execution-specs/blob/master/network-upgrades/mainnet-upgrades/istanbul.md)
INFO [04-00:17:44:52.733] - Berlin: #null (https://github.com/ethereum/execution-specs/blob/master/network-upgrades/mainnet-upgrades/berlin.md)
INFO [04-00:17:44:52.733] - London: #null (https://github.com/ethereum/execution-specs/blob/master/network-upgrades/mainnet-upgrades/london.md)

```

Step 7: Open new terminal tab and run Node 2 (member)

Step 8: Connect the two nodes In Node 1 console, get its enode

Now, switch to Node 2 console and paste the Node 1 enode

```
> admin.addPeer("enode://37a3db1c1e34eeeb3611941ca5b7001168ac8eec28506649e6a5290979c7a78e7277565b37ec5d703871a9970a4794c51bde075d7991723256e0c07c8cb367eb034.21.175.36:30303")
true
```

In Node 2 console, check they are connected

```
> admin.peersINFO [04-08|18:01:18.017] imported new chain segment      number=196 hash=9ac569..83e85f blocks=1 txs=0 mgas=0.000 elapsed=7.74ms mgasps=0.000 trieditt
[
  {
    caps: ["eth/65", "snap/1"],
    enode: "enode://37a3db1c1e34eac3611941ca5b7001168ac8eac28506649e6a5290979c7a78e7277565b37ec5d703871a9970a4794c51bde075d7991723256e0c07c8cb367eb8127.0.0.1:30303",
    id: "ab0dd2ac241e15eac929ff71bbb9d19f854213e77207f9ee0b74c4911801aef7",
    name: "geth/v1.13.14-stable-2bd6bd01/linux-amd64/go1.21.6",
    network: {
      inbound: false,
      localAddress: "127.0.0.1:60510",
      remoteAddress: "127.0.0.1:30303",
      static: true,
      trusted: false
    },
    protocols: {
      eth: {
        version: 65
      },
      snap: {
        version: 1
      }
    }
  }
]
```

Step 9: Send a transaction In Node 1 console, send transaction to Node 2

```
> eth.sendTransaction({
  .... from: "0xC8Fe5b129bb5e23eC4bD84CaB363ed92a2f9878",
  .... to: "0x41911D883221851DD8C0Fe7C2a3E623D507bBb19",
  .... value: "0xde0b6b3a7640000"
  .... })
INFO [04-08|18:03:16.769] Setting new local account      address=0xC8Fe5b129bb5e23eC4bD84CaB363ed92a2f9878
INFO [04-08|18:03:16.770] Submitted transaction      hash=0x6e201621cef61dc5e3cf114cbd03055c23b1f083ebf98411f8c91cb48e47265 from=0xC8Fe5b129bb5e23eC4bD84CaB363ed92a2f9878 nonce=
0 recipient=0x41911D883221851DD8C0Fe7C2a3E623D507bBb19 value=1,000,000,000,000,000
0x6e201621cef61dc5e3cf114cbd03055c23b1f083ebf98411f8c91cb48e47265
```

and check the balance of Node 2

```
> web3.fromWei(
  ...   eth.getBalance("0x41911D883221851DD8C0Fe7C2a3E623D507bBb19"),
  ...   "ether"
  ... )
101
```

CONCLUSION:

Through this experiment, two smart contracts - OrderRecord and ReviewVerification, were successfully created and deployed on the Ethereum Sepolia test network using Solidity and Remix IDE. The experiment demonstrated how blockchain technology can be integrated into a full-stack web application to permanently and transparently record order transactions and product reviews. The use of the Sepolia test network and MetaMask allowed safe testing without real funds. Overall, this experiment provided practical understanding of designing, compiling, deploying, and testing smart contracts on the Ethereum blockchain.



**VIVEKANAND EDUCATION SOCIETY'S
INSTITUTE OF TECHNOLOGY
Department of Information Technology
Academic Year: 2025-26**

ITC801 : Blockchain & DLT Lab
Experiment 10

Aim: Explore the Cryptocurrency Landscape

Roll No.	07
Name	Vedang V. Wajge
Class	D20A
Subject	ITC801: Blockchain and DLT Lab
CO Mapped	CO1:Describe the basic concept of Blockchain and Distributed Ledger Technology. CO2:Interpret the knowledge of the Bitcoin network, nodes, keys, wallets and transactions CO5:Interpret different Crypto assets and Crypto currencies CO6:Analyze the use of Blockchain with AI, IoT and Cyber Security using case studies.

Blockchain Experiment 10

AIM: Explore the Cryptocurrency Landscape

THEORY:

Cryptocurrencies are decentralized digital assets built on blockchain technology, enabling secure, transparent, and immutable transactions without the need for intermediaries. Among various types of cryptocurrencies, **stablecoins** are designed to maintain a stable value by being pegged to fiat currency such as the US Dollar.

Stablecoins are a type of cryptocurrency designed to maintain a stable value by being pegged to an underlying asset such as a fiat currency (e.g., US Dollar), cryptocurrency, or algorithmic mechanism. Unlike volatile cryptocurrencies like Bitcoin, stablecoins aim to reduce price fluctuations, making them suitable for trading, payments, and decentralized finance (DeFi) applications.

Fiat-backed stablecoins are supported by real-world currencies like USD, EUR, etc. For every coin issued, an equivalent amount of money is stored in reserves (like banks).

Example: USDT (Tether), USDC, TUSD

Crypto-backed stablecoins are backed by other cryptocurrencies (like ETH). Since crypto prices fluctuate, extra collateral is required to maintain stability.

Example: DAI

Hybrid stablecoins use a combination of collateral (like USDC) and algorithmic mechanisms to maintain price stability.

Example: FRAX

This experiment focuses on analyzing stablecoins including USDT, USDC, DAI, FRAX, and TUSD using real-time data collected from multiple sources:

- **CoinGecko API:** Provided historical price, trading volume, and market capitalization data
- **CoinMarketCap / Crypto platforms:** Used for validation and market insights
- **Online resources:** Used for understanding adoption and use cases

Cryptocurrency Landscape:

- **Stablecoins** – Maintain stable value (USDT, USDC, DAI)
- **Store of Value** – Bitcoin
- **Utility Tokens** – Ethereum
- **Governance Tokens** – UNI, AAVE

Key Characteristics of Stablecoins:

- Low volatility compared to other cryptocurrencies
- Used for trading, payments, and DeFi applications
- Backed by fiat reserves, crypto assets, or hybrid mechanisms

TECHNOLOGICAL ANALYSIS:

Recent advancements in blockchain technology include:

- Consensus Mechanisms: PoW, PoS for secure validation
- Scalability: Layer-2 solutions like rollups
- Interoperability: Cross-chain communication
- Privacy: Zero-knowledge proofs

Stablecoins leverage these technologies to provide efficient and scalable financial solutions in decentralized ecosystems.

MARKET TRENDS & ADOPTION:

- Increasing use of stablecoins in DeFi platforms
- Growing merchant acceptance for payments
- Rising number of crypto wallets and users
- Integration with traditional financial systems
- Regulatory developments across countries

TASK PERFORMED:

1. Collected historical price and volume data using CoinGecko API
2. Processed data using Python libraries (Pandas, Requests)
3. Visualized data using Matplotlib and Seaborn
4. Compared price stability, volatility, and market dominance
5. Classified stablecoins based on type and use case

CODE:

```
!pip install requests pandas matplotlib seaborn -q
```

```
import requests
import pandas as
pd
import matplotlib.pyplot as plt
import matplotlib.ticker as mticker
import seaborn as sns
import time
STABLECOINS = {
    'tether':      'USDT',
    'usd-coin':   'USDC',
```



```

        'dai':                'DAI',
        'frax':               'FRAX',
        'true-usd':           'TUSD'
    }
    DAYS = 90                 # last 90 days
    COLOR = ['#2196F3', '#4CAF50', '#FF9800', '#9C27B0', '#F44336']
    def fetch_market_chart(coin_id, days=DAYS, retries=3):
        url =
        f"https://api.coingecko.com/api/v3/coins/{coin_id}/market_chart"
        params = {"vs_currency": "usd", "days": days, "interval": "daily"}

        for attempt in range(retries):
            try:
                r = requests.get(url, params=params, timeout=20)
                if r.status_code == 200:
                    return r.json()
                elif r.status_code == 429:
                    wait = 15 * (attempt + 1)    # 15s → 30s → 45s
                    print(f" ⌚ Rate limited. Waiting {wait}s before retry
{attempt+1}/{retries}...")
                    time.sleep(wait)
                else:
                    print(f" ⚠ Could not fetch {coin_id}: HTTP
{r.status_code}")
                    return None
            except Exception as e:
                print(f" Error fetching {coin_id}: {e}")
                return None

        print(f" Failed after {retries} retries: {coin_id}")
        return None

    # --- Fetch historical data for each stablecoin ---
    price_frames = {}
    volume_frames = {}

    print("Fetching data from CoinGecko API (with rate-limit handling)...\n")
    for coin_id, ticker in STABLECOINS.items():
        print(f" → Fetching {ticker} ({coin_id})...")
        data = fetch_market_chart(coin_id)
        if data:

```

```

pdf = pd.DataFrame(data['prices'], columns=['ts', 'price'])
vdf = pd.DataFrame(data['total_volumes'], columns=['ts',
'volume'])
pdf['date'] = pd.to_datetime(pdf['ts'], unit='ms').dt.normalize()
vdf['date'] = pd.to_datetime(vdf['ts'], unit='ms').dt.normalize()
pdf = pdf.drop_duplicates('date').set_index('date')
vdf =
vdf.drop_duplicates('date').set_index('date')
price_frames[ticker] = pdf['price']
volume_frames[ticker] = vdf['volume']
print(f" {ticker} fetched successfully!")
time.sleep(8)    # ← increased from 1.2s to 8s between each

call prices_df = pd.DataFrame(price_frames)

volumes_df = pd.DataFrame(volume_frames)
print("\n All data fetched!")
def fetch_current_data():
    """Fetch current market cap, volume, price for all stablecoins."""
    ids = ",".join(STABLECOINS.keys())
    url = "https://api.coingecko.com/api/v3/coins/markets?vs_currency=usd"
    params = {
        "vs_currency": "usd",
        "ids": ids,
        "order": "market_cap_desc",
        "per_page": 10,
        "page": 1,
        "sparkline":
False
    }
    r = requests.get(url, params=params, timeout=15)
    return pd.DataFrame(r.json())
# Current snapshot
current_df = fetch_current_data()
print("\n Data collection complete!\n")
print(current_df[['name', 'current_price', 'market_cap', 'total_volume']].t
o_string(index=False))
import seaborn as sns
import matplotlib.pyplot as plt
import matplotlib.ticker as mticker

# Define colors

```

```

COLOR = ['blue', 'orange', 'green', 'red', 'purple']
sns.set_theme(style="darkgrid")

fig = plt.figure(figsize=(12, 6))

tickers =
list(STABLECOINS.values())
plt.figure(figsize=(10,5))

for ticker in tickers:
    if ticker in prices_df.columns:
        plt.plot(prices_df.index, prices_df[ticker], label=ticker)

plt.axhline(1.0, linestyle='--', label='$1 Peg')

plt.title('Price Stability vs $1 Peg')
plt.xlabel('Date')
plt.ylabel('Price
(USD)') plt.legend()
plt.grid()

plt.show()
plt.figure(figsize=(10,5))

for ticker in tickers:
    if ticker in prices_df.columns:
        deviation = (prices_df[ticker] - 1.0) * 100
        plt.plot(prices_df.index, deviation, label=ticker)

plt.axhline(0, linestyle='--')

plt.title('Deviation from $1 Peg (%)')
plt.xlabel('Date')
plt.ylabel('Deviation (%)')
plt.legend()
plt.grid()

plt.show()
plt.figure(figsize=(8,5))

if not current_df.empty:

```

```

names = current_df['symbol'].str.upper()
mcaps = current_df['market_cap'] / 1e9

plt.bar(names, mcaps)

for i, val in enumerate(mcaps):
    plt.text(i, val, f'{val:.1f}B', ha='center')

plt.title('Market Capitalization (Billion USD)')
plt.xlabel('Stablecoin')
plt.ylabel('Market Cap (Billion USD)')
plt.grid()

plt.show()
plt.figure(figsize=(6,6))

if not current_df.empty:
    labels =
    current_df['symbol'].str.upper() sizes =
    current_df['market_cap']

    plt.pie(sizes, labels=labels, autopct='%1.1f%%', startangle=140)

plt.title('Market Cap
Dominance') plt.show()
plt.figure(figsize=(10,5))

for ticker in tickers:
    if ticker in volumes_df.columns:
        plt.plot(volumes_df.index, volumes_df[ticker] / 1e9, label=ticker)

plt.title('Trading Volume (Billion USD)')
plt.xlabel('Date')
plt.ylabel('Volume')
plt.legend()
plt.grid()

plt.show()
plt.figure(figsize=(10,5))

for ticker in tickers:

```

```

if ticker in prices_df.columns:
    returns = prices_df[ticker].pct_change()
    volatility = returns.rolling(7).std() * 100
    plt.plot(prices_df.index, volatility,
             label=ticker)

plt.title('7-Day Volatility
(%)') plt.xlabel('Date')
plt.ylabel('Volatility')
plt.legend()
plt.grid()

plt.show()
classification = {
    'Coin': ['USDT', 'USDC', 'DAI', 'FRAX', 'TUSD'],
    'Type': ['Fiat-backed', 'Fiat-backed', 'Crypto-backed',
            'Hybrid', 'Fiat-backed'],
    'Use Case': ['Trading & Liquidity', 'Payments & DeFi', 'DeFi Lending',
                'Stable DeFi Ecosystem', 'Cross-border Payments']
}
cls_df = pd.DataFrame(classification)
print(cls_df.to_string(index=False))

```

OUTPUT:

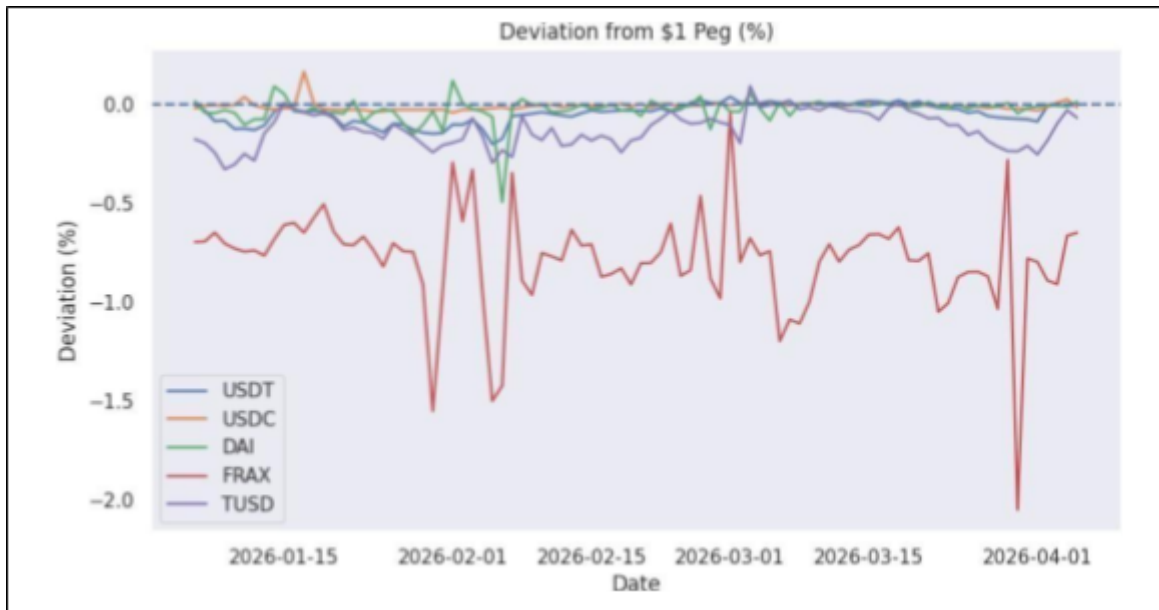
1. Price Stability

This graph shows the price movement of stablecoins over time. All selected stablecoins remain close to the \$1 value, demonstrating their ability to maintain price stability through pegging mechanisms.



2. Deviation from \$1 Peg

This graph represents the percentage deviation of stablecoin prices from the \$1 peg. The minimal fluctuations indicate low de-pegging risk and high reliability of stablecoins in maintaining consistent value.



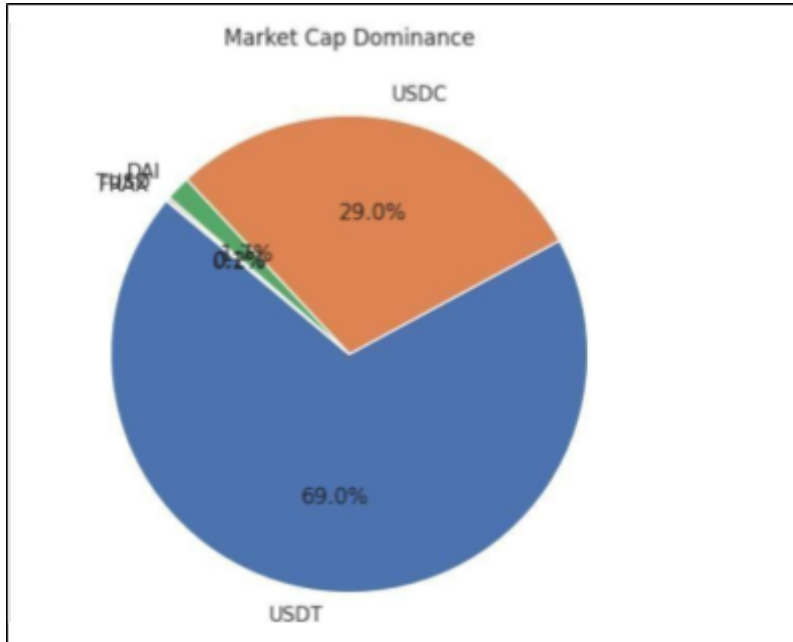
3. Market Capitalization

This graph compares the market capitalization of different stablecoins. It shows that USDT and USDC dominate the market, indicating higher adoption, liquidity, and trust among users.



4. Market Dominance

This pie chart illustrates the proportion of total market share held by each stablecoin. It highlights that a few major stablecoins control a large portion of the market.



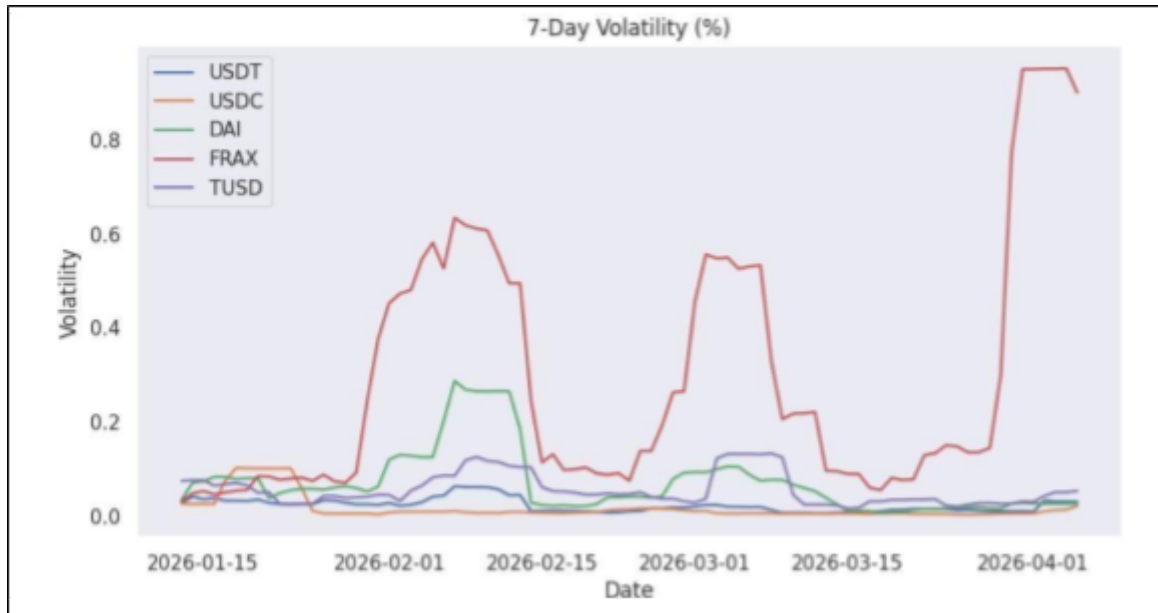
5. Trading Volume

This graph shows the daily trading volume of stablecoins over time. Higher trading volume reflects greater usage in transactions, exchanges, and DeFi applications.



6. 7-Day Volatility

This graph represents the rolling volatility of stablecoin prices. The low volatility confirms that stablecoins experience minimal price fluctuations compared to other cryptocurrencies.



The selected cryptocurrencies are classified based on their backing mechanism and use cases.

***	Coin	Type	Use Case
	USDT	Fiat-backed	Trading & Liquidity
	USDC	Fiat-backed	Payments & DeFi
	DAI	Crypto-backed	DeFi Lending
	FRAX	Hybrid Stable	DeFi Ecosystem
	TUSD	Fiat-backed	Cross-border Payments

CONCLUSION:

The experiment successfully explored the cryptocurrency landscape by analyzing stablecoins using real-time data. The results demonstrated that stablecoins maintain price stability, exhibit low volatility, and play a crucial role in trading, payments, and decentralized finance. This study highlights the importance of blockchain technology in real-world financial applications.



**VIVEKANAND EDUCATION SOCIETY'S
INSTITUTE OF TECHNOLOGY
Department of Information Technology
Academic Year: 2025-26**

ITC801 : Blockchain & DLT Lab
Experiment 11

Aim: Implementation of Hyperledger Fabric Network and Asset Management using Chaincode

Roll No.	07
Name	Vedang V. Wajge
Class	D20A
Subject	ITC801: Blockchain and DLT Lab
CO Mapped	CO4:Develop applications in permissioned Hyperledger Fabric network.

Blockchain Experiment 11

AIM: To implement a Hyperledger Fabric network, deploy chaincode, and perform asset creation, querying, and transfer operations.

PROCEDURE

Step 1: Update the system

Go to <https://shell.cloud.google.com> and run the following commands:

```
sudo apt update
sudo apt upgrade
-y
```

```
g2022 aditi_taksale@cloudshell:~$ sudo apt update
Get:1 https://cli.github.com/packages stable InRelease [3,917 B]
Get:2 https://packages.cloud.google.com/apt gcsfuse-noble InRelease [1,227 B]
Get:3 https://packages.cloud.google.com/apt cloud-sdk InRelease [1,620 B]
Get:4 https://apt.postgresql.org/pub/repos/apt noble-pgdg InRelease [180 kB]
Hit:5 http://archive.ubuntu.com/ubuntu noble InRelease
Get:6 http://security.ubuntu.com/ubuntu noble-security InRelease [126 kB]
Get:7 https://packages.cloud.google.com/apt gcsfuse-noble/main amd64 Packages [66.1 kB]
Get:8 https://repo.mongodb.org/apt/ubuntu noble/mongodb-org/8.0 InRelease [3,005 B]
Get:9 http://archive.ubuntu.com/ubuntu noble-updates InRelease [126 kB]
Get:10 https://packages.cloud.google.com/apt cloud-sdk/main all Packages [1,982 kB]
Get:11 https://apt.postgresql.org/pub/repos/apt noble-pgdg/main amd64 Packages [675 kB]
Get:12 https://packages.cloud.google.com/apt cloud-sdk/main amd64 Packages [4,640 kB]
Hit:13 https://ppa.launchpadcontent.net/dotnet/backports/ubuntu noble InRelease
Get:14 https://repo.mongodb.org/apt/ubuntu noble/mongodb-org/8.0/multiverse amd64 Packages [68.7 kB]
Get:15 http://archive.ubuntu.com/ubuntu noble-backports InRelease [126 kB]
Get:16 http://security.ubuntu.com/ubuntu noble-security/restricted amd64 Packages [3,505 kB]
Get:17 http://archive.ubuntu.com/ubuntu noble-updates/main amd64 Packages [2,377 kB]
Get:18 http://archive.ubuntu.com/ubuntu noble-updates/restricted amd64 Packages [3,692 kB]
Get:19 http://security.ubuntu.com/ubuntu noble-security/main amd64 Packages [1,989 kB]
Get:20 http://archive.ubuntu.com/ubuntu noble-updates/universe amd64 Packages [2,155 kB]
Get:21 http://security.ubuntu.com/ubuntu noble-security/universe amd64 Packages [1,508 kB]
```

```
g2022 aditi_taksale@cloudshell:~$ sudo apt upgrade -y
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
Calculating upgrade... Done
The following upgrades have been deferred due to phasing:
  apparmor libapparmor1
The following packages will be upgraded:
  docker-buildx-plugin docker-ce docker-ce-cli docker-ce-rootless-extras docker-compose-plugin google-cloud-cli
  google-cloud-cli-anthoscli google-cloud-cli-app-engine-go google-cloud-cli-app-engine-java google-cloud-cli-app-engine-python
  google-cloud-cli-app-engine-python-extras google-cloud-cli-bigtable-emulator google-cloud-cli-cbt google-cloud-cli-cloud-run-proxy
  google-cloud-cli-datastore-emulator google-cloud-cli-gke-gcloud-auth-plugin google-cloud-cli-istioctl google-cloud-cli-kpt
  google-cloud-cli-local-extract google-cloud-cli-minikube google-cloud-cli-nomos google-cloud-cli-package-go-module
  google-cloud-cli-pubsub-emulator google-cloud-cli-scaffold google-cloud-cli-spanner-cli kubectrl libarchive13t64 libgdk-pixbuf-2.0-0
  libgdk-pixbuf2.0-common libpam-systemd libpq-dev libpq5 libruby3.2 libssl-dev libssl3t64 libsystemd-shared libsystemd0 libudev1
  linux-libc-dev openssl php8.3-bcmath php8.3-cgi php8.3-cli php8.3-common php8.3-dev php8.3-mbstring php8.3-mysql php8.3-opcache
  php8.3-readline php8.3-xml qemu-user qemu-user-static ruby3.2 ruby3.2-dev systemd systemd-dev systemd-sysv tzdata udev
59 upgraded, 0 newly installed, 0 to remove and 2 not upgraded.
Need to get 894 MB of archives.
```

Step 2: Install Required Dependencies

```
g2022_aditi_taksale@cloudshell:~$ sudo apt install -y jq curl git
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
jq is already the newest version (1.7.1-3ubuntu0.24.04.1).
curl is already the newest version (8.5.0-2ubuntu10.8).
git is already the newest version (1:2.43.0-1ubuntu7.3).
0 upgraded, 0 newly installed, 0 to remove and 2 not upgraded.
```

Then verify everything is present:

```
node -v
```

```
npm -v
```

```
git --version
```

```
docker --version
```

```
docker ps
```

```
g2022_aditi_taksale@cloudshell:~$ node -v
v24.14.1
g2022_aditi_taksale@cloudshell:~$ npm -v
11.12.1
g2022_aditi_taksale@cloudshell:~$ git --version
git version 2.43.0
g2022_aditi_taksale@cloudshell:~$ docker --version
Docker version 29.4.0, build 9d7ad9f
g2022_aditi_taksale@cloudshell:~$
g2022_aditi_taksale@cloudshell:~$ docker ps
CONTAINER ID   IMAGE          COMMAND                  CREATED        STATUS        PORTS        NAMES
g2022_aditi_taksale@cloudshell:~$
```

Step 3: Fix Docker Compose

```
DOCKER_CONFIG=${DOCKER_CONFIG:-$HOME/.docker}
mkdir -p $DOCKER_CONFIG/cli-plugins
curl -SL
https://github.com/docker/compose/releases/download/v2.24.5/docker-compo
se
-linux-x86_64 \
  -o $DOCKER_CONFIG/cli-plugins/docker-compose
chmod +x
$DOCKER_CONFIG/cli-plugins/docker-compose docker
compose version
sudo mv /usr/bin/docker-compose /usr/bin/docker-compose-old 2>/dev/null ||
true
sudo tee /usr/local/bin/docker-compose > /dev/null <<'EOF'
#!/bin/bash
exec docker compose "$@"
EOF
sudo chmod +x /usr/local/bin/docker-compose
docker compose version
```

docker-compose version

```
g2022_anuprita_mhapankar@cloudshell:~$ DOCKER_CONFIG=${DOCKER_CONFIG:-$HOME/.docker}
mkdir -p $DOCKER_CONFIG/cli-plugins
curl -SL https://github.com/docker/compose/releases/download/v2.24.5/docker-compose-linux-x86_64 \
-o $DOCKER_CONFIG/cli-plugins/docker-compose
chmod +x $DOCKER_CONFIG/cli-plugins/docker-compose
docker compose version
% Total      % Received % Xferd Average Speed   Time    Time     Time  Current
   0      0     0     0         0      0      0      0      0
100 58.5M  100 58.5M    0     0  79.4M      0  --:--:-- --:--:-- --:--:--  79.4M
Docker Compose version v2.24.5
g2022_anuprita_mhapankar@cloudshell:~$ sudo mv /usr/bin/docker-compose /usr/bin/docker-compose-old 2>/dev/null || true
g2022_anuprita_mhapankar@cloudshell:~$ sudo tee /usr/local/bin/docker-compose > /dev/null <<'EOF'
#!/bin/bash
exec docker compose "$@"
EOF
sudo chmod +x /usr/local/bin/docker-compose
g2022_anuprita_mhapankar@cloudshell:~$ docker compose version
docker-compose version
Docker Compose version v2.24.5
Docker Compose version v2.24.5
g2022_anuprita_mhapankar@cloudshell:~$
```

Step 4: Clone Fabric Samples

```
cd $HOME
git clone https://github.com/hyperledger/fabric-samples
cd fabric-samples
```

```
g2022_anuprita_mhapankar@cloudshell:~$ cd $HOME
g2022_anuprita_mhapankar@cloudshell:~$ git clone https://github.com/hyperledger/fabric-samples
Cloning into 'fabric-samples'...
remote: Enumerating objects: 15618, done.
remote: Counting objects: 100% (238/238), done.
remote: Compressing objects: 100% (129/129), done.
remote: Total 15618 (delta 161), reused 109 (delta 109), pack-reused 15380 (from 3)
Receiving objects: 100% (15618/15618), 24.07 MiB | 17.13 MiB/s, done.
Resolving deltas: 100% (8809/8809), done.
g2022_anuprita_mhapankar@cloudshell:~$ cd fabric-samples
g2022_anuprita_mhapankar@cloudshell:~/fabric-samples$
```

Step 5: Install Fabric Binaries and Docker Images

```
curl -sSLO
https://raw.githubusercontent.com/hyperledger/fabric/main/scripts/install-
fabric.sh
chmod +x install-fabric.sh
./install-fabric.sh docker samples binary
```



```
g2022 anuprita_mhapankar@cloudshell:~/fabric-samples$ curl -sSLO https://raw.githubusercontent.com/hyperledger/fabric/main/scripts/install-fabric.sh
g2022 anuprita_mhapankar@cloudshell:~/fabric-samples$ chmod +x install-fabric.sh
./install-fabric.sh docker samples binary
```

Clone hyperledger/fabric-samples repo

==> Already in fabric-samples repo
fabric-samples v2.5.15 does not exist, defaulting to main. fabric-samples main branch is intended to work with recent versions of fabric.

Pull Hyperledger Fabric binaries

```
==> Downloading version 2.5.15 platform specific fabric binaries
==> Downloading: https://github.com/hyperledger/fabric/releases/download/v2.5.15/hyperledger-fabric-linux-amd64-2.5.15.tar.gz
==> Will unpack to: /home/g2022 anuprita_mhapankar/fabric-samples
% Total    % Received % Xferd Average Speed   Time    Time     Time  Current
           Dload Upload   Total   Spent    Left   Speed
  0     0    0     0    0     0      0      0 --:--:-- --:--:-- --:--:--    0
100 120M  100 120M    0    0 22.5M    0  0:00:05  0:00:05 --:--:-- 24.4M
==> Done.
==> Downloading version 1.5.17 platform specific fabric-ca-client binary
==> Downloading: https://github.com/hyperledger/fabric-ca/releases/download/v1.5.17/hyperledger-fabric-ca-linux-amd64-1.5.17.tar.gz
==> Will unpack to: /home/g2022 anuprita_mhapankar/fabric-samples
% Total    % Received % Xferd Average Speed   Time    Time     Time  Current
           Dload Upload   Total   Spent    Left   Speed
  0     0    0     0    0     0      0      0 --:--:-- --:--:-- --:--:--    0
100 31.6M  100 31.6M    0    0 15.9M    0  0:00:01  0:00:01 --:--:-- 31.6M
==> Done.
```

Pull Hyperledger Fabric docker images

```
FABRIC_IMAGES: peer orderer ccenv baseos
==> Pulling fabric Images
==> ghcr.io/hyperledger/fabric-peer:2.5.15
```

```
====> ghcr.io/hyperledger/fabric-ca:1.5.17
1.5.17: Pulling from hyperledger/fabric-ca
18afb8c8d2a4: Pull complete
6f4ebca3e823: Pull complete
b9649d3f054a: Pull complete
0b966e606d9a: Pull complete
c36bacd85367: Download complete
Digest: sha256:453a0a727e82c4960b797e379adc231e853ee23ae7526db53afef77b5074117e
Status: Downloaded newer image for ghcr.io/hyperledger/fabric-ca:1.5.17
ghcr.io/hyperledger/fabric-ca:1.5.17
==> List out hyperledger images
WARNING: This output is designed for human readability. For machine-readable output, please use --format.
ghcr.io/hyperledger/fabric-baseos:2.5.15      654bd4554bf9      250MB      80.4MB
ghcr.io/hyperledger/fabric-ca:1.5.17         453a0a727e82      381MB      123MB
ghcr.io/hyperledger/fabric-ccenv:2.5.15      86dded7eda92      1GB        255MB
ghcr.io/hyperledger/fabric-orderer:2.5.15    9972c209be47      178MB      49.5MB
ghcr.io/hyperledger/fabric-peer:2.5.15       3e25adc31a75      230MB      66.2MB
hyperledger/fabric-baseos:2.5                654bd4554bf9      250MB      80.4MB
hyperledger/fabric-baseos:2.5.15             654bd4554bf9      250MB      80.4MB
hyperledger/fabric-baseos:latest              654bd4554bf9      250MB      80.4MB
hyperledger/fabric-ca:1.5                    453a0a727e82      381MB      123MB
hyperledger/fabric-ca:1.5.17                 453a0a727e82      381MB      123MB
hyperledger/fabric-ca:latest                  453a0a727e82      381MB      123MB
hyperledger/fabric-ccenv:2.5                 86dded7eda92      1GB        255MB
hyperledger/fabric-ccenv:2.5.15              86dded7eda92      1GB        255MB
hyperledger/fabric-ccenv:latest              86dded7eda92      1GB        255MB
hyperledger/fabric-orderer:2.5               9972c209be47      178MB      49.5MB
hyperledger/fabric-orderer:2.5.15            9972c209be47      178MB      49.5MB
hyperledger/fabric-orderer:latest            9972c209be47      178MB      49.5MB
hyperledger/fabric-peer:2.5                  3e25adc31a75      230MB      66.2MB
hyperledger/fabric-peer:2.5.15               3e25adc31a75      230MB      66.2MB
hyperledger/fabric-peer:latest               3e25adc31a75      230MB      66.2MB
g2022 anuprita_mhapankar@cloudshell:~/fabric-samples$
```

Step 6: Verify Docker images

docker images | grep fabric

```
g2022_anuprita_mhapankar@cloudshell:~/fabric-samples$ docker images | grep fabric
WARNING: This output is designed for human readability. For machine-readable output, please use --format.
ghcr.io/hyperledger/fabric-baseos:2.5.15      654bd4554bf9      250MB      80.4MB
ghcr.io/hyperledger/fabric-ca:1.5.17          453a0a727e82      381MB      123MB
ghcr.io/hyperledger/fabric-ccenv:2.5.15       86dded7eda92      1GB        255MB
ghcr.io/hyperledger/fabric-orderer:2.5.15     9972c209be47      178MB      49.5MB
ghcr.io/hyperledger/fabric-peer:2.5.15        3e25adc31a75      230MB      66.2MB
hyperledger/fabric-baseos:2.5                 654bd4554bf9      250MB      80.4MB
hyperledger/fabric-baseos:2.5.15              654bd4554bf9      250MB      80.4MB
hyperledger/fabric-baseos:latest              654bd4554bf9      250MB      80.4MB
hyperledger/fabric-ca:1.5                     453a0a727e82      381MB      123MB
hyperledger/fabric-ca:1.5.17                  453a0a727e82      381MB      123MB
hyperledger/fabric-ca:latest                   453a0a727e82      381MB      123MB
hyperledger/fabric-ccenv:2.5                   86dded7eda92      1GB        255MB
hyperledger/fabric-ccenv:2.5.15                86dded7eda92      1GB        255MB
hyperledger/fabric-ccenv:latest                86dded7eda92      1GB        255MB
hyperledger/fabric-orderer:2.5                 9972c209be47      178MB      49.5MB
hyperledger/fabric-orderer:2.5.15              9972c209be47      178MB      49.5MB
hyperledger/fabric-orderer:latest              9972c209be47      178MB      49.5MB
hyperledger/fabric-peer:2.5                    3e25adc31a75      230MB      66.2MB
hyperledger/fabric-peer:2.5.15                 3e25adc31a75      230MB      66.2MB
hyperledger/fabric-peer:latest                 3e25adc31a75      230MB      66.2MB
g2022_anuprita_mhapankar@cloudshell:~/fabric-samples$
```

Step 7: Set Environment Variables

```
export PATH=$PATH:$HOME/fabric-samples/bin
```

```
export
```

```
FABRIC_CFG_PATH=$HOME/fabric-samples/config peer
```

```
g2022_anuprita_mhapankar@cloudshell:~/fabric-samples$ export PATH=$PATH:$HOME/fabric-samples/bin
g2022_anuprita_mhapankar@cloudshell:~/fabric-samples$ export FABRIC_CFG_PATH=$HOME/fabric-samples/config
g2022_anuprita_mhapankar@cloudshell:~/fabric-samples$ peer version
peer:
  Version: v2.5.15
  Commit SHA: 83c7930
  Go version: go1.26.0
  OS/Arch: linux/amd64
  Chaincode:
    Base Docker Label: org.hyperledger.fabric
    Docker Namespace: hyperledger
g2022_anuprita_mhapankar@cloudshell:~/fabric-samples$
```

Step 8: Start the Network and Create Channel

```
cd $HOME/fabric-samples/test-network
```

```
./network.sh up createChannel
```


Step 9: Deploy Chaincode

```
./network.sh deployCC \
```

```
-ccn basic \
```

```
-ccp ../asset-transfer-basic/chaincode-javascript \
```

```
-ccl javascript
```

```
g2022_anuprita_mhapankar@cloudshell:~/fabric-samples/test-network$ cd $HOME/fabric-samples/test-network

./network.sh deployCC \
-ccn basic \
-ccp ../asset-transfer-basic/chaincode-javascript \
-ccl javascript
Using docker and docker-compose
deploying chaincode on channel 'mychannel'
executing with the following
- CHANNEL_NAME: mychannel
- CC_NAME: basic
- CC_SRC_PATH: ../asset-transfer-basic/chaincode-javascript
- CC_SRC_LANGUAGE: javascript
- CC_VERSION: 1.0
- CC_SEQUENCE: auto
- CC_END_POLICY: NA
- CC_COLL_CONFIG: NA
- CC_INIT_FCN: NA
- DELAY: 3
- MAX_RETRY: 5
- VERBOSE: false
executing with the following
- CC_NAME: basic
- CC_SRC_PATH: ../asset-transfer-basic/chaincode-javascript
- CC_SRC_LANGUAGE: javascript
- CC_VERSION: 1.0
+ '[' false = true ']'
+ peer lifecycle chaincode package basic.tar.gz --path ../asset-transfer-basic/chaincode-javascript --lang node --label basic_1.0
+ res=0
Chaincode is packaged
Installing chaincode on peer0.org1...
Using organization 1
```

```
Checking the commit readiness of the chaincode definition successful on peer0.org2 on channel 'mychannel'
Using organization 1
Using organization 2
+ peer lifecycle chaincode commit -o localhost:7050 --ordererTLSHostnameOverride orderer.example.com --tls --cafile /home/g2022_anuprita_mhapankar/fabric-samples/test-network/organizations/ordererOrganizations/example.com/tlsca/tlsca.example.com-cert.pem --channelID mychannel --name basic --peerAddresses localhost:7051 --tlsRootCertFiles /home/g2022_anuprita_mhapankar/fabric-samples/test-network/organizations/peerOrganizations/org1.example.com/tlsca/tlsca.org1.example.com-cert.pem --peerAddresses localhost:9051 --tlsRootCertFiles /home/g2022_anuprita_mhapankar/fabric-samples/test-network/organizations/peerOrganizations/org2.example.com/tlsca/tlsca.org2.example.com-cert.pem --version 1
sequence 1
+ res=0
2026-04-09 15:02:00.641 UTC 0001 INFO [chaincodeCmd] ClientWait -> txid [d578fad7d5ea9fbbab1c00b0f3bb78e4c57add2a11c8387b9bed0a90efd9559c] committed with status (VALID)
localhost:7051
2026-04-09 15:02:00.646 UTC 0002 INFO [chaincodeCmd] ClientWait -> txid [d578fad7d5ea9fbbab1c00b0f3bb78e4c57add2a11c8387b9bed0a90efd9559c] committed with status (VALID)
localhost:9051
Chaincode definition committed on channel 'mychannel'
Using organization 1
Querying chaincode definition on peer0.org1 on channel 'mychannel'...
Attempting to query committed status on peer0.org1, Retry after 3 seconds.
+ peer lifecycle chaincode querycommitted --channelID mychannel --name basic
+ res=0
Committed chaincode definition for chaincode 'basic' on channel 'mychannel':
Version: 1.0, Sequence: 1, Endorsement Plugin: escc, Validation Plugin: vscc, Approvals: [Org1MSP: true, Org2MSP: true]
Query chaincode definition successful on peer0.org1 on channel 'mychannel'
Using organization 2
Querying chaincode definition on peer0.org2 on channel 'mychannel'...
Attempting to query committed status on peer0.org2, Retry after 3 seconds.
+ peer lifecycle chaincode querycommitted --channelID mychannel --name basic
+ res=0
Committed chaincode definition for chaincode 'basic' on channel 'mychannel':
Version: 1.0, Sequence: 1, Endorsement Plugin: escc, Validation Plugin: vscc, Approvals: [Org1MSP: true, Org2MSP: true]
Query chaincode definition successful on peer0.org2 on channel 'mychannel'
Chaincode initialization is not required
g2022_anuprita_mhapankar@cloudshell:~/fabric-samples/test-network$
```

Step 10: Set Peer Environment Variables

```
export
```

```
FABRIC_CFG_PATH=$HOME/fabric-samples/config
```

```
export PATH=$PATH:$HOME/fabric-samples/bin
```

```
export CORE_PEER_TLS_ENABLED=true
```

```
export CORE_PEER_LOCALMSPID="Org1MSP"
```

```
export
```

```
CORE_PEER_TLS_ROOTCERT_FILE=$HOME/fabric-samples/test-network/organizations/peerOrganizations/org1.example.com/peers/peer0.org1.example.com/tls/ca.crt
```



```
export
CORE_PEER_MSPCONFIGPATH=$HOME/fabric-samples/test-network/organizations/
peerOrganizations/org1.example.com/users/Admin@org1.example.com/msp
export CORE_PEER_ADDRESS=localhost:7051
```

```
g2022_anuprita_mhapankar@cloudshell:~/fabric-samples/test-network$ export FABRIC_CFG_PATH=$HOME/fabric-samples/config
export PATH=$PATH:$HOME/fabric-samples/bin
export CORE_PEER_TLS_ENABLED=true
export CORE_PEER_LOCALMSPID="Org1MSP"
export CORE_PEER_TLS_ROOTCERT_FILE=$HOME/fabric-samples/test-network/organizations/peerOrganizations/org1.example.com/peers/peer0.org1.example.com/tls/ca.crt
export CORE_PEER_MSPCONFIGPATH=$HOME/fabric-samples/test-network/organizations/peerOrganizations/org1.example.com/users/Admin@org1.example.com/msp
export CORE_PEER_ADDRESS=localhost:7051
g2022_anuprita_mhapankar@cloudshell:~/fabric-samples/test-network$
```

Step 11: Initialize Ledger

```
peer chaincode invoke \
  -o localhost:7050 \
  --ordererTLSHostnameOverride orderer.example.com \
  --tls \
  --cafile \
  $HOME/fabric-samples/test-network/organizations/ordererOrganizations/example.com/orderers/orderer.example.com/msp/tlscacerts/tlsca.example.com-cert.pem \
  -C mychannel \
  -n basic \
  --peerAddresses localhost:7051 \
  --tlsRootCertFiles \
  $HOME/fabric-samples/test-network/organizations/peerOrganizations/org1.example.com/peers/peer0.org1.example.com/tls/ca.crt \
  --peerAddresses localhost:9051 \
  --tlsRootCertFiles \
  $HOME/fabric-samples/test-network/organizations/peerOrganizations/org2.example.com/peers/peer0.org2.example.com/tls/ca.crt \
  -c '{"function": "InitLedger", "Args": []}'
```

```
g2022_anuprita_mhapankar@cloudshell:~/fabric-samples/test-network$ peer chaincode invoke \
-o localhost:7050 \
--ordererTLSHostnameOverride orderer.example.com \
--tls \
--cafile $HOME/fabric-samples/test-network/organizations/ordererOrganizations/example.com/orderers/orderer.example.com/msp/tlscacerts/tlsca.example.com-cert.pem \
-C mychannel \
-n basic \
--peerAddresses localhost:7051 \
--tlsRootCertFiles $HOME/fabric-samples/test-network/organizations/peerOrganizations/org1.example.com/peers/peer0.org1.example.com/tls/ca.crt \
--peerAddresses localhost:9051 \
--tlsRootCertFiles $HOME/fabric-samples/test-network/organizations/peerOrganizations/org2.example.com/peers/peer0.org2.example.com/tls/ca.crt \
-c '{"function": "InitLedger", "Args": []}'
g2022_anuprita_mhapankar@cloudshell:~/fabric-samples/test-network$
```

Step 12: Query All Assets

```
peer chaincode query -C mychannel -n basic -c '{"Args": ["GetAllAssets"]}'
```

```
g2022_anuprita_mhapankar@cloudshell:~/fabric-samples/test-network$ peer chaincode query -C mychannel -n basic -c '{"Args": ["GetAllAssets"]}'
[{"AppraisedValue":300,"Color":"blue","ID":"asset1","Owner":"Tomoko","Size":5,"docType":"asset"}, {"AppraisedValue":400,"Color":"red","ID":"asset2","Owner":"Brad","Size":5,"docType":"asset"}, {"AppraisedValue":500,"Color":"green","ID":"asset3","Owner":"Jin Soo","Size":10,"docType":"asset"}, {"AppraisedValue":600,"Color":"yellow","ID":"asset4","Owner":"Max","Size":10,"docType":"asset"}, {"AppraisedValue":700,"Color":"black","ID":"asset5","Owner":"Adriana","Size":15,"docType":"asset"}, {"AppraisedValue":800,"Color":"white","ID":"asset6","Owner":"Michel","Size":15,"docType":"asset"}]
g2022_anuprita_mhapankar@cloudshell:~/fabric-samples/test-network$
```

Step 13: Read a Specific Asset

```
peer chaincode query -C mychannel -n basic -c
 '{"Args": ["ReadAsset", "asset5"]}'
```

```
g2022_anuprita_mhapankar@cloudshell:~/fabric-samples/test-network$ peer chaincode query -C mychannel -n basic -c '{"Args": ["ReadAsset", "asset5"]}'
{"AppraisedValue":700,"Color":"black","ID":"asset5","Owner":"Adriana","Size":15,"docType":"asset"}
g2022_anuprita_mhapankar@cloudshell:~/fabric-samples/test-network$
```

Step 14: Create a New Asset

```
peer chaincode invoke \  
  -o localhost:7050 \  
  --ordererTLSHostnameOverride orderer.example.com \  
  --tls \  
  --cafile \  
$HOME/fabric-samples/test-network/organizations/ordererOrganizations/exa  
mp  
le.com/orderers/orderer.example.com/msp/tlscacerts/tlsca.example.com-cer  
t.pem \  
  -C mychannel \  
  -n basic \  
  --peerAddresses localhost:7051 \  
  --tlsRootCertFiles  
$HOME/fabric-samples/test-network/organizations/peerOrganizations/org1.e  
xa mple.com/peers/peer0.org1.example.com/tls/ca.crt \  
  --peerAddresses localhost:9051 \  
  --tlsRootCertFiles  
$HOME/fabric-samples/test-network/organizations/peerOrganizations/org2.e  
xa mple.com/peers/peer0.org2.example.com/tls/ca.crt \  
  -C  
'{"function": "CreateAsset", "Args": ["asset7", "blue", "10", "Anuprita", "500"]  
}'
```

```
g2022_anuprita_mhapankar@cloudshell:~/fabric-samples/test-network$ peer chaincode invoke \  
-o localhost:7050 \  
--ordererTLSHostnameOverride orderer.example.com \  
--tls \  
--cafile $HOME/fabric-samples/test-network/organizations/ordererOrganizations/example.com/orderers/orderer.example.com/msp/tlscacerts/tlsca.example.com-cert.pem \  
-C mychannel \  
-n basic \  
--peerAddresses localhost:7051 \  
--tlsRootCertFiles $HOME/fabric-samples/test-network/organizations/peerOrganizations/org1.example.com/peers/peer0.org1.example.com/tls/ca.crt \  
--peerAddresses localhost:9051 \  
--tlsRootCertFiles $HOME/fabric-samples/test-network/organizations/peerOrganizations/org2.example.com/peers/peer0.org2.example.com/tls/ca.crt \  
-c '{"function": "CreateAsset", "Args": ["asset7", "blue", "10", "Anuprita", "500"]}'  
2024-04-04 15:09:15.874 UTC [chaincodeCmd] chaincodeInvokeOrQuery -> Chaincode invoke successful. result: status:200 payload:"{\"ID\":\"asset7\",\"Color\":\"blue\",  
\"Size\":10,\"Owner\":\"Anuprita\",\"AppraisedValue\":500}"  
g2022_anuprita_mhapankar@cloudshell:~/fabric-samples/test-network$
```

Step 15: Transfer Asset

```
peer chaincode invoke \  
  -o localhost:7050 \  
  --ordererTLSHostnameOverride orderer.example.com \  
  --tls \  
  --cafile  
$HOME/fabric-samples/test-network/organizations/ordererOrganizations/exa  
mp  
le.com/orderers/orderer.example.com/msp/tlscacerts/tlsca.example.com-cer  
t.pem \  
  -C mychannel \  
  -n basic \  
  --peerAddresses localhost:7051 \  
  --tlsRootCertFiles  
$HOME/fabric-samples/test-network/organizations/peerOrganizations/org1.e  
xa mple.com/peers/peer0.org1.example.com/tls/ca.crt \  
  --peerAddresses localhost:9051  
$HOME/fabric-samples/test-network/organizations/peerOrganizations/org2.e  
xa mple.com/peers/peer0.org2.example.com/tls/ca.crt
```

```
--peerAddresses localhost:9051 \
--tlsRootCertFiles
$HOME/fabric-samples/test-network/organizations/peerOrganizations/org2.e
xample.com/peers/peer0.org2.example.com/tls/ca.crt \
-c '{"function":"TransferAsset","Args":["asset7","Sneha"]}'
```

```
q2022_anuprita_mhapankar@cloudshell:~/fabric-samples/test-network$ peer chaincode invoke \
-o localhost:7050 \
--ordererTLSHostnameOverride orderer.example.com \
--tls \
--cafile $HOME/fabric-samples/test-network/organizations/ordererOrganizations/example.com/orderers/orderer.example.com/msp/tlscacerts/tlsca.example.com-cert.pem \
-C mychannel \
-n basic \
--peerAddresses localhost:7051 \
--tlsRootCertFiles $HOME/fabric-samples/test-network/organizations/peerOrganizations/org1.example.com/peers/peer0.org1.example.com/tls/ca.crt \
--peerAddresses localhost:9051 \
--tlsRootCertFiles $HOME/fabric-samples/test-network/organizations/peerOrganizations/org2.example.com/peers/peer0.org2.example.com/tls/ca.crt \
-c '{"function":"TransferAsset","Args":["asset7","Sneha"]}'
2026-04-09 15:06:55.967 UTC [chaincodeCmd] chaincodeInvokeOrQuery -> Chaincode invoke successful. result: status:200 payload:"Anuprita"
q2022_anuprita_mhapankar@cloudshell:~/fabric-samples/test-network$
```

Step 16: Stop the Network

```
cd $HOME/fabric-samples/test-network
./network.sh down
```

```
q2022_anuprita_mhapankar@cloudshell:~/fabric-samples/test-network$ cd $HOME/fabric-samples/test-network
./network.sh down
Using docker and docker-compose
Stopping network.
project name can't be empty. Use '--project-name' to set a valid name
Error response from daemon: get docker_orderer.example.com: no such volume
Error response from daemon: get docker_peer0.org1.example.com: no such volume
Error response from daemon: get docker_peer0.org2.example.com: no such volume
Removing remaining containers
c72f6a9969f0
a811e27601bb
3f5198e286ca
3bcfb908afeb
8d2fc9e74375
Removing generated chaincode docker images
Untagged: dev-peer0.org2.example.com-basic_1.0-1d4d445573112813889f87c307bc2b5fbc664c87b24c035bee15cbcc7f59b629-c68f8ecc2a0ff0cc1497615f5e33a8bfa908e0fd3555f864cfa1398b52d341c
b:latest
Deleted: sha256:fa120af33c4db851b5dded871d445c39e7e2567eb3d753fcccfa850a32284087
Untagged: dev-peer0.org1.example.com-basic_1.0-1d4d445573112813889f87c307bc2b5fbc664c87b24c035bee15cbcc7f59b629-d3cf9b1f4b9747cc8433ac0e74bfaeade1cf0b97f5c1163835f604aa805d60d
c:latest
Deleted: sha256:edc7441a9a8c48b025dba4a79878631a900a26be317335ae9c9b2512c869f810
q2022_anuprita_mhapankar@cloudshell:~/fabric-samples/test-network$
```

CONCLUSION:

The Hyperledger Fabric network was successfully implemented and chaincode was deployed. Asset operations such as creation, querying, and transfer were performed successfully. This experiment demonstrated secure and efficient asset management using a permissioned blockchain system.



**VIVEKANAND EDUCATION SOCIETY'S
INSTITUTE OF TECHNOLOGY
Department of Information Technology
Academic Year: 2025-26**

ITC801: Blockchain and DLT Lab
Experiment 12

Aim: Mini Project

Roll No.	07
Name	Vedang V. Wajge
Class	D20A
Subject	ITC801: Blockchain and DLT Lab
CO Mapped	CO1:Describe the basic concept of Blockchain and Distributed Ledger Technology. CO2:Interpret the knowledge of the Bitcoin network, nodes, keys, wallets and transactions CO3:Implement smart contracts in Ethereum using different development frameworks. CO4:Develop applications in permissioned Hyperledger Fabric network. CO5:Interpret different Crypto assets and Crypto currencies CO6:Analyze the use of Blockchain with AI, IoT and Cyber Security using case studies.



**VIVEKANAND EDUCATION SOCIETY'S
INSTITUTE OF TECHNOLOGY
Department of Information Technology
Academic Year: 2025-26**

ITC801: Blockchain and DLT Lab
Experiment 13

Aim: Assignment 1

Roll No.	07
Name	Vedang V. Wajge
Class	D20A
Subject	ITC801: Blockchain and DLT Lab Question : Explore following 4 websites and answer the questions bitcoin.org Blockchain.com https://coinmarketcap.com/ https://www.investopedia.com/
CO Mapped	CO1,CO2

Hard Copy Submitted